

# Robofun Training

Robotics for University 2024

Updated on: June 2024

# Overview

- Prerequisite
- Trainings:
  - Mapping
  - Localization and Navigation
  - Object Detection
  - Octomap Generation
  - 3D Mapping
- Node-RED
- ROSBoard

| Day                                                         | Agenda                                                                                              |
|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Day 1:<br>Introduction to Autonomous Mobile Robot           | 1.1 AMR,<br>Scuttle introduction and HW Setup<br>1.2 ROS Introduction and SW Setup and Installation |
| Day 2:<br>Familiarising with AMR SW components and features | 2.1 Sensors introduction and Mapping of AMR<br>2.2 Localisation and Navigation of AMR               |
| Day 3:<br>Use case examples                                 | 3.1 Object Detection with AMR<br>3.2 Fast Mapping<br>3.3 Rtabmap                                    |
| Day 4:<br>Visualisation                                     | 4.1 Node red dashboard<br>4.2 Rosboard                                                              |



# Prerequisite Setup

# Requirement Before Training

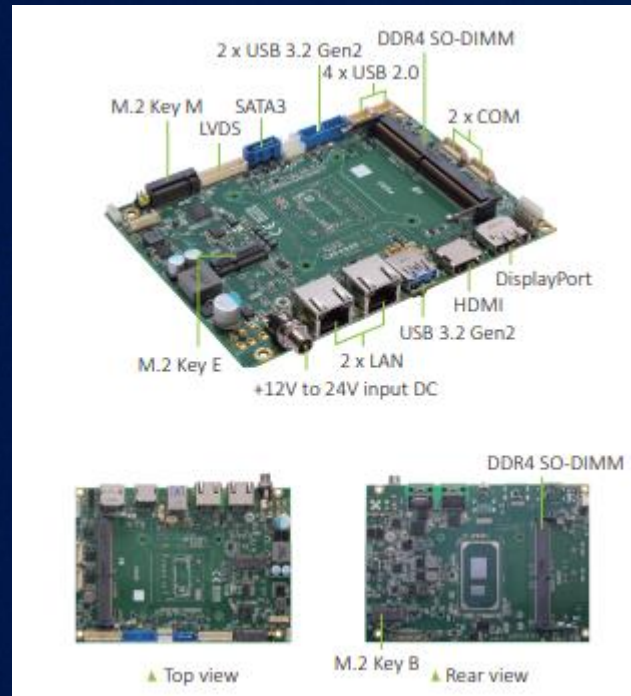
|                                                                                                                                                                                                                        | Source       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| Assembled robot with Intel® Platform                                                                                                                                                                                   |              |
| Internet Connection                                                                                                                                                                                                    |              |
| AMR power supply <ul style="list-style-type: none"><li>Fully Charged Battery (if you're using battery)</li><li>Connect to direct power brick</li></ul>                                                                 |              |
| Training Material <ul style="list-style-type: none"><li>Robofun-Training-Guide.pdf</li></ul>                                                                                                                           | Google Drive |
| Scuttle driver program <ul style="list-style-type: none"><li>robofun-1.0.zip</li></ul>                                                                                                                                 | Google Drive |
| Text file for commands/codes use in training <ul style="list-style-type: none"><li>day1.txt</li><li>day2.txt</li><li>day3.txt</li><li>day4.txt</li><li>image_publisher.txt</li><li>object_detection_node.txt</li></ul> | Google Drive |

Only continue to this section if  
you are using Axiomtek  
CAPA55R system



# Processor (Brain of the AMR)

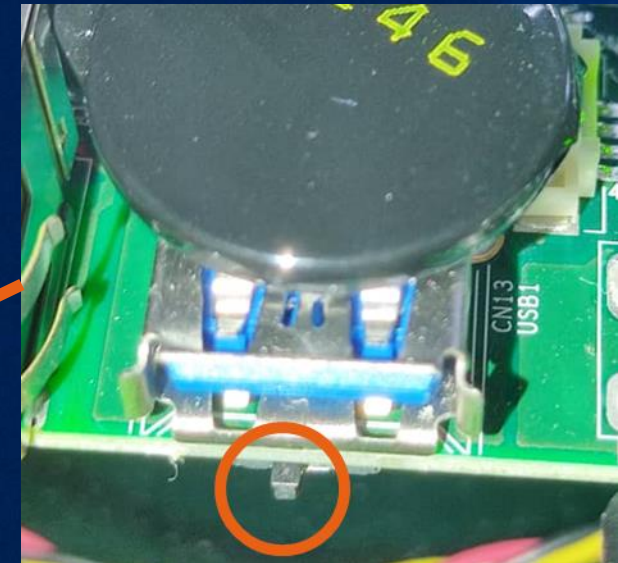
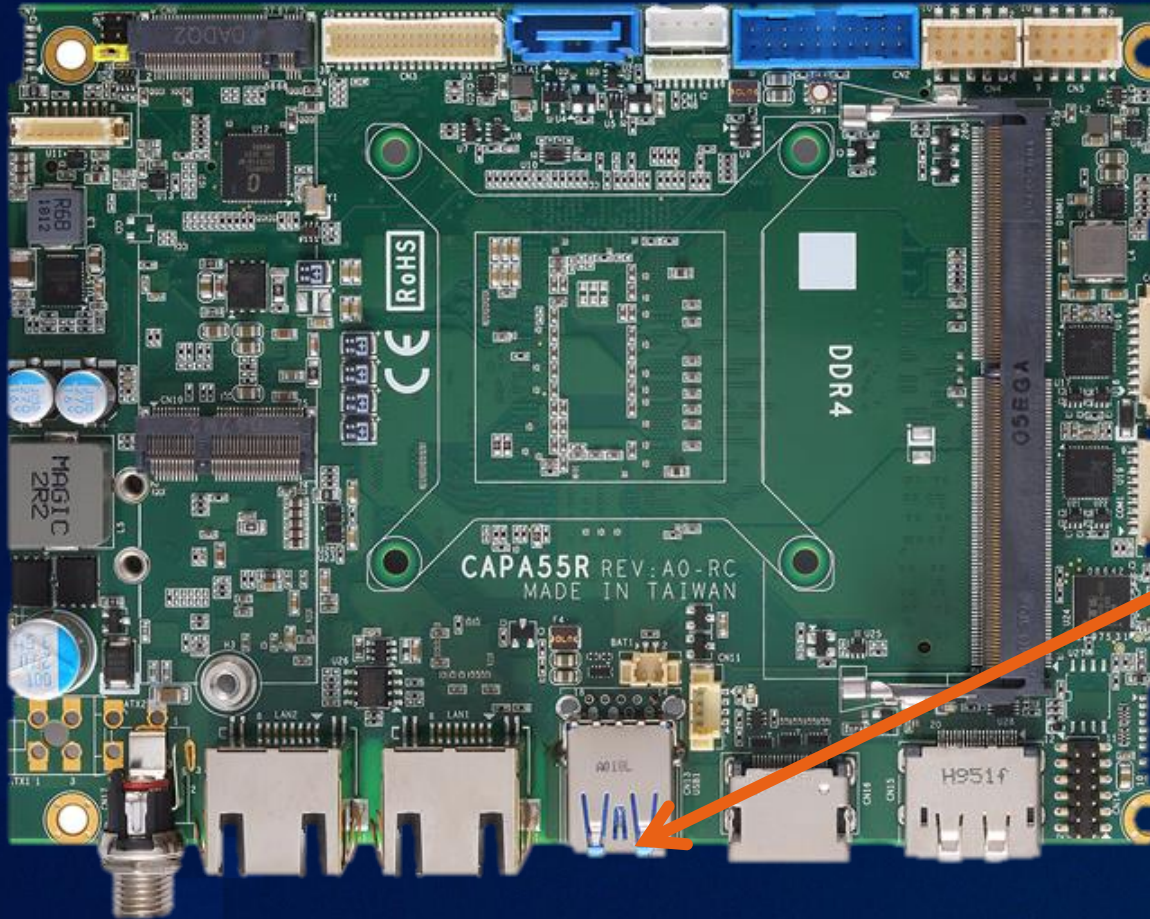
Install Ubuntu 22.04 LTS on the Axiomtek CAPA55R with embedded DP display off.



|              | Specifications                                        |
|--------------|-------------------------------------------------------|
| Processor    | 11 <sup>th</sup> Gen Intel® Core i7-1185G7E Processor |
| Graphics     | Intel® Xe Graphics                                    |
| Memory (RAM) | 32 GB                                                 |
| Disk         | 128 GB SSD                                            |

# AT Power toggle on CAPA55R

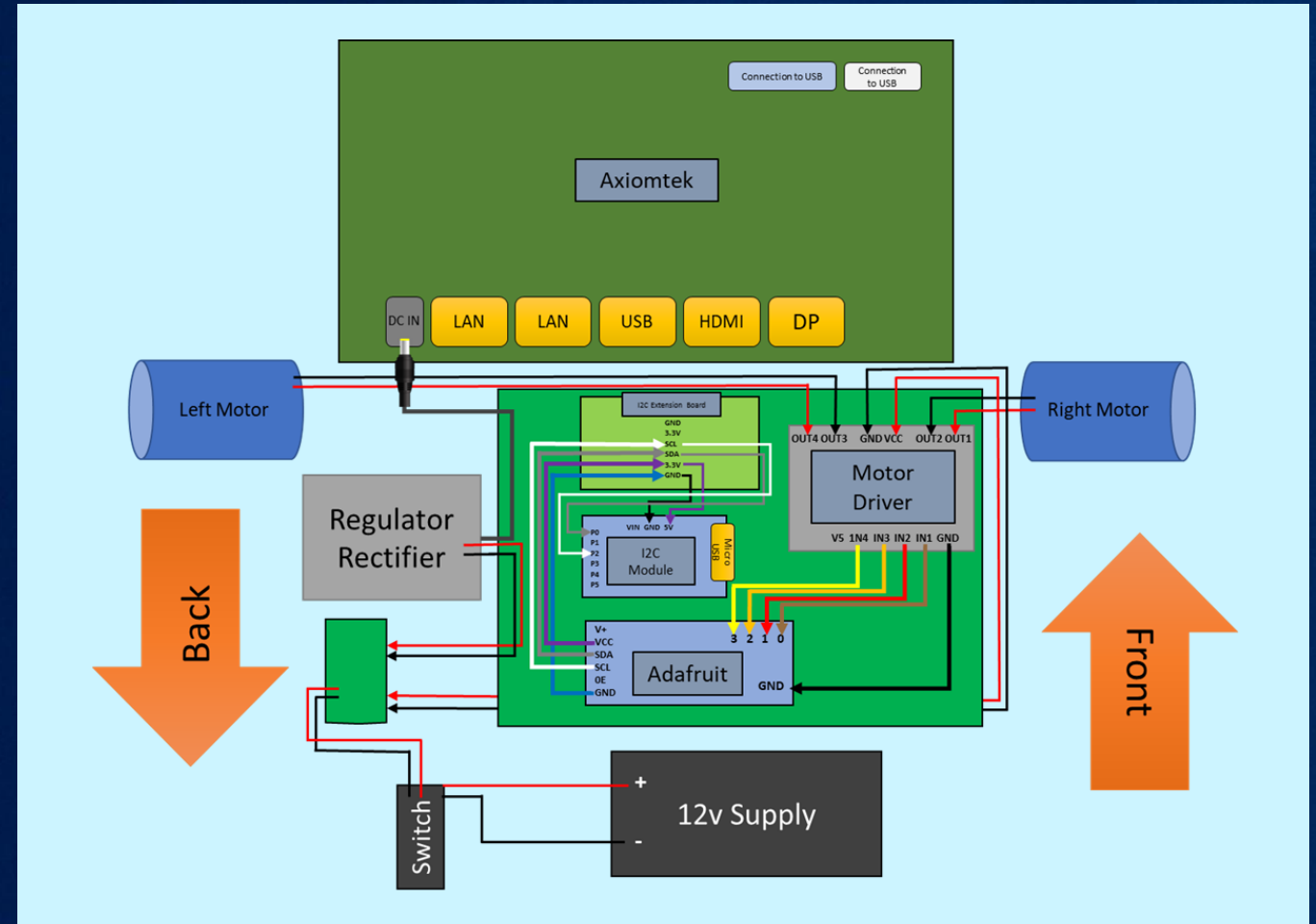
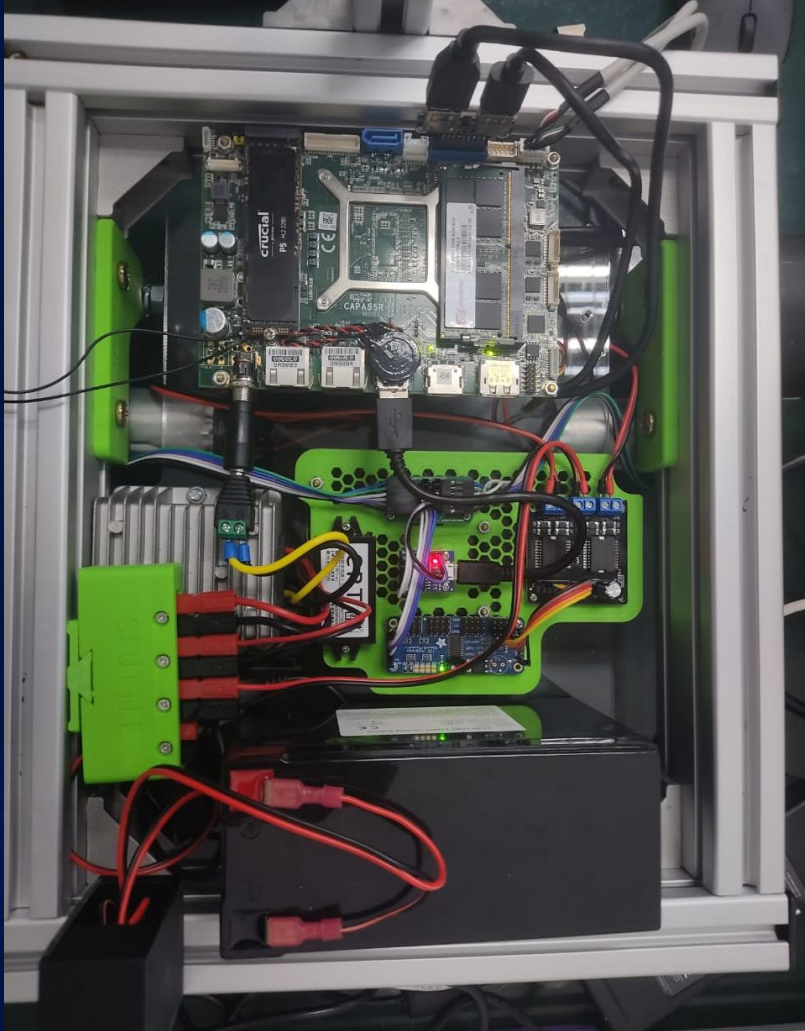
- A tiny toggle switch sits below the USB port, pressed it to turn on the system.



CAPA55R



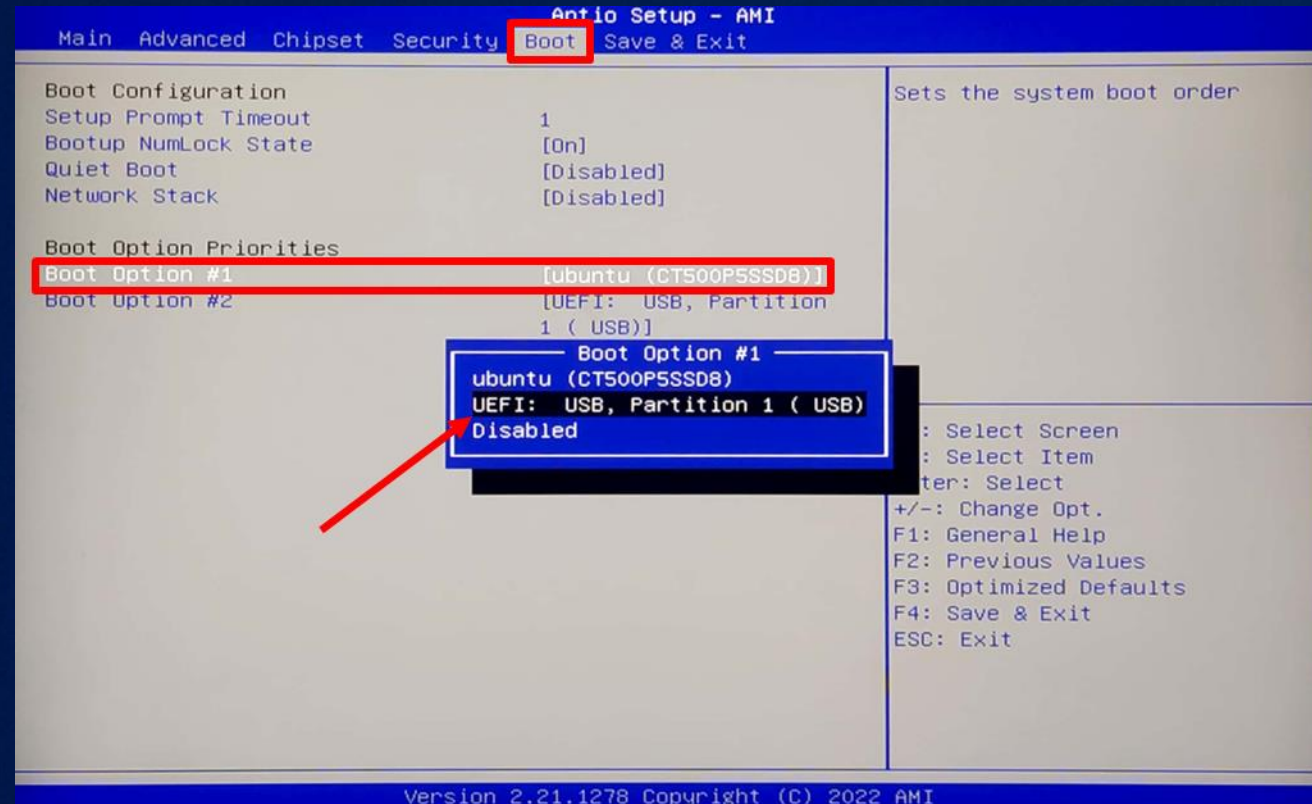
# The Connection Circuit's set





# Prerequisite

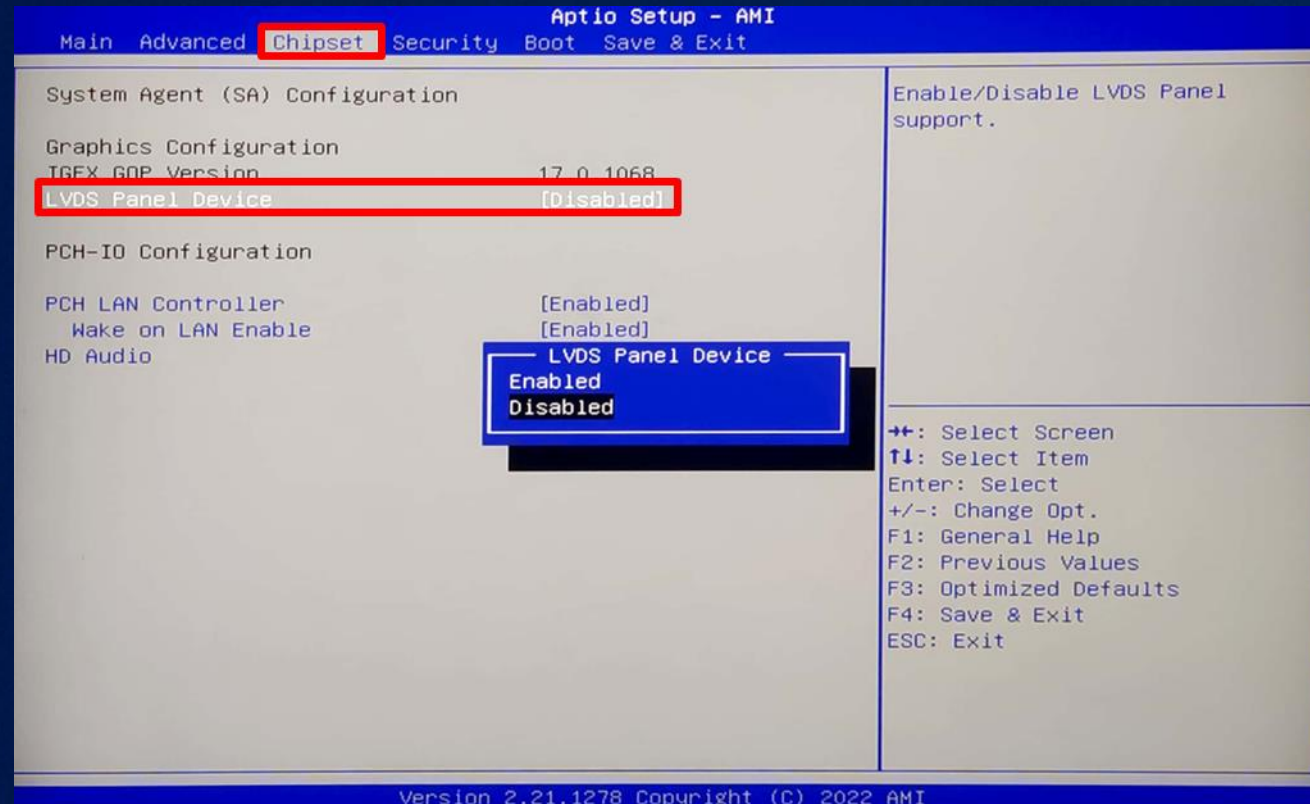
- As the system logo appear, promptly press DEL or F2 to access the BIOS.
- Choose "Boot" > Boot Option #1 > "UEFI: USB, "



# Prerequisite

Axiomtek has an embedded DP display that may cause the ubuntu installation menu to not show up on display. To resolve this, disable LVDS Panel Device in Chipset menu:

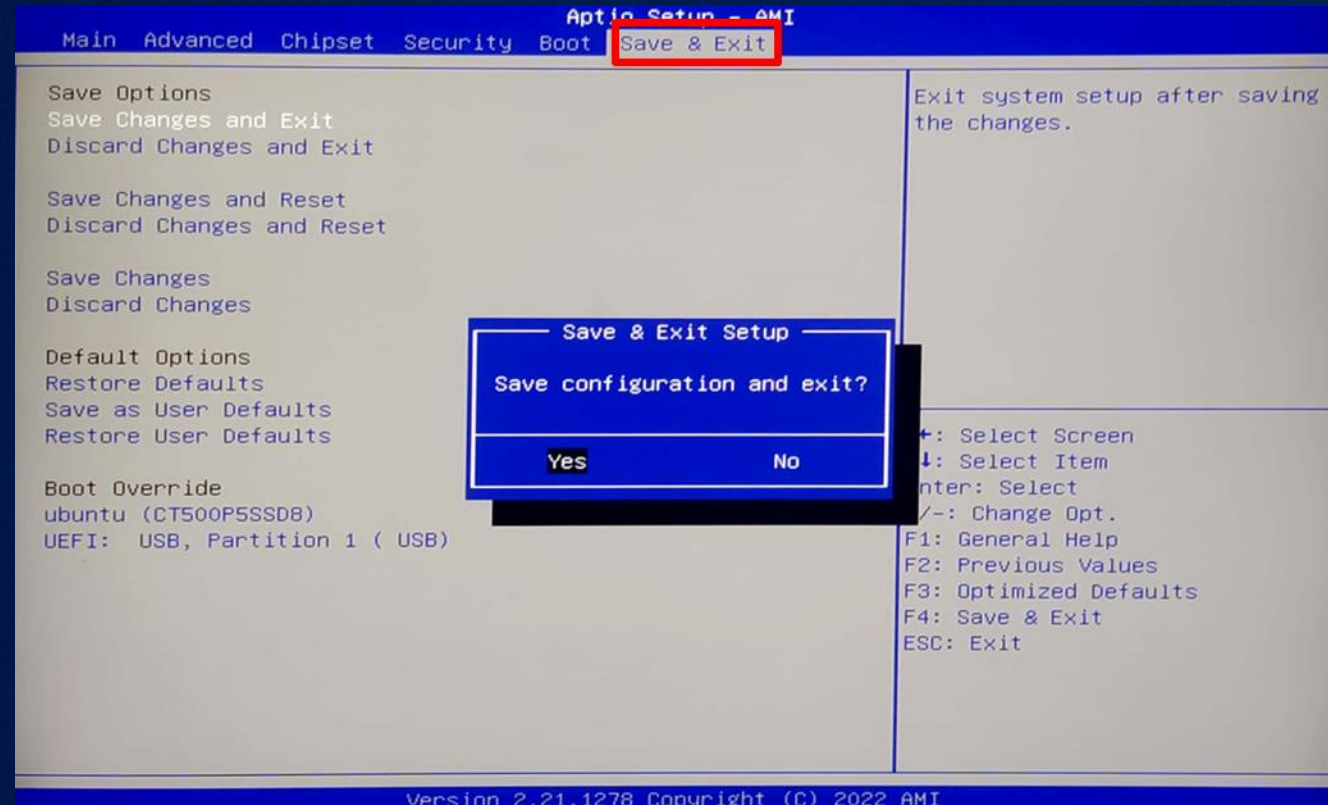
- Choose "Chipset" and disabled the LVDS Panel Device.





# Prerequisite

- Go to “Save & Exit” and save the configuration.



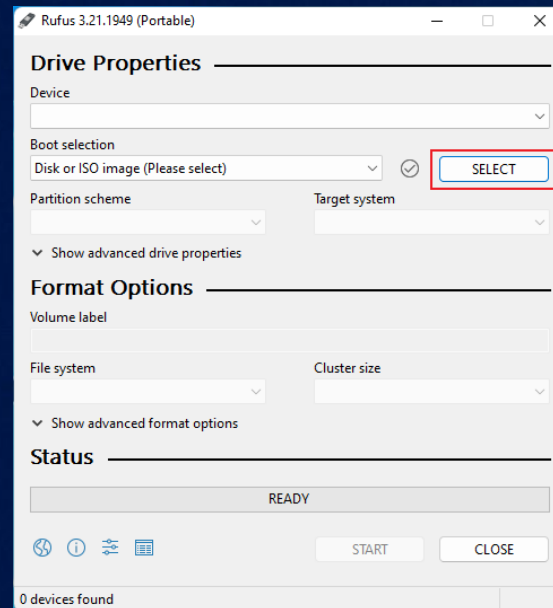
# Link

1. Axiomtek CAPA55R:  
<https://www.axiomtek.com/Default.aspx?MenuId=Products&FunctionId=ProductView&ItemId=26529&C=CAPA55R&upcat=270>

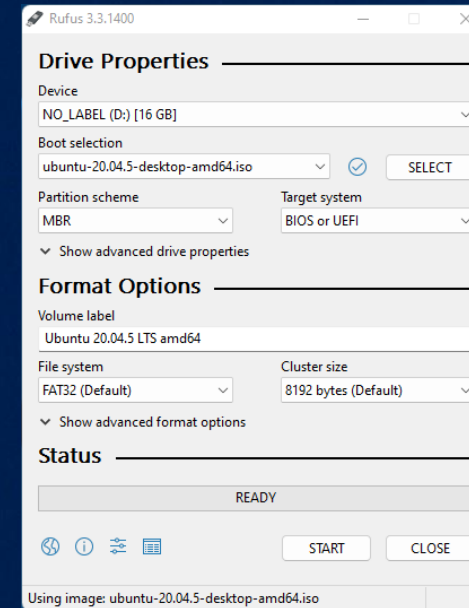


# Install Ubuntu 22.04 LTS

- Download the Ubuntu ISO at: [link](#)
- Download Rufus: [link](#)
- Create Ubuntu 22.04 LTS bootable drive



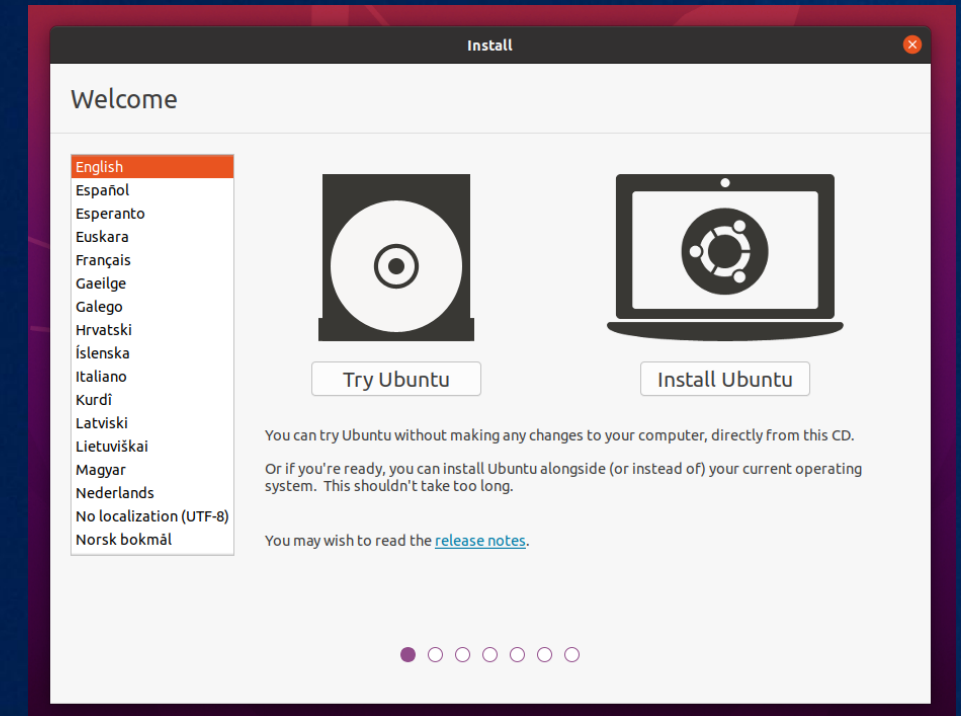
Select downloaded Ubuntu ISO file



Select Ubuntu 22.04 ISO and create bootable drive

# Prerequisite

- You will enter Ubuntu page, then:
  - i. Choose to install Ubuntu
  - ii. Choose keyboard layout (English) > continue
  - iii. Choose 'normal installation' > continue
  - iv. Choose 'erase disk and reinstall ubuntu'
  - v. On the map, choose 'Kuala Lumpur'
  - vi. Insert your computer name as "amrkit<your robot number>"
    - Example: amrkit017
  - vii. Continue
  - viii. Installation is done and restart, wait for instruction to remove the USB.



# Link

1. Linux Basic Commands: <https://www.digitalocean.com/community/tutorials/linux-commands>



# Prepare Target System (Robot)

# Install Kernel

- Install the kernel

```
$ sudo apt install linux-image-5.15.0-1049-intel-iotg linux-modules-extra-5.15.0-1049-intel-iotg
```

- Modify GRUB boot options to set the new kernel as the default:

```
$ sudo sed -i 's/GRUB_DEFAULT=.* /GRUB_DEFAULT="Advanced options for Ubuntu>Ubuntu, with Linux 5.15.0-1049-intel-iotg"/g' /etc/default/grub && sudo update-grub
```

- Reboot system

```
$ sudo reboot
```

# Install Kernel

- Run command to confirm kernel version

```
$ uname -r
```

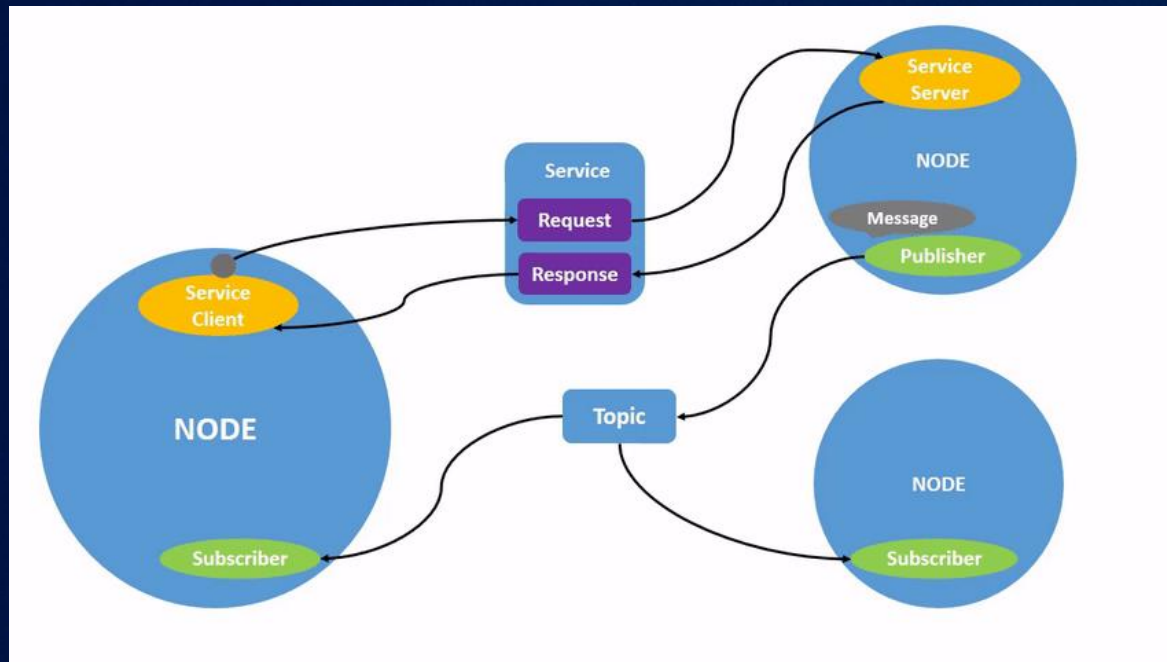
- Expected Output:

```
5.15.0-1049-intel-iotg
```



# Robotic Operating System (ROS2)

# ROS2 Basics - Node

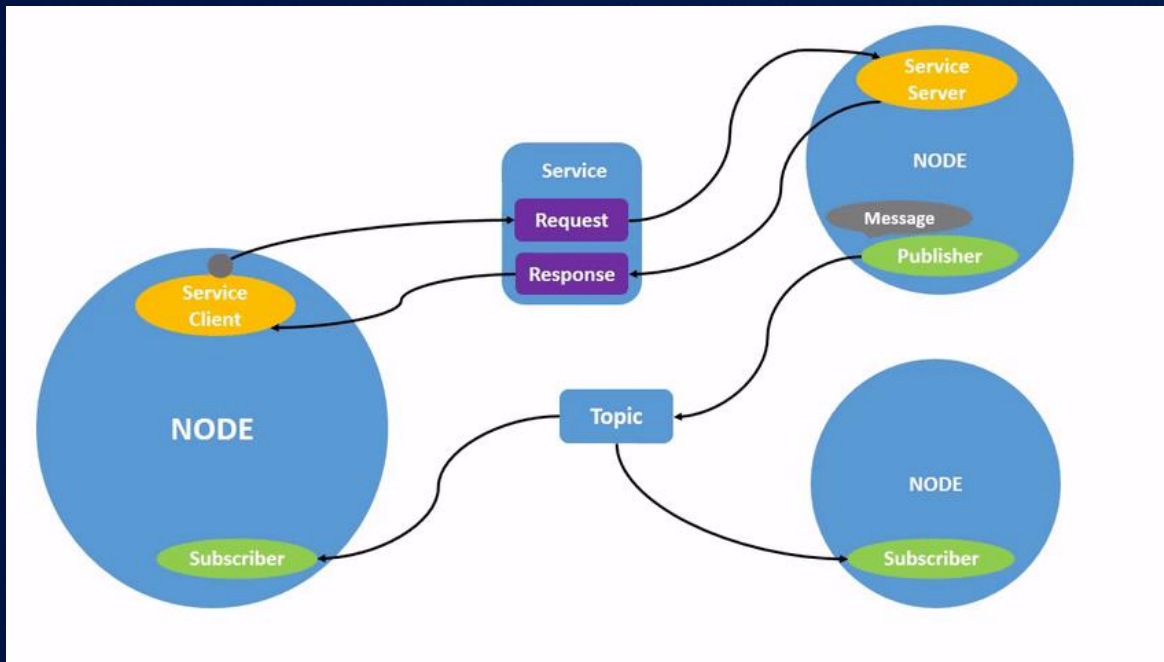


- Each node responsible for a single, module purpose
- Each node can send and receive data to other nodes via topics, services, actions or parameters
- Useful ROS2 commands to list, print info of the nodes running:
  - To list ROS2 nodes running:

```
$ ros2 node list
```
  - To print info of a specific running node:

```
$ ros2 node info <node_name>
```

# ROS2 Basics - Topic



- One of the primary ways in which data is moved between nodes and different parts of a ROS2 system is through topics. Topics serve as a communication pathway for nodes to send and receive messages, which can be one-to-many, many-to-one, or many-to-many depending on the system's needs.
- To send data over a topic, a node must act as a publisher, while to receive data, a node must act as a subscriber. Publishers and subscribers must agree on the message type being sent and received, otherwise communication will not be possible.
- Some useful ROS2 commands for working with topics include:
  - Displays a list of active topics in the ROS2 system  

```
$ ros2 topic list
```
  - Prints detailed information about a specific topic  

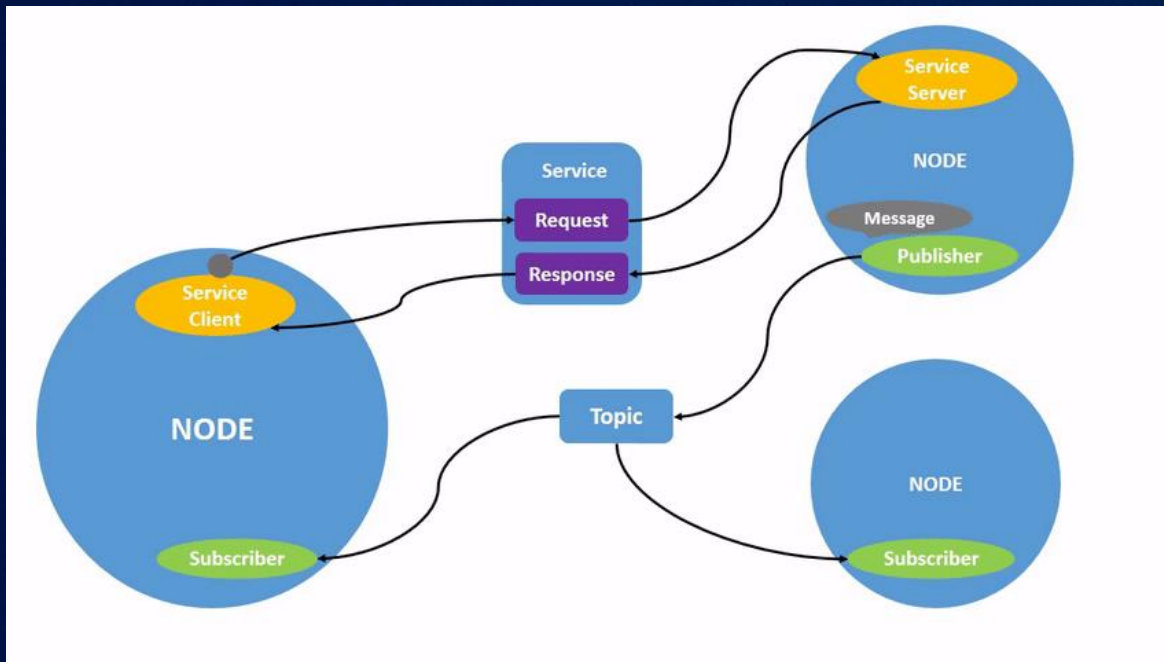
```
$ ros2 topic info <topic_name>
```
  - Calculates and prints the publishing frequency (in Hz)  

```
$ ros2 topic hz <topic_name>
```
  - Prints the data being published on a topic in real-time, useful for debugging and testing purposes  

```
$ ros2 topic echo <topic_name>
```



# ROS2 Basics - Services



- ROS2 services are a way for nodes to communicate with each other by making requests and receiving responses. They are useful for implementing functionality that requires a one-time or intermittent interaction between nodes.
- The following are the key concepts to understand when working with ROS2 services:
  - Service Definition: A service definition is a message type that specifies the request and response types for the service. It is defined in a .srv file and must be compiled before it can be used.
  - Service Client: A service client is a node that makes requests to a service. It sends a request message to the service and waits for a response message.
  - Service Server: A service server is a node that provides a service. It receives request messages from clients, processes them, and sends response messages back to the clients.
  - Service Name: A service name is a unique identifier for a service. It is composed of two parts: the name of the package that contains the service definition and the name of the service.
- Useful command for the ROS2 services:
  - To list available ROS2 service:

```
$ ros2 service list
```

# ROS2 Basics – Compile & Run a ROS2 applications

- Create a ROS2 Workspace: Once ROS2 is installed, you need to create a workspace to develop your ROS2 application. You can use the ros2 command-line tool to create a new workspace:
  - Command to create a new workspace and compile a ROS2 applications

```
$ mkdir -p ~/ros2_ws/src  
$ cd ~/ros2_ws/  
$ colcon build --symlink-install
```

- To run a ROS2 program after compile a program:
  - Source the workspace environment:

```
$ cd ~/ros2_ws/  
$ source install/setup.bash
```

- Run the ROS2 node

```
$ ros2 run <package_name> <node_name>
```

- Example:

```
$ ros2 run demo_nodes_cpp talker
```

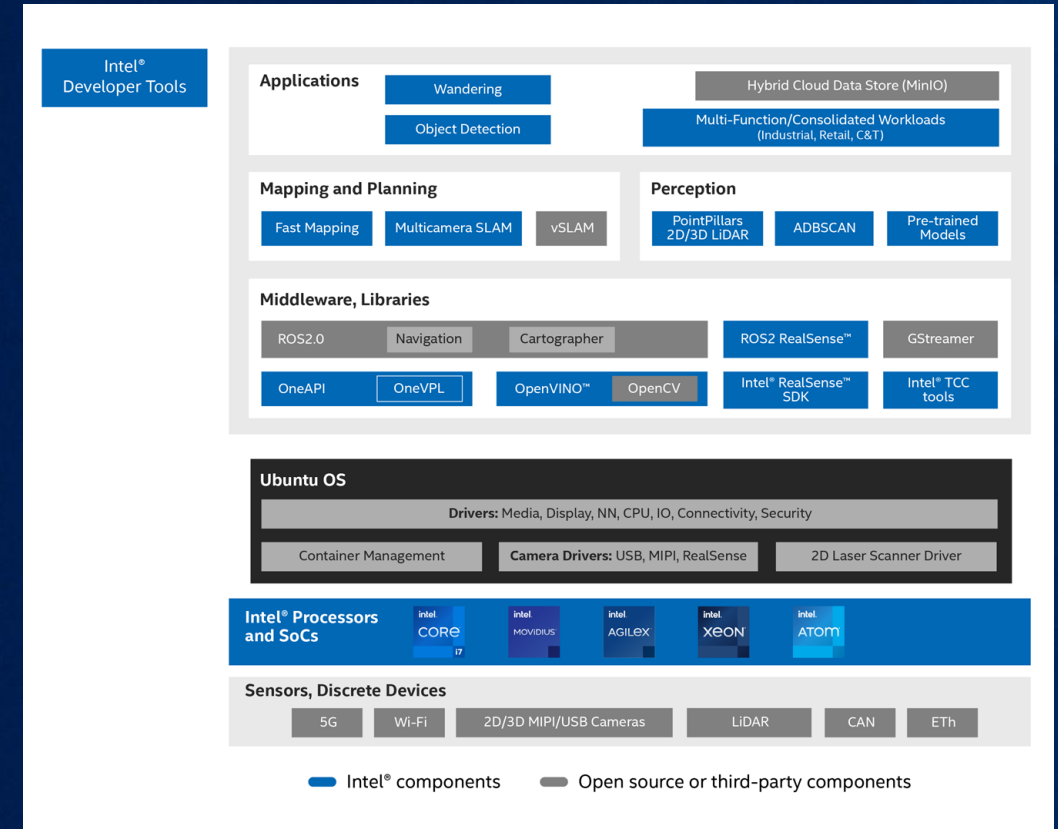
# Link

1. ROS2: <https://docs.ros.org/en/humble/index.html>



# Edge Insights for Autonomous Mobile Robots (EI for AMR)

- Develop, build, and deploy end-to-end mobile robot applications with this purpose-built, open, and modular software development kit that includes libraries, middleware, and sample applications based on the open-source Robot Operating System 2\* (ROS 2).
- Advantages:
  - Enables code to be implemented once and deployed to multiple hardware configurations.
  - Accelerates deployment of customer ROS 2-based applications by reducing evaluation and development time.
  - Provides prevalidated, scalable EI for AMR platforms through development partners.
- Featured Components:
  - Intel® Distribution of OpenVINO™ Toolkit
  - Intel® oneAPI Base Toolkit
  - Intel® RealSense™ SDK 2.1
  - AMR Algorithms
  - ROS 2 Sample Applications



# Download and install Robotics SDK 2.1

# Download and install Robotics SDK 2.1

- Link to install Robotics SDK2.1: [link](#)
- Register your account for Standard access.
- Fill in your information

## Register

Sign up to get the most out of the Intel® Developer Zone.

[> Register now for Standard access](#)

With a **Standard account**, you can:

- › Download development tools and code
- › Post questions to the developer support community
- › Bookmark content to receive notifications about updates

Registration is simple and immediate.

## Contact Information

|                          |                          |
|--------------------------|--------------------------|
| <input type="text"/>     | <input type="text"/>     |
| <input type="text"/>     | <input type="text"/>     |
| <input type="password"/> | <input type="password"/> |
| <input type="text"/>     | <input type="text"/>     |
| <input type="text"/>     | <input type="text"/>     |

Next Step



# Download and install Robotics SDK 2.1

- Submit
- Sign into your account

## Communication Subscriptions

Subscribe to optional email updates from Intel

☐

Select all subscriptions below



Developer Zone Newsletter



Edge Software Hub Product Communication



Programmable Logic Product Announcements



Programmable Logic Newsletters



Software Developer Product Insights



Yes, I would like to subscribe to stay connected to the latest Intel technologies and industry trends by email and telephone. I can unsubscribe at any time.

Next Step

# Download and install Robotics SDK 2.1

- Once you have sign in, choose the configuration as below and click on Review License Agreement.
- Go to the link displayed after clicking the Review License Agreement

## Robotics SDK

Robotics SDK 2.1 (Edge Insights for Autonomous Mobile Robots) offers Debian based software packages and pre-validated hardware modules for sensor data ingestion, classification, environment modelling, action planning, and



Select options below to download

Version ⓘ

Robotics SDK 2.1

Robotics SDK 2.0

Target System ⓘ

Ubuntu 22.04 LTS

Please review and accept the license agreement before proceeding

[Learn More](#)

[Documentation](#)

[Recommended Hardware](#)

Review License Agreement

# Download and install Robotics SDK 2.1

- Follow instructions provided in the [link](#) to setup Robotics SDK APT Repositories

## Setup the Robotics SDK APT Repositories

A target running a compatible OS can install Deb packages from hosted APT repositories.



Target System



# Download and install Robotics SDK 2.1

- Install Intel® Robotics SDK Deb packages

```
$ sudo apt update
```

```
$ sudo apt install ros-humble-robotics-sdk
```

- (Optional) to install complete package

```
$ sudo apt install ros-humble-robotics-sdk-complete
```

# Robot Software Setup

# Robot Setup

- Open a terminal and create a workspace directory

```
$ mkdir ~/workspace
```

- Copy the scuttle driver program from the google drive (robofun-1.0.zip) and put in **~/workspace**. This file contains every necessary components that needed to run the scuttle robot.

```
$ cd ~/workspace  
$ unzip robofun-1.0.zip
```

- Go to the robofun-1.0 directory:

```
$ cd ~/workspace/robofun-1.0/
```

- Install dependency:

```
$ chmod 777 install_dependencies.sh
```



# Robot Setup

- Please remove the "*--ignore-installed PyYAML*" line from 33 in `install_dependencies.sh`

```
$ gedit install_dependencies.sh
```

```
31 apt install -y build-essential python3-dev python3-pip
32 python3 -m pip install -q --upgrade --no-cache-dir pip > /dev/null
33 python3 -m pip install -q --upgrade --ignore-installed PyYAML --no-cache-dir -r requirements.txt > /dev/null
34 python3 -m pip install -q --upgrade --no-cache-dir -r requirements.txt > /dev/null
```

(Old Line 33)

```
31 apt install -y build-essential python3-dev python3-pip
32 python3 -m pip install -q --upgrade --no-cache-dir pip > /dev/null
33 python3 -m pip install -q --no-cache-dir -r requirements.txt > /dev/null
```

(New Line 33)

- Once edited, run the script

```
$ sudo ./install_dependencies.sh
```

# Robot Setup

- Reboot the system to update the registry

```
$ sudo reboot
```

- Open your terminal again and go to ../platform/scuttle directory and check the list if it contained the hardware-preq.sh file:

```
$ cd ~/workspace/robofun-1.0/platform/scuttle/  
$ ls
```

- Run the command inside:

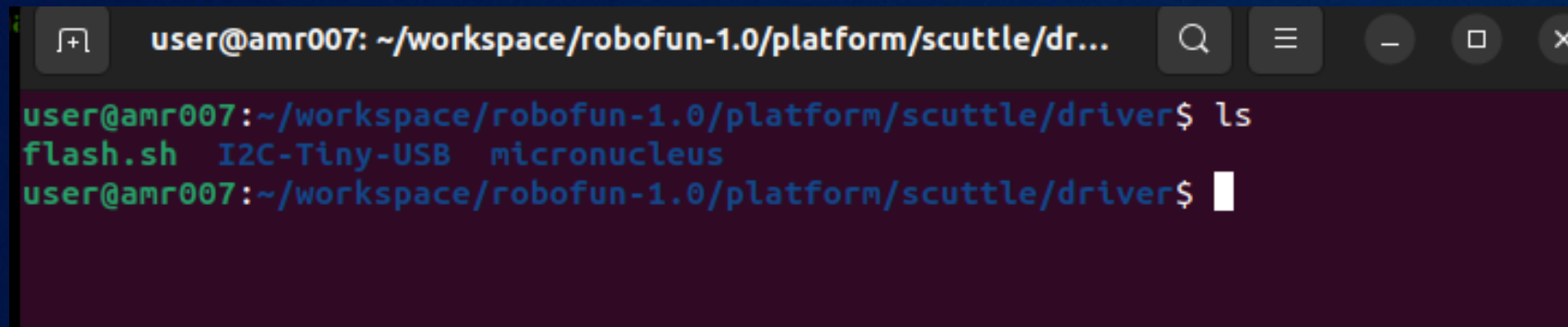
```
$ sudo bash ./hardware-preq.sh
```

```
[sudo] password for user:  
Setting up scuttle required udev rules  
user@amrkit047:~/workspace/robofun-1.0/platform/scuttle$
```

# Robot Setup

- Check the driver directory if it contains flash file:

```
$ cd ~/workspace/robofun-1.0/platform/scuttle/driver/  
$ ls
```



A terminal window titled "user@amr007: ~/workspace/robofun-1.0/platform/scuttle/dr..." with standard window controls. The terminal shows the following commands and output:

```
user@amr007:~/workspace/robofun-1.0/platform/scuttle/driver$ ls  
flash.sh I2C-Tiny-USB micronucleus  
user@amr007:~/workspace/robofun-1.0/platform/scuttle/driver$
```



# Robot Setup

- Install for flash:

```
$ sudo apt install libusb-dev gcc-avr avr-libc
$ chmod 777 flash.sh
$ sudo ./flash.sh
```

```
rm -f main.hex main.eep.hex
avr-objcopy -j .text -j .data -O ihex main.elf main.hex
avr-size -C --mcu=attiny85 main.elf
AVR Memory Usage
-----
Device: attiny85

Program:   3076 bytes (37.5% Full)
(.text + .data + .bootloader)

Data:      68 bytes (13.3% Full)
(.data + .bss + .noinit)

{status:"waiting",step:1,steps:6,progress:0.500000}
> Please plug in the device ...
> Press CTRL+C to terminate the program.
```

- Unplug and plug the i2c USB again

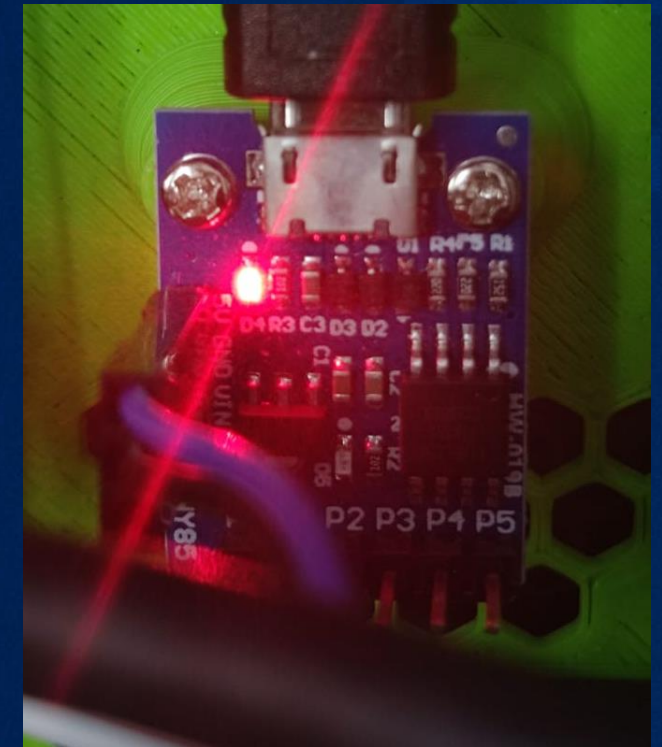


# Robot Setup

- Expected results after plug the USB again.

```
status:"writing",step:5,steps:6,progress:0.809049}  
status:"writing",step:5,steps:6,progress:0.819694}  
status:"writing",step:5,steps:6,progress:0.830339}  
status:"writing",step:5,steps:6,progress:0.840985}  
status:"writing",step:5,steps:6,progress:0.851630}  
status:"writing",step:5,steps:6,progress:0.862275}  
status:"writing",step:5,steps:6,progress:0.872921}  
status:"writing",step:5,steps:6,progress:0.883566}  
status:"writing",step:5,steps:6,progress:0.894212}  
status:"writing",step:5,steps:6,progress:0.904857}  
status:"writing",step:5,steps:6,progress:0.915502}  
status:"writing",step:5,steps:6,progress:0.926148}  
status:"writing",step:5,steps:6,progress:0.936793}  
status:"writing",step:5,steps:6,progress:0.947438}  
status:"writing",step:5,steps:6,progress:0.958084}  
status:"writing",step:5,steps:6,progress:0.968729}  
status:"writing",step:5,steps:6,progress:0.979375}  
status:"writing",step:5,steps:6,progress:0.990020}  
status:"writing",step:5,steps:6,progress:1.000000}  
Starting the user app ...  
status:"running",step:6,steps:6,progress:0.000000}  
status:"running",step:6,steps:6,progress:1.000000}  
> Micronucleus done. Thank you!  
user@amrkit047:~/workspace/robofun-1.0/platform/scuttle/driver$
```

- LED light up.





# Robot Setup

- Check the i2c bus:

```
$ sudo i2cdetect -l
```

```
[ 1.047895] i2c i2c-0: 1/1 memory slots populated (from DMI)
[ 1.048361] i2c i2c-0: Successfully instantiated SPD at 0x50
[ 1.507370] usb 3-1: Product: i2c-tiny-usb
[ 2.589432] i2c-tiny-usb 3-1:1.0: version 2.01 found at bus 003 address 002
[ 2.590879] i2c i2c-12: connected i2c-tiny-usb device
[ 2.590911] usbcore: registered new interface driver i2c-tiny-usb
```

- Run `sudo chmod 777 /dev/i2c-<i2c_bus_id>`

```
$ sudo chmod 777 /dev/i2c-12
```



# Robot Setup

- To check whether the encoders are connected correctly to i2c, run:

```
$ sudo i2cdetect -r -y <i2c_bus_id>
```

```
user@amrkit041:~$ sudo i2cdetect -r -y 12
      0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:      -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: 40 41 -- 43 -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- --
user@amrkit041:~$
```

*\* Make sure 40, 41 and 43 are available. Ignore the 70 if it is visible.*

# Robot Setup

- Go to standalone directory and set the docker-compose.yml:

```
$ cd ~/workspace/robofun-1.0/platform/scuttle/standalone
```

- Open the docker-compose.yml

```
$ gedit docker-compose.yml
```

# Robot Setup

- Set the docker-compose.yml
  - ROBOT\_BASE="scuttle"
  - ROBOT\_BASE\_TYPE="devkit"
  - ROBOT\_DOMAIN\_ID=<your\_robot\_number>
  - ROBOT\_NAMESPACE=<your\_robot\_name>
- Everyone will have a different robot namespace based on their robot's tag.



Ctrl + S to save.

```
Open  [icon]  ~/workspace/frameworks.robotics.mobile.d...k-1.0-robofun/platform/scuttle/stan...  Save  [icon]  [icon]  [icon]  [icon]
7  dockerfile: ./standalone/Dockerfile
8  args:
9    - ROBOT_VERSION=2023.1
10 container_name: amr.platform.scuttle
11 image: amr.platform.scuttle:2023.1
12 restart: "no"
13 ipc: host
14 pid: host
15 network_mode: "host"
16 privileged: true
17 volumes:
18   - /dev:/dev:rw
19 environment:
20   - ROBOT_BASE=scuttle
21   - ROBOT_BASE_TYPE=devkit
22   - ROS_DOMAIN_ID=17
23   - ROBOT_NAMESPACE=i017
24   - RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
25 command: "ros2 launch generic_robot_driver robot_bringup.launch.py"
26
27 amr-platform-scuttle-teleop:
28 container_name: amr.platform.scuttle.teleop
29 image: amr.platform.scuttle:2023.1
30 restart: "no"
31 ipc: host
32 pid: host
33 network_mode: "host"
34 privileged: true
35 volumes:
36   - /dev:/dev:rw
37 environment:
38   - ROBOT_BASE=scuttle
39   - ROBOT_BASE_TYPE="devkit"
40   - ROS_DOMAIN_ID=17
41   - ROBOT_NAMESPACE=i017
42   - RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
43 command: "ros2 launch generic_robot_driver robot_teleop.launch.py"

YAML  Tab Width: 8  Ln 1, Col 15  INS
```

Both amr-platform-scuttle and amr-platform-scuttle-teleop need to set



# Robot Setup

- Run the command to build docker-compose.yml package:

```
$ docker compose -f docker-compose.yml build
```

```
user@amrkit017:~/workspace/frameworks.robotics.nobile.distributed-at.amr-stack-1.0-robofun/platform/scuttle/standalone$ docker compose -f docker-compose.yml build
[+] Building 1057.0s (14/29)
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 3.54kB
=> [internal] load metadata for docker.io/library/ros:humble-ros-core
=> [internal] load build context
=> => transferring context: 53.14kB
=> [ 1/25] FROM docker.io/library/ros:humble-ros-core@sha256:b69e30779bca5958137a2fe63d3267d2b497d58038d7c5f5196a28214bfa0638
=> => resolve docker.io/library/ros:humble-ros-core@sha256:b69e30779bca5958137a2fe63d3267d2b497d58038d7c5f5196a28214bfa0638
=> sha256:10ac4908093d4325f2c94b2c9a571fa1071a17a72dd9c21c1ffb2c86f68ca028 30.43MB / 30.43MB
=> sha256:b69e30779bca5958137a2fe63d3267d2b497d58038d7c5f5196a28214bfa0638 549B / 549B
=> sha256:ba8e85191b19e61f3c24f06a4baffd5bba4849b11a6021d294f88d4adbd2ebf9 1.79kB / 1.79kB
=> sha256:5c80534496f6b5f76be21cd9803c18fb05ae946c67b959f4121c787956f88c55 4.01kB / 4.01kB
=> sha256:841d9fe8d0ff117e288a003adeabeb14357e33e4aeb9257fa966be44f649e8e 1.17MB / 1.17MB
=> sha256:c427b1bb1855b88b68681612cfd3c47ddc0f7bf0af96dfbe3f1180a6f2413a92 3.83MB / 3.83MB
=> sha256:f78c7c7c697e8abb3149aa5d42619ca359de2bb03e4c2804b4136cc438866512 231B / 231B
=> sha256:740dbc132b076b321e969fa6e07fa61d94522609beffcd4716b01ba43c7c191 1.99kB / 1.99kB
=> sha256:d7dbb0503db5f19ae74e3c7c940df7c5248d499d30ce0985e6409ac424799731 106.35MB / 106.35MB
=> sha256:4ec811a3c99d671ab7f5d5f315d7f9d697eee754e3341d514f9655a6268f4e70 197B / 197B
=> extracting sha256:10ac4908093d4325f2c94b2c9a571fa1071a17a72dd9c21c1ffb2c86f68ca028
=> extracting sha256:841d9fe8d0ff117e288a003adeabeb14357e33e4aeb9257fa966be44f649e8e
=> extracting sha256:c427b1bb1855b88b68681612cfd3c47ddc0f7bf0af96dfbe3f1180a6f2413a92
=> extracting sha256:f78c7c7c697e8abb3149aa5d42619ca359de2bb03e4c2804b4136cc438866512
=> extracting sha256:740dbc132b076b321e969fa6e07fa61d94522609beffcd4716b01ba43c7c191
=> extracting sha256:d7dbb0503db5f19ae74e3c7c940df7c5248d499d30ce0985e6409ac424799731
=> extracting sha256:4ec811a3c99d671ab7f5d5f315d7f9d697eee754e3341d514f9655a6268f4e70
=> [ 2/25] RUN apt update -q && apt upgrade -yq && DEBIAN_FRONTEND=noninteractive apt install -yq --no-install-recommends keyboard-configuration language-pack-en && DEBIAN_FRONTEND=
=> [ 3/25] RUN locale-gen en_US.UTF-8
=> [ 4/25] RUN apt update && apt install -y --no-install-recommends python3-argcomplete python3-colcon-common-extensions python3-rosdep python3-colcon-mixin
=> [ 5/25] RUN apt update && apt install -y --no-install-recommends ros-humble-cv-bridge ros-humble-joy ros-humble-robot-localization ros-humble-diagnostic-u
=> [ 6/25] RUN apt update && apt install -y --no-install-recommends python3-pip && pip3 install transforms3d snbus2
=> [ 7/25] RUN apt update && apt install -qq -y --no-install-recommends build-essential cmake git libssl-dev libusb-1.0-0-dev pkg-config libgtk-3-dev libglfw3
=> [ 8/25] RUN apt install -qq -y --no-install-recommends i2c-tools && rm -rf /var/lib/apt/lists/*
=> [ 9/25] WORKDIR /tmp
=> [10/25] RUN git clone https://github.com/IntelRealSense/librealsense.git && cd librealsense && git checkout af7510c8d58f70381056a758cd3f4553aa37fc09
=> [11/25] RUN rm -rf librealsense/build && mkdir -p librealsense/build && cd librealsense/build && cmake .. && make -j$(nproc-1) && make install
=> => # [ 54%] Built target rs-multicam
=> => # [ 55%] Building CXX object examples/pointcloud/CMakeFiles/rs-pointcloud.dir/rs-pointcloud.cpp.o
=> => # [ 55%] Linking CXX executable rs-pointcloud
=> => # [ 55%] Built target rs-pointcloud
=> => # [ 56%] Building CXX object examples/align/CMakeFiles/rs-align.dir/rs-align.cpp.o
```

# Robot Setup

*\* Only if you see this error when build:*

```
mkdir /home/user/.docker/buildx: permission denied
```

- Run the following command and try to build the package again:

```
$ sudo chown -R $USER:$USER /home/$USER/.docker
```

Or Example: `sudo chown -R user:user /home/user/.docker`

```
user@amrk1t017:
```

```
$ sudo usermod -aG docker $USER  
$ sudo reboot
```



# Robot Setup

*\* Only if you are using proxy:*

- Set up a Docker proxy if you are using proxy network

<https://docs.docker.com/network/proxy/#configure-proxy-settings-per-daemon>

<https://docs.docker.com/config/daemon/systemd/#httphttps-proxy>



# Robot Setup

- Start the package:

```
$ docker compose -f docker-compose.yml up
```

```
user@anrkit017:~/workspace/frameworks.noblie.distributed-at.anr-stack-1.0-robefun/platform/scuttle/standalone$ docker compose -f docker-compose.yml up
[+] Running 2/0
  Container anr.platform.scuttle.teleop Created
  Container anr.platform.scuttle Created
Attaching to anr.platform.scuttle, anr.platform.scuttle.teleop
anr.platform.scuttle.teleop | [INFO] [launch]: All log files can be found below /root/.ros/log/2023-02-21-07-43-33-010748-anrkit017-23831
anr.platform.scuttle.teleop | [INFO] [launch]: Default logging verbosity is set to INFO
anr.platform.scuttle | [INFO] [launch]: All log files can be found below /root/.ros/log/2023-02-21-07-43-33-021390-anrkit017-23830
anr.platform.scuttle | [INFO] [launch]: Default logging verbosity is set to INFO
anr.platform.scuttle.teleop | [SCUTTLE_TELEOP] - Namespace settings found. Running node with namespace: /017
anr.platform.scuttle | [SCUTTLE_BRINGUP] - Running scuttle base type: devkit, motor type:hw231, YDLIDAR config:/ros2_ws/ROS2/install/generic_robot_driver/share/generic_robot_driver/config/ydlid
.yaml
anr.platform.scuttle | [SCUTTLE_BRINGUP] - Namespace settings found. Running node with namespace: /017
anr.platform.scuttle.teleop | [INFO] [joy_node-1]: process started with pid [23942]
anr.platform.scuttle | [INFO] [teleop_node-2]: process started with pid [23944]
anr.platform.scuttle.teleop | [teleop_node-2] [INFO] [1676965413.131730208] [TeleopTwistJoy]: Teleop enable button 2.
anr.platform.scuttle.teleop | [teleop_node-2] [INFO] [1676965413.131778319] [TeleopTwistJoy]: Turbo on button 5.
anr.platform.scuttle.teleop | [teleop_node-2] [INFO] [1676965413.131783935] [TeleopTwistJoy]: Linear axis x on 1 at scale 0.150000.
anr.platform.scuttle.teleop | [teleop_node-2] [INFO] [1676965413.131793049] [TeleopTwistJoy]: Turbo for linear axis x is scale 1.000000.
anr.platform.scuttle.teleop | [teleop_node-2] [INFO] [1676965413.131799019] [TeleopTwistJoy]: Angular axis yaw on 0 at scale 0.750000.
anr.platform.scuttle.teleop | [teleop_node-2] [INFO] [1676965413.131803933] [TeleopTwistJoy]: Turbo for angular axis yaw is scale 1.000000.
anr.platform.scuttle | [INFO] [ydlidar_ros2_driver_node-4]: process started with pid [23972]
anr.platform.scuttle | [INFO] [joint_state_publisher-1]: process started with pid [23966]
anr.platform.scuttle | [INFO] [robot_state_publisher-2]: process started with pid [23968]
anr.platform.scuttle | [INFO] [generic_robot_driver-3]: process started with pid [23970]
anr.platform.scuttle | [INFO] [realsense2_camera_node-5]: process started with pid [23974]
anr.ydlidar_ros2_driver_node-4 | [INFO] [1676965413.288927264] [ydlidar_ros2_driver_node]: [YDLIDAR INFO] Current ROS Driver Version: 1.0.1
anr.ydlidar_ros2_driver_node-4 | [ydlidar_ros2_driver_node-4] YDLidar SDK initializing
anr.ydlidar_ros2_driver_node-4 | [ydlidar_ros2_driver_node-4] YDLidar SDK has been initialized
anr.ydlidar_ros2_driver_node-4 | [YDLIDAR]:SDK Version: 1.0.3
anr.robot_state_publisher-2 | [INFO] [1676965413.294067996] [017.robot_state_publisher]: got segment base_footprint
anr.robot_state_publisher-2 | [INFO] [1676965413.294505102] [017.robot_state_publisher]: got segment base_link
anr.robot_state_publisher-2 | [INFO] [1676965413.294520235] [017.robot_state_publisher]: got segment camera_bottom_screw_frame
anr.robot_state_publisher-2 | [INFO] [1676965413.294526221] [017.robot_state_publisher]: got segment camera_link
anr.robot_state_publisher-2 | [INFO] [1676965413.294533478] [017.robot_state_publisher]: got segment l_caster_swivel_1
anr.robot_state_publisher-2 | [INFO] [1676965413.294538850] [017.robot_state_publisher]: got segment l_caster_wheel_1
anr.robot_state_publisher-2 | [INFO] [1676965413.294543917] [017.robot_state_publisher]: got segment l_wheel_1
anr.robot_state_publisher-2 | [INFO] [1676965413.294549733] [017.robot_state_publisher]: got segment lidar_1
anr.robot_state_publisher-2 | [INFO] [1676965413.294554889] [017.robot_state_publisher]: got segment r_caster_swivel_1
anr.robot_state_publisher-2 | [INFO] [1676965413.294560008] [017.robot_state_publisher]: got segment r_caster_wheel_1
anr.robot_state_publisher-2 | [INFO] [1676965413.294566226] [017.robot_state_publisher]: got segment r_wheel_1
anr.ydlidar_ros2_driver_node-4 | LIDAR successfully connected
anr.realsense2_camera_node-5 | [INFO] [1676965413.422744569] [017.camera.scuttle_camera_node]: RealSense R05 v4.51.1
anr.realsense2_camera_node-5 | [INFO] [1676965413.423146888] [017.camera.scuttle_camera_node]: Built with LibRealSense v2.51.1
anr.realsense2_camera_node-5 | [INFO] [1676965413.423166323] [017.camera.scuttle_camera_node]: Running with LibRealSense v2.51.1
anr.realsense2_camera_node-5 | [INFO] [1676965413.550359838] [017.camera.scuttle_camera_node]: Device with serial number 215122253056 was found.
anr.realsense2_camera_node-5 | [INFO] [1676965413.550444866] [017.camera.scuttle_camera_node]: Device with physical ID /sys/devices/pci0000:00/0000:00:14.0/usb4/4-3/4-3:1.0.
anr.realsense2_camera_node-5 | found.
anr.realsense2_camera_node-5 | [INFO] [1676965413.550462279] [017.camera.scuttle_camera_node]: Device with name Intel RealSense D455 was found.
anr.realsense2_camera_node-5 | [INFO] [1676965413.550817377] [017.camera.scuttle_camera_node]: Device with port number 4-3 was found.
anr.realsense2_camera_node-5 | [INFO] [1676965413.550852264] [017.camera.scuttle_camera_node]: Device USB type: 3.2
anr.realsense2_camera_node-5 | [INFO] [1676965413.550876078] [017.camera.scuttle_camera_node]: Resetting device...
anr.joint_state_publisher-1 | [INFO] [1676965413.562242453] [017.joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic...
anr.joy_node-1 | [INFO] [1676965413.840110721] [017.joy_node]: Opened joystick: Xbox 360 Controller. deadzone: 0.300000
anr.ydlidar_ros2_driver_node-4 | [YDLIDAR]:lidar running correctly ! The health status: good
anr.ydlidar_ros2_driver_node-4 | LIDAR init success!
anr.ydlidar_ros2_driver_node-4 | [YDLIDAR1]:Fixed Size: 720
anr.ydlidar_ros2_driver_node-4 | [YDLIDAR1]:Sample Rate: 3K
```

# Robot Setup

- The i2c LED will blink (showing it is functional). The robot can move if you try it using a controller. Make sure your connection is correct and the controller is turned on.
- To stop the package:

```
$ docker compose -f docker-compose.yml down
```

or

Ctrl + C

```
^CGracefully stopping... (press Ctrl+C again to force)
Aborting on container exit...
[+] Running 2/2
  :: Container amr.platform.scuttle      Stopped
  :: Container amr.platform.scuttle.teleop Stopped
canceled
user@amrkit017:~/workspace/scuttle/standalone$
```



# Robot Setup

- In the new terminal, enter the amr.platform.scuttle docker.:

```
$ docker exec -it amr.platform.scuttle bash
```

- Check to see that the ROS\_DOMAIN\_ID matches what you entered on this [slide](#):

```
$ env | grep ROS
```

\* Run the command if the ROS\_DOMAIN\_ID is incorrect:

```
$ export ROS_DOMAIN_ID=<ROBOT_DOMAIN_ID>
```



# Robot Setup

- Run: `$ source install/setup.bash`
- Run: `$ source /opt/ros/humble/setup.bash`

```
root@amrkit017:/ros2_ws/ROS2# export ROS_DOMAIN_ID=17
root@amrkit017:/ros2_ws/ROS2# source install/setup.bash
root@amrkit017:/ros2_ws/ROS2# source /opt/ros/humble/setup.bash
root@amrkit017:/ros2_ws/ROS2#
```

# Robot Setup

- Make sure all the needed topic is available:

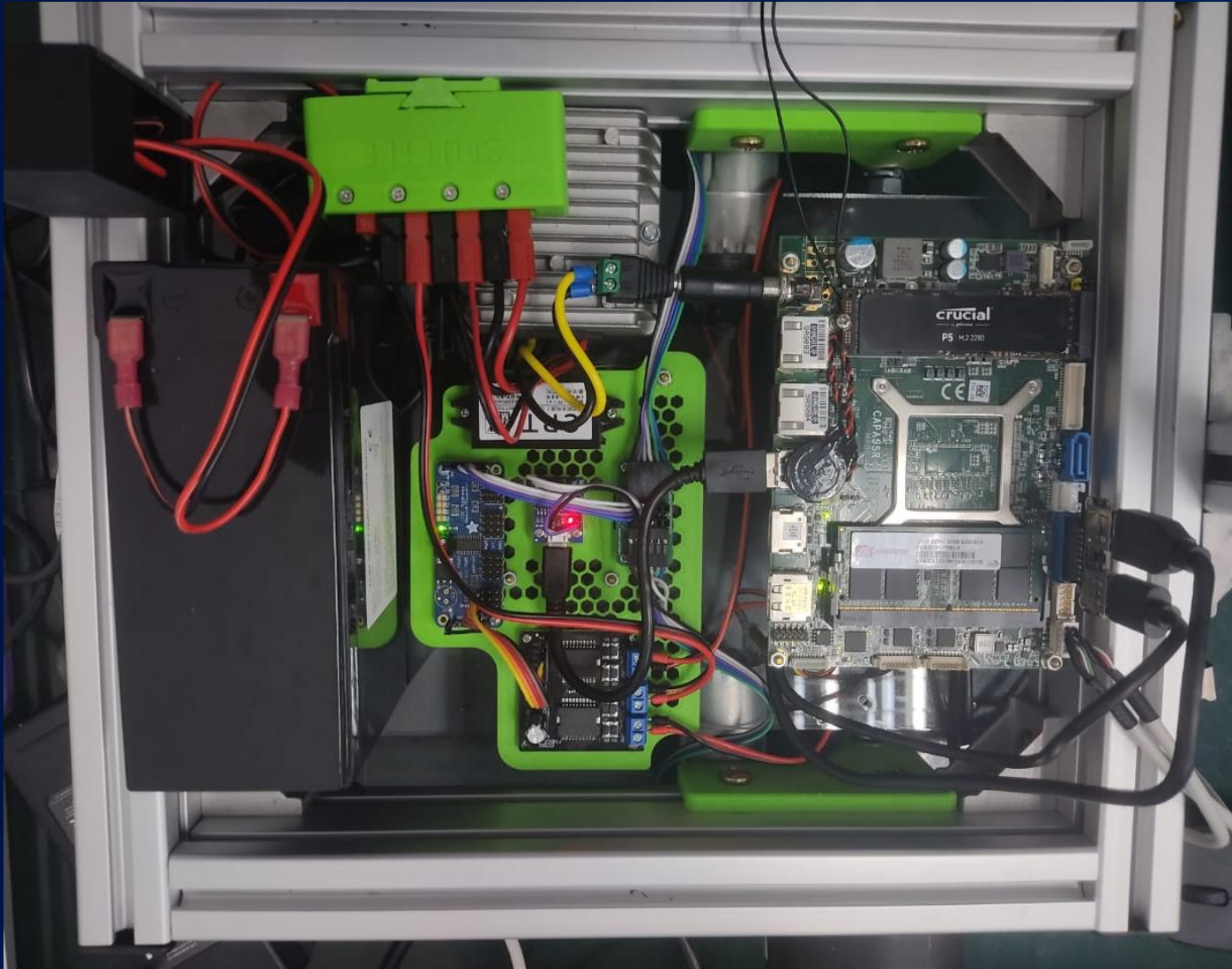
```
$ ros2 topic list
```

If all the setup is done, you are ready to launch your robot!

```
user@anrk1017:~/Desktop$ ros2 topic list
/i017/camera/accel/inu_info
/i017/camera/accel/netadata
/i017/camera/accel/sample
/i017/camera/aligned_depth_to_color/camera_info
/i017/camera/aligned_depth_to_color/image_raw
/i017/camera/aligned_depth_to_color/image_raw/compressed
/i017/camera/aligned_depth_to_color/image_raw/compressedDepth
/i017/camera/aligned_depth_to_color/image_raw/theora
/i017/camera/color/camera_info
/i017/camera/color/image_raw
/i017/camera/color/image_raw/compressed
/i017/camera/color/image_raw/compressedDepth
/i017/camera/color/image_raw/theora
/i017/camera/color/netadata
/i017/camera/depth/camera_info
/i017/camera/depth/image_rect_raw
/i017/camera/depth/image_rect_raw/compressed
/i017/camera/depth/image_rect_raw/compressedDepth
/i017/camera/depth/image_rect_raw/theora
/i017/camera/depth/netadata
/i017/camera/extrinsics/depth_to_accel
/i017/camera/extrinsics/depth_to_color
/i017/camera/extrinsics/depth_to_gyro
/i017/camera/gyro/inu_info
/i017/camera/gyro/netadata
/i017/camera/gyro/sample
/i017/camera/inu
/i017/client_count
/i017/cnd_vel
/i017/connected_clients
/i017/joint_states
/i017/joy
/i017/joy/set_feedback
/i017/odom/anr
/i017/robot_description
/i017/scan
/i017/tf
/i017/tf_static
/parameter_events
/rosout
```



# Complete Robot Setup





# Training Sensors

# RealSense Camera

- RealSense is a technology platform developed by Intel that enables computers and devices to perceive and interact with their environment in a more natural way. It uses a combination of **cameras, sensors, and software** to capture depth, motion, and image data, allowing for accurate and detailed recognition of objects and people in real-time.
- RealSense technology is used in a variety of applications, including
  - gaming
  - virtual reality
  - robotics & autonomous navigation
  - 3D scanning
- Specifications:
  - Depth Technology: Active IR stereo
  - Depth Field of View (FoV): 87 degrees (H) x 58 degrees (V) x 95 degrees (D)
  - Depth Output Resolution: Up to 1280 x 720 at 30 frames per second (fps)
  - RGB Sensor Resolution: Up to 1920 x 1080 at 30 fps
  - RGB Sensor FoV: 69.4 degrees (H) x 42.5 degrees (V) x 77 degrees (D)
  - Depth Distance: 10cm (min), 10meters (max)
  - Depth Accuracy: +/- 2% of distance (in the range of 0.2m to 6m)
  - Power Consumption: 3.5W



RealSense Camera D455

# Visualize RealSense Stream with Intel® RealSense™ viewer

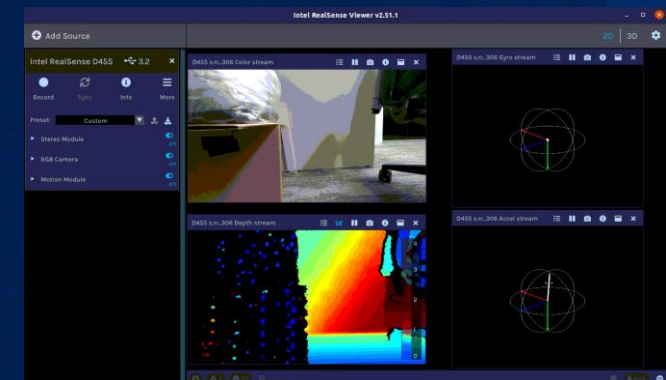
- In a new terminal, run the following command to install the realsense viewer.

```
$ sudo apt install -y ros-humble-realsense2-tutorial-demo
$ source /opt/ros/humble/setup.bash
$ LIBREALSENSE_VER=2.54.2-0~realsense.10773
$ sudo apt install -y librealsense2=$LIBREALSENSE_VER librealsense2-
gl=$LIBREALSENSE_VER
$ sudo apt install --allow-downgrades -y librealsense2-dkms
librealsense2-utils=$LIBREALSENSE_VER librealsense2-udev-
rules=$LIBREALSENSE_VER
```

- Run the following command to start the Intel® RealSense™ viewer app.

```
$ realsense-viewer
```

*Note: ensure your docker-compose is down to launch realsense-viewer*





# YDLidar X2

- YDLIDAR X2 is a 360-degree laser scanner designed for a wide range of robotic applications.
- Hardware Specifications:
  - Range: 8 meters (indoors) or 1-3 meters (outdoors)
  - Angular resolution: 0.4-1 degree
  - Scan frequency: up to 6,000 Hz
  - Operating voltage: 5V DC
  - Power consumption: less than 5W
  - Dimensions: 62mm x 35mm
- Features of the YDLIDAR X2 in robotics application includes:
  - High accuracy and stability
  - Real-time mapping and obstacle avoidance
  - Easy to integrate into various systems
  - Compact and lightweight design
  - Compatible with ROS (Robot Operating System)



# Link

1. RealSense: [link](#)
2. RealSense D455 datasheet: [link](#)
3. Run the RealSense example: [link](#)
4. YDLidar: [link](#)

# Training Simultaneous Localization and Mapping (SLAM)



# SLAM

- Mapping is a process in which an AMR perceives and comprehends its surrounding environment using sensor devices in combination with SLAM (Simultaneous Localization and Mapping), enabling it to create an accurate and detailed representation of the surrounding.
- The process of mapping can be summarized in the diagram below:



- In this section, we will use 4 new terminals to run different nodes to create an occupancy grid map which include:
  - 1<sup>st</sup> terminal: cartographer node
  - 2<sup>nd</sup> terminal: occupancy grid node
  - 3<sup>rd</sup> terminal: Rviz2 visualization node
  - 4<sup>th</sup> terminal: save map node

# SLAM

- Ensure docker in [slide 46](#) is up, open new terminal (*1<sup>st</sup> terminal: cartographer node*) and run :

```
$ ros2 run cartographer_ros cartographer_node -configuration_directory  
/home/$USER/workspace/robofun-  
1.0/platform/scuttle/ROS2/src/generic_robot_driver/config -  
configuration_basename cartographer.lua --ros-args -p  
use_sim_time:=false -r odom:=odom/amr -r  
/constraint_list:=constraint_list -r  
/parameter_events:=parameter_events -r /rosout:=rosout -r /scan:=scan -  
r /scan_matched_points2:=scan_matched_points2 -r  
/submap_list:=submap_list -r /tf:=tf -r /tf_static:=tf_static -r  
/trajectory_node_list:=trajectory_node_list -r __ns:=/robot_ns
```

Note: Remember to check your *path to access the cartographer.lua*

Note: each *\_\_ns:=* must change to your own **ROBOT\_NAMESPAC**



# SLAM

```
root@P15HL01NJASMAN:/home/eiforamr/workspace# ros2 run cartographer_ros cartographer_node -configuration_directory /home/eiforamr/ros2_ws/src/config -configuration_basename cartographer.lua --ros-args -p us
ameter_events:=parameter_events -r /rosout:=rosout -r /scan:=scan -r /scan_matched_points2:=scan_matched_points2 -r /submap_list:=submap_list -r /tf:=tf -r /tf_static:=tf_static -r /trajectory_node_list:=tr
[INFO] [1678011115.238798015] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/home/eiforamr/ros2_ws/src/config/cartographer.lua' for 'cartographer.lua'.
[INFO] [1678011115.239373028] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/opt/ros/foxy/share/cartographer/configuration_files/map_builder.lua' for 'map_builder
[INFO] [1678011115.239399779] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/opt/ros/foxy/share/cartographer/configuration_files/map_builder.lua' for 'map_builder
[INFO] [1678011115.239613378] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/opt/ros/foxy/share/cartographer/configuration_files/pose_graph.lua' for 'pose_graph.l
[INFO] [1678011115.239636026] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/opt/ros/foxy/share/cartographer/configuration_files/pose_graph.lua' for 'pose_graph.l
[INFO] [1678011115.239869225] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/opt/ros/foxy/share/cartographer/configuration_files/trajectory_builder.lua' for 'traj
[INFO] [1678011115.239890956] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/opt/ros/foxy/share/cartographer/configuration_files/trajectory_builder.lua' for 'traj
[INFO] [1678011115.240024094] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/opt/ros/foxy/share/cartographer/configuration_files/trajectory_builder_2d.lua' for 't
[INFO] [1678011115.240055424] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/opt/ros/foxy/share/cartographer/configuration_files/trajectory_builder_2d.lua' for 't
[INFO] [1678011115.240232297] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/opt/ros/foxy/share/cartographer/configuration_files/trajectory_builder_3d.lua' for 't
[INFO] [1678011115.240253388] [cartographer_ros]: I0305 10:11:55.000000 43 configuration_file_resolver.cc:41] Found '/opt/ros/foxy/share/cartographer/configuration_files/trajectory_builder_3d.lua' for 't
[INFO] [1678011115.271644304] [cartographer_ros]: I0305 10:11:55.000000 43 submap_2d.cc:187] Added submap 1
[INFO] [1678011115.271719879] [cartographer_ros]: I0305 10:11:55.000000 43 map_builder_bridge.cc:132] Added trajectory with ID '0'.
[INFO] [1678011115.475091053] [cartographer_ros]: I0305 10:11:55.000000 43 ordered_multi_queue.cc:172] All sensor data for trajectory 0 is available starting at '638136079154708553'.
[INFO] [1678011115.475132863] [cartographer_ros]: I0305 10:11:55.000000 43 local_trajectory_builder_2d.cc:295] Extrapolator not yet initialized.
[INFO] [1678011115.475195688] [cartographer_ros]: I0305 10:11:55.000000 43 local_trajectory_builder_2d.cc:138] Extrapolator is still initializing.
[INFO] [1678011130.280329305] [cartographer_ros]: I0305 10:12:10.000000 43 collated_trajectory_builder.cc:72] odom rate: 50.00 Hz 2.00e-02 s +/- 4.53e-05 s (pulsed at 98.34% real time)
[INFO] [1678011130.280366793] [cartographer_ros]: I0305 10:12:10.000000 43 collated_trajectory_builder.cc:72] scan rate: 3.58 Hz 2.79e-01 s +/- 1.09e-03 s (pulsed at 99.94% real time)
[INFO] [1678011145.357771042] [cartographer_ros]: I0305 10:12:25.000000 43 collated_trajectory_builder.cc:72] odom rate: 50.00 Hz 2.00e-02 s +/- 5.37e-05 s (pulsed at 100.03% real time)
[INFO] [1678011145.357809296] [cartographer_ros]: I0305 10:12:25.000000 43 collated_trajectory_builder.cc:72] scan rate: 3.58 Hz 2.79e-01 s +/- 1.15e-03 s (pulsed at 99.97% real time)
[INFO] [1678011160.435011507] [cartographer_ros]: I0305 10:12:40.000000 43 collated_trajectory_builder.cc:72] odom rate: 50.00 Hz 2.00e-02 s +/- 4.48e-05 s (pulsed at 99.87% real time)
[INFO] [1678011160.435095053] [cartographer_ros]: I0305 10:12:40.000000 43 collated_trajectory_builder.cc:72] scan rate: 3.58 Hz 2.79e-01 s +/- 1.34e-03 s (pulsed at 100.12% real time)
[INFO] [1678011175.521812022] [cartographer_ros]: I0305 10:12:55.000000 43 collated_trajectory_builder.cc:72] odom rate: 50.00 Hz 2.00e-02 s +/- 6.38e-05 s (pulsed at 100.07% real time)
[INFO] [1678011175.521862332] [cartographer_ros]: I0305 10:12:55.000000 43 collated_trajectory_builder.cc:72] scan rate: 3.58 Hz 2.79e-01 s +/- 1.08e-03 s (pulsed at 99.92% real time)
[INFO] [1678011190.602691954] [cartographer_ros]: I0305 10:13:10.000000 43 collated_trajectory_builder.cc:72] odom rate: 50.
00 Hz 2.00e-02 s +/- 4.12e-05 s (pulsed at 99.99% real time)
[INFO] [1678011190.602731778] [cartographer_ros]: I0305 10:13:10.000000 43 collated_trajectory_builder.cc:72] scan rate: 3.5
8 Hz 2.79e-01 s +/- 1.18e-03 s (pulsed at 100.02% real time)
```

Cartographer node launch correctly



# SLAM

- Open new terminal (*2<sup>nd</sup> terminal: occupancy grid node*), run the following to Run occupancy grid node

```
$ ros2 run cartographer_ros cartographer_occupancy_grid_node -  
resolution 0.05 -publish_period_sec 1.0 --ros-args -p  
use_sim_time:=false -r /map:=map -r /submap_list:=submap_list -r  
__ns:=/robot_ns
```

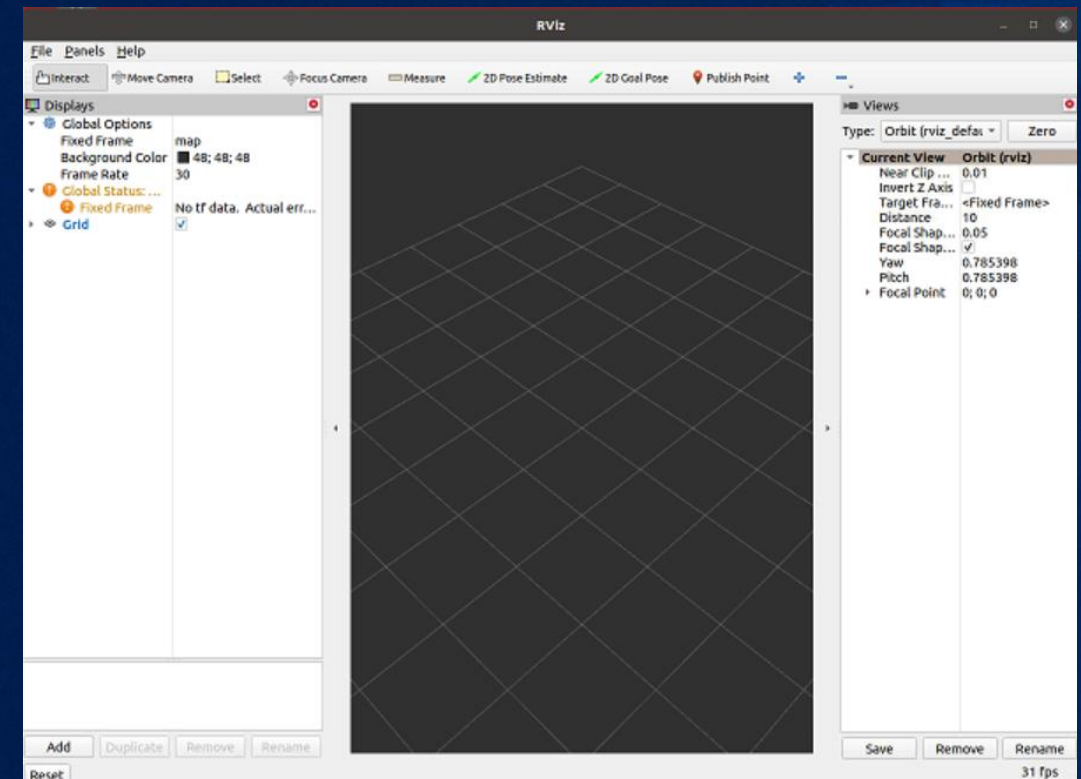
Note: each *\_\_ns:=* must change to your own **ROBOT\_NAMESPACE**

# SLAM

- Open new terminal (3<sup>rd</sup> terminal: *Rviz2 visualization node*) and run the following command in the same terminal to start RVIZ2

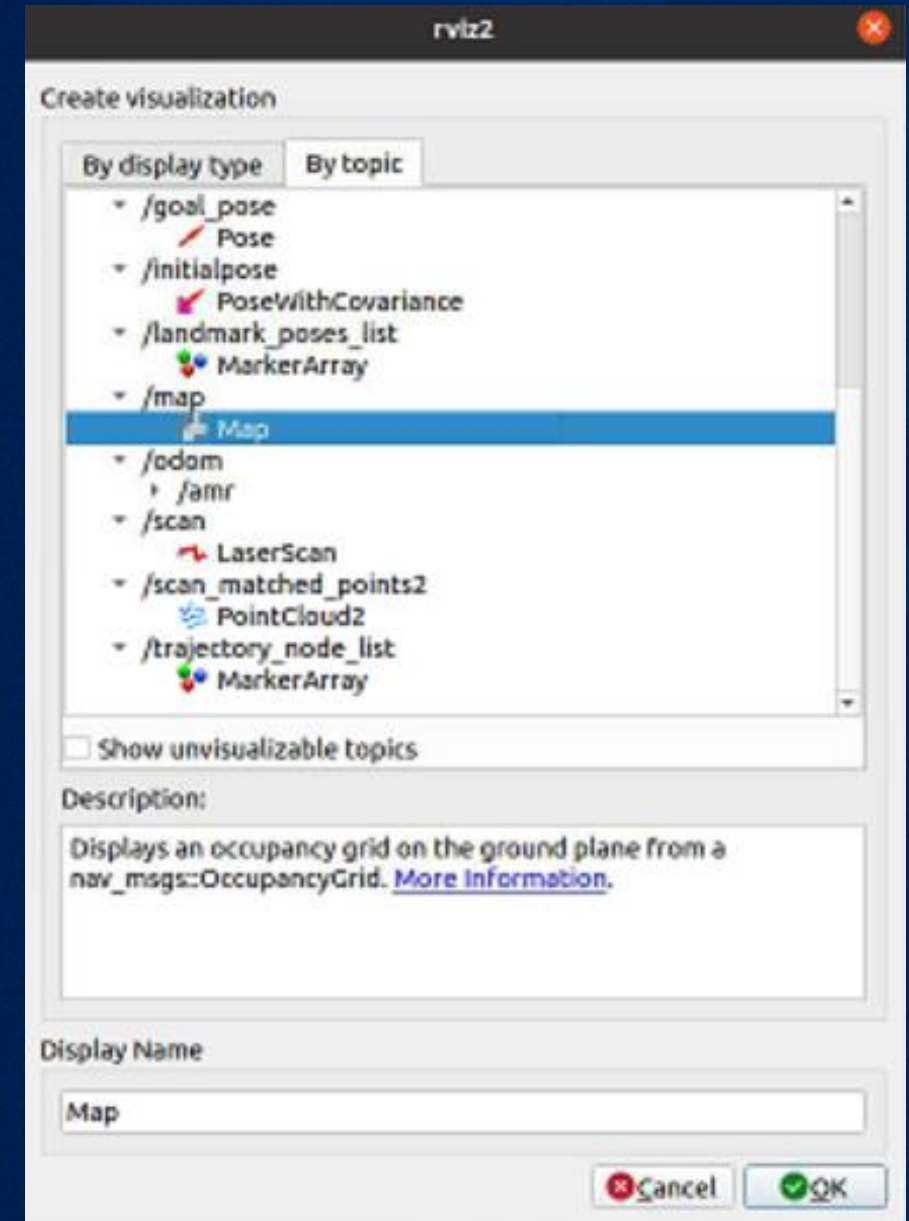
```
$ rviz2 --ros-args -r  
__ns:=/robot_ns -r /tf:=tf  
-r /tf_static:=tf_static -r  
/clicked_point:=clicked_point  
-r /goal_pose:=goal_pose -r  
/initialpose:=initialpose
```

Note: each `__ns:=` must change to your own  
`ROBOT_NAMESPACE`



# SLAM

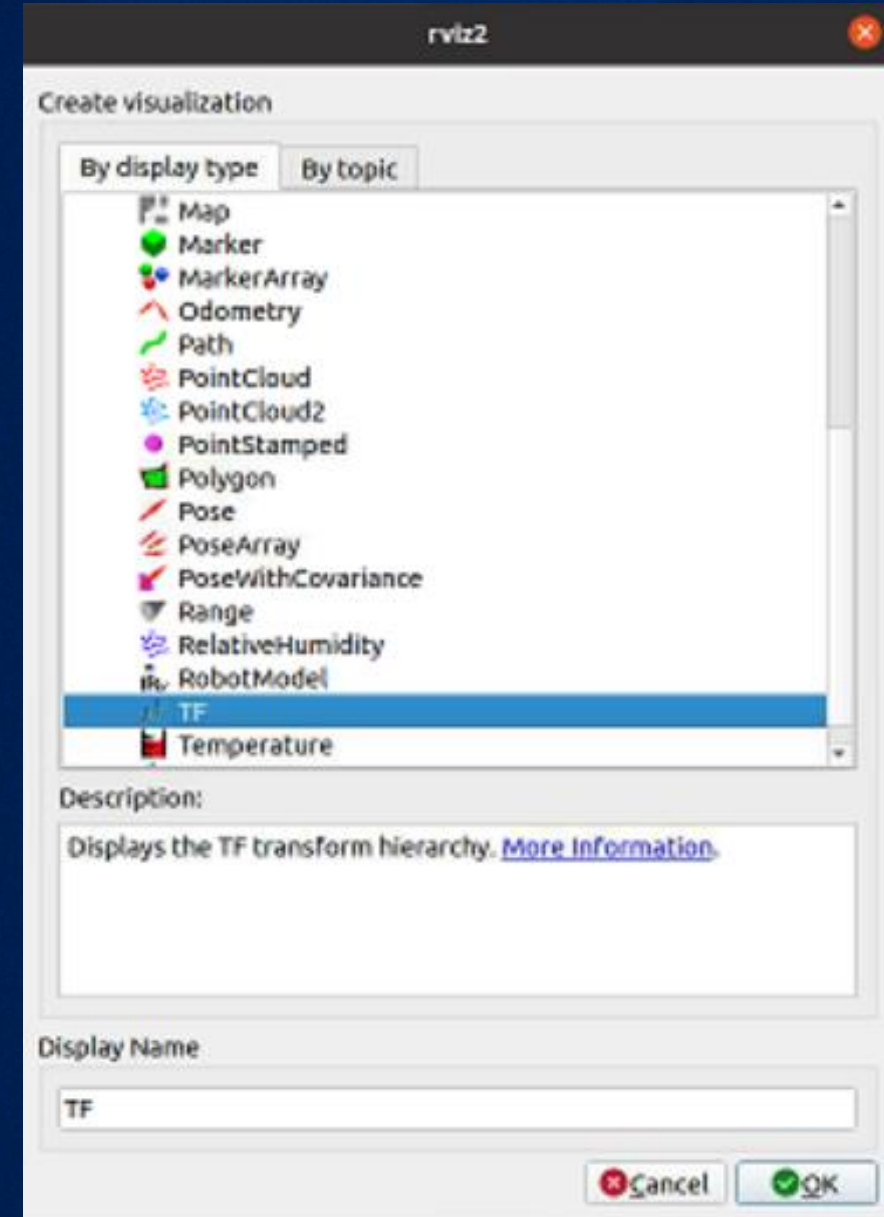
- Add topic <ROBOT\_NAMESPACE> /map at 'By topic' section to display the generated map.
- Add topic <ROBOT\_NAMESPACE>/scan at the same section.





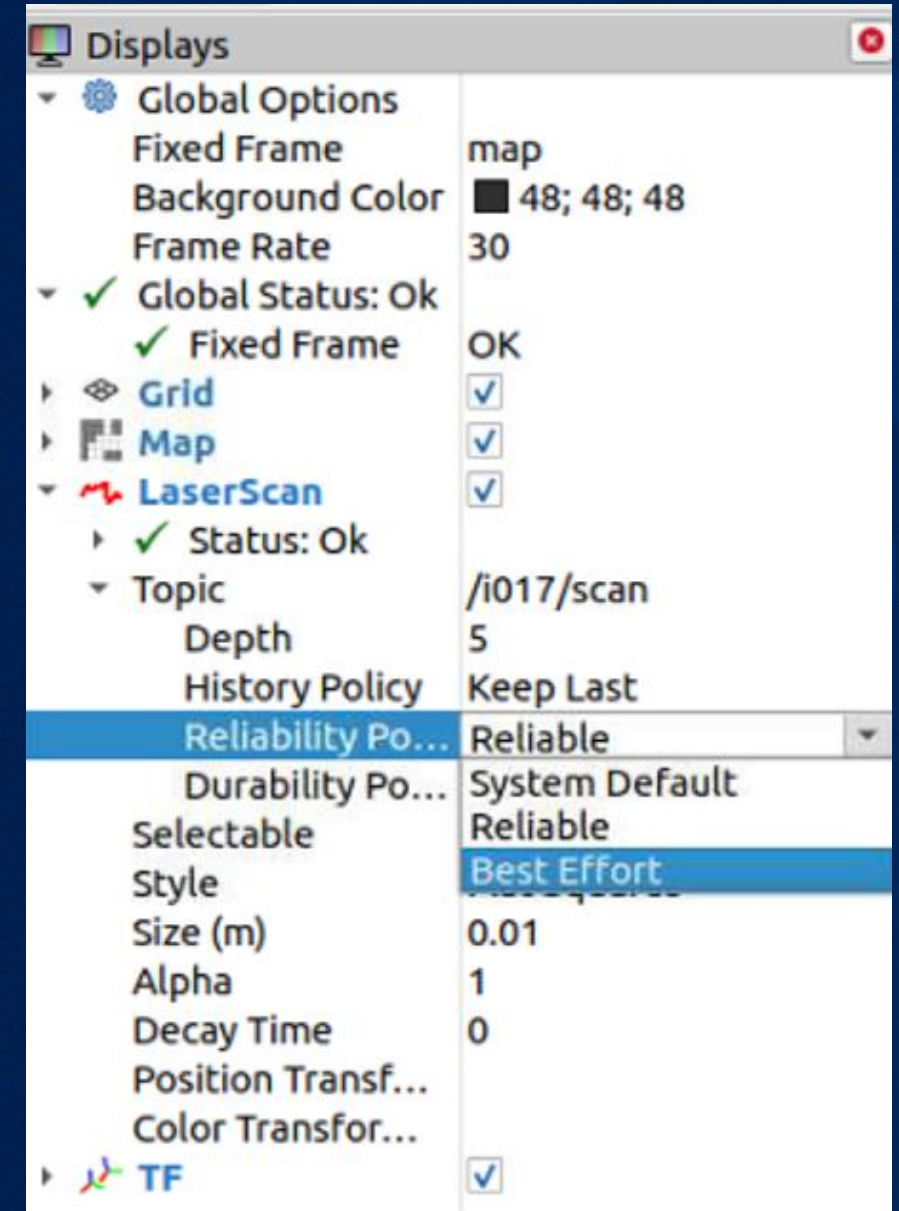
# SLAM

- Add topic TF at 'By display type' section to display the transform hierarchy.



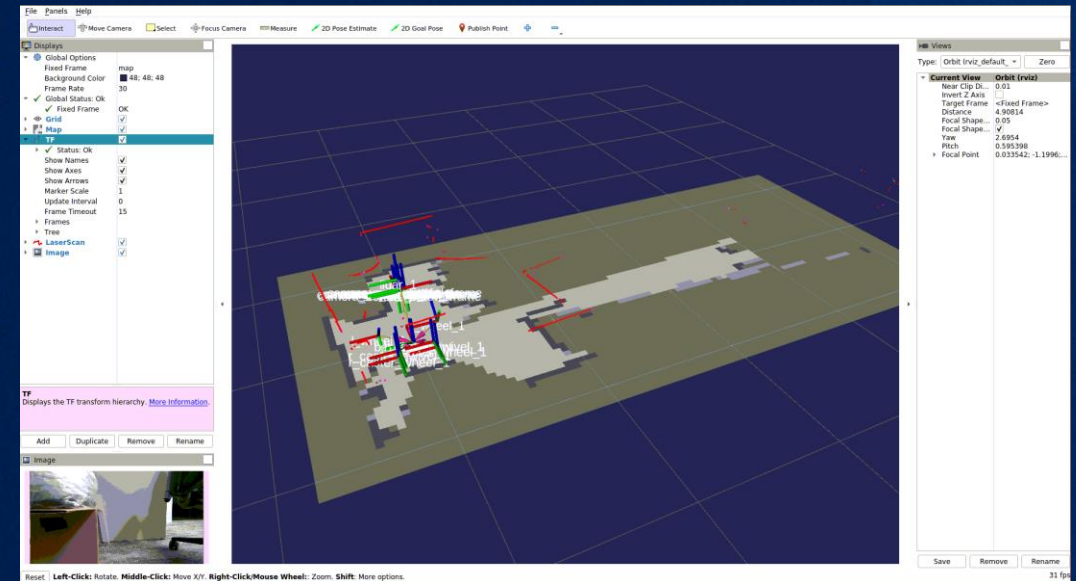
# SLAM

- In LaserScan:
  - Make sure the status is Ok.
  - Click on Topic and set the Reliability Policy to “Best Effort”.
- Save configuration:
  - Save the rviz2 configuration using Ctrl+S



# SLAM

- The robot is starting to map the location
- You can map the location by moving the robot around.



RVIZ2 visualizes the lidar data & map data



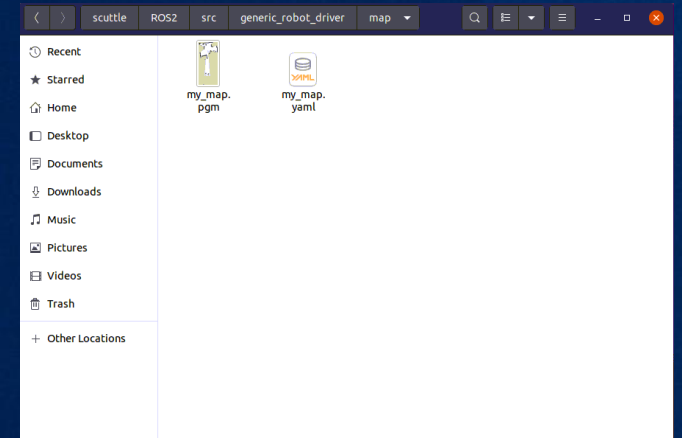
# SLAM

- Open new terminal (*4<sup>th</sup> terminal: save map node*) and run the following command to save the map:

```
$ ros2 run nav2_map_server map_saver_cli  
-f /home/$USER/workspace/robofun-  
1.0/my_map --ros-args -p  
free_thresh_default:=0.196 -r /map:=map -  
r __ns:=/robot_ns
```

Note: *check the directory for the map to be saved*

Note: *each \_\_ns:= must change to your own ROBOT\_NAMESPACE*



Map saved in  
home/<username>/workspace/robofun-1.0

# Link

1. SLAM: <https://ouster.com/blog/introduction-to-slam-simultaneous-localization-and-mapping/>

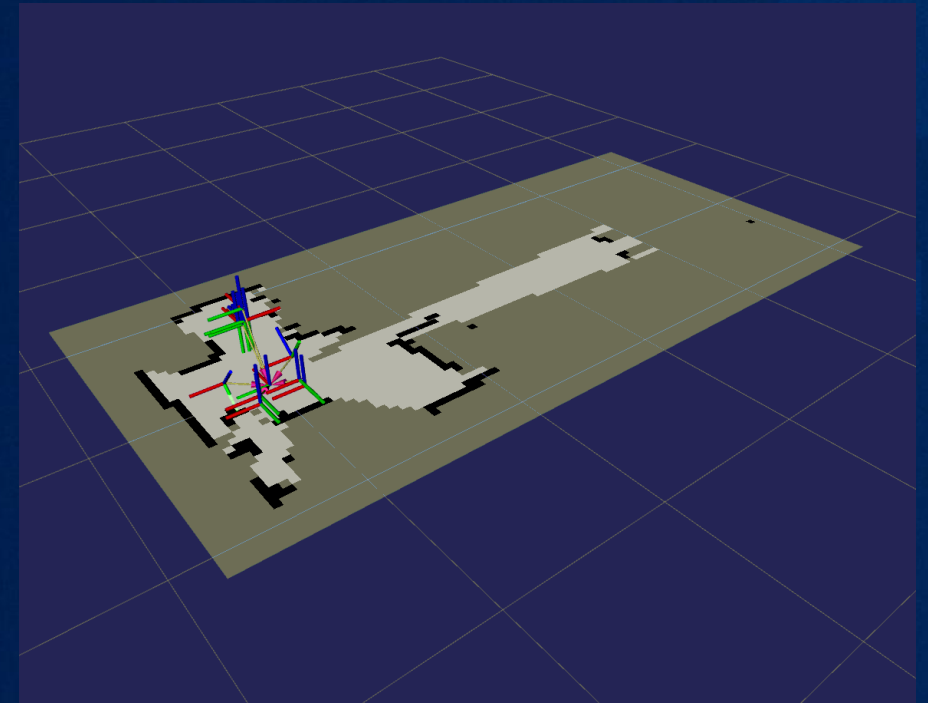
Training

# Localization & Navigation



# Localization & Navigation

- Localization
  - Localization is the process of determining the position and orientation of a robot in its environment
  - Common package used in ROS2 for localization is AMCL (Adaptive Monte Carlo Localization)
  - AMCL is a probabilistic algorithm that uses a particle filter to estimate the robot's pose based on sensor measurements
- Navigation
  - Process of planning and controlling the motion of a robot to reach a target location while avoiding obstacles
  - Common package used in ROS2 for navigation is ROS2 Navigation Stack
  - The Navigation Stack is a collection of ROS2 packages that provide algorithms for global and local path planning, obstacle avoidance, and control



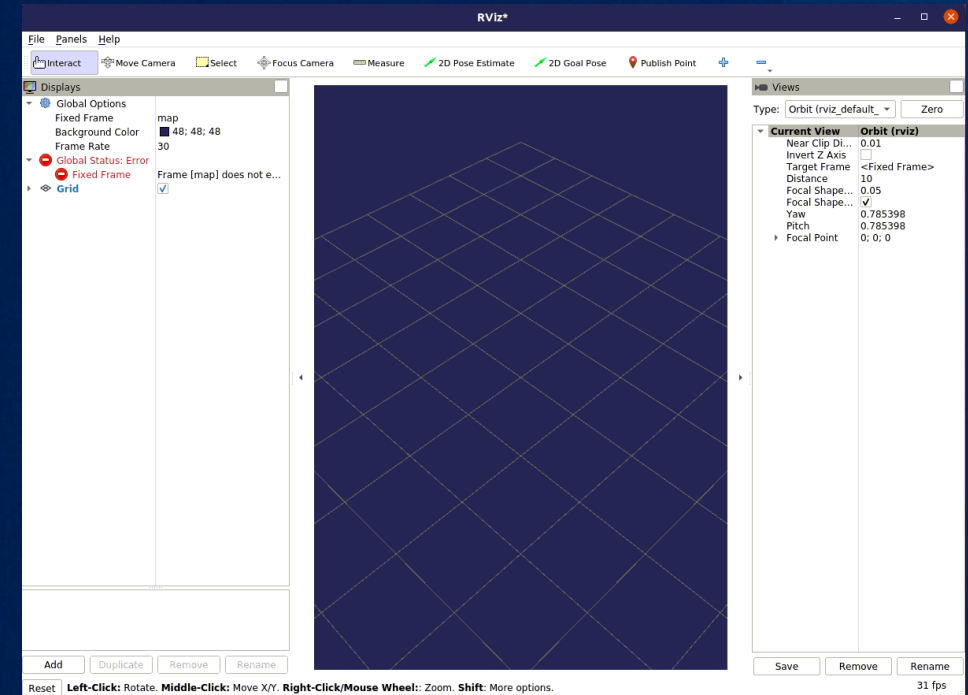
AMR localization & navigation on a saved map

# Localization & Navigation

- In terminal 1 (1<sup>st</sup>) run the following command start RVIZ2

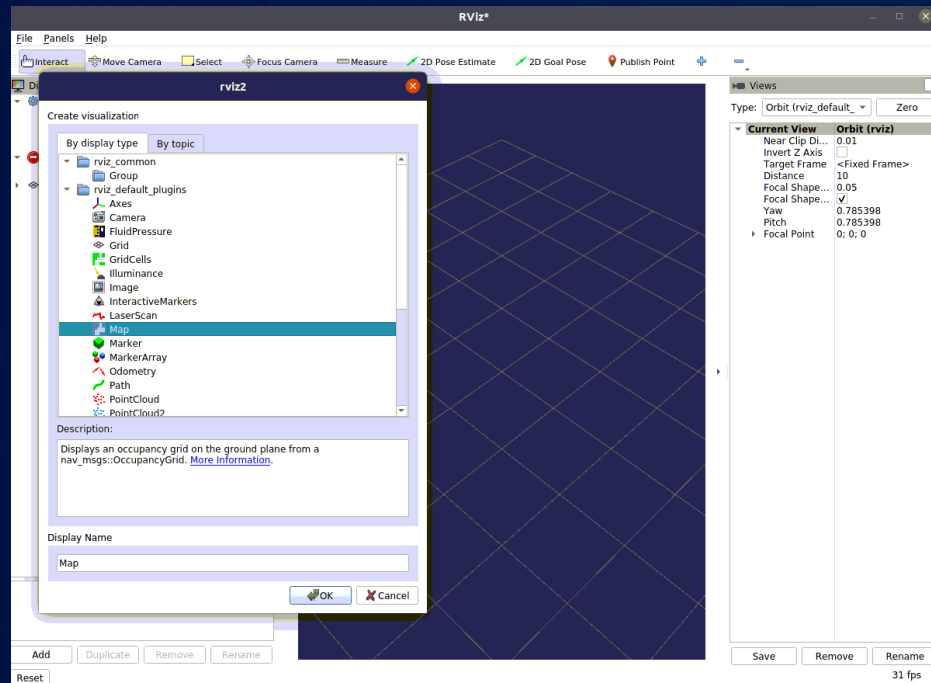
```
$ rviz2 --ros-args -r  
__ns:=/<robot_ns> -r /tf:=tf -r  
/tf_static:=tf_static -r  
/clicked_point:=clicked_point -r  
/goal_pose:=goal_pose -r  
/initialpose:=initialpose
```

Note: each `__ns:=` must change to your own  
`ROBOT_NAMESPACE`

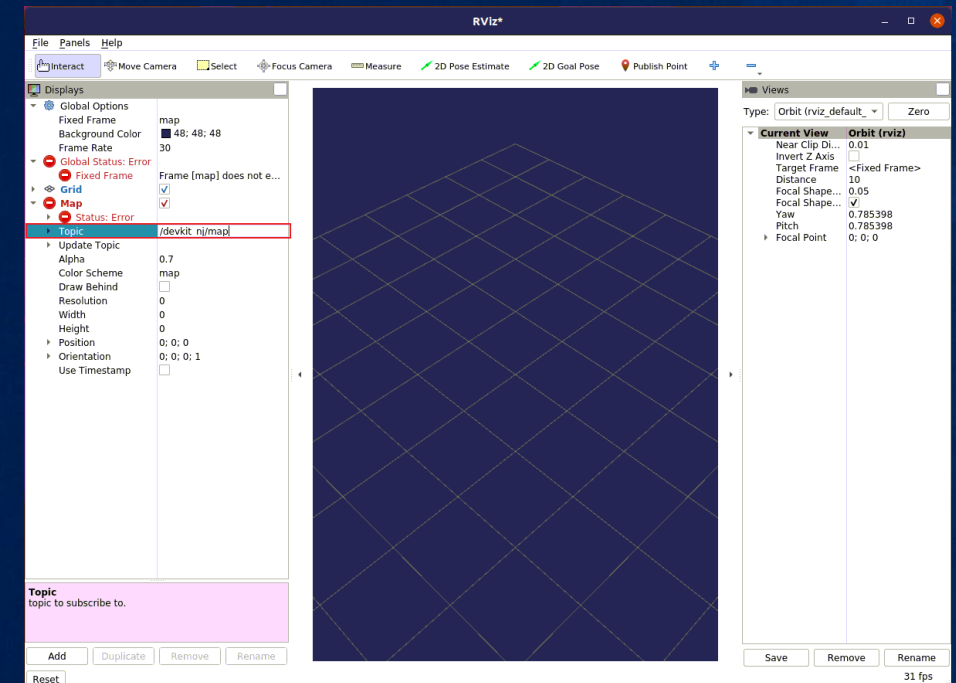


# Localization & Navigation

- Add /map topic to visualize the map when the navigation node start later.
- The topic of the Map should be in the format: <ROBOT\_NAMESPACE>/map



Select *Map* Display type

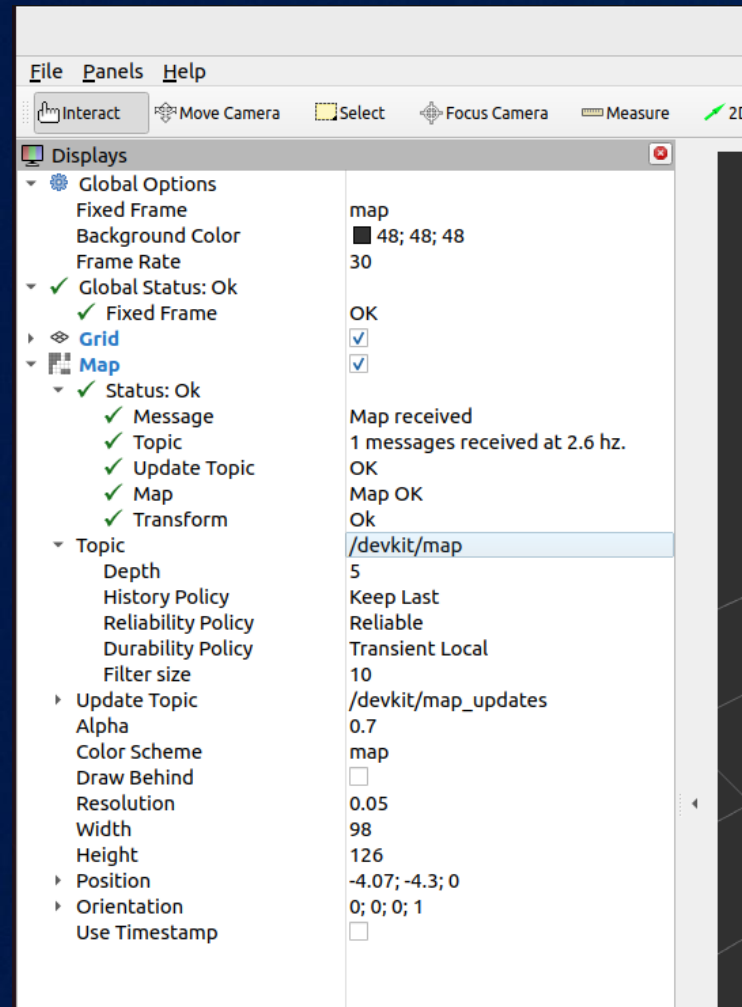


Add map topic into the newly added Map display type



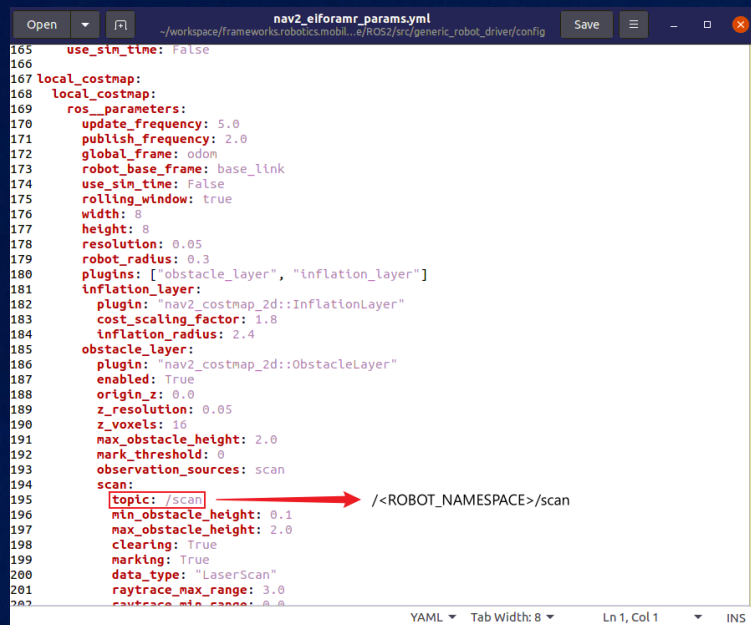
# Localization & Navigation

- Ensure you have set the correct map topic



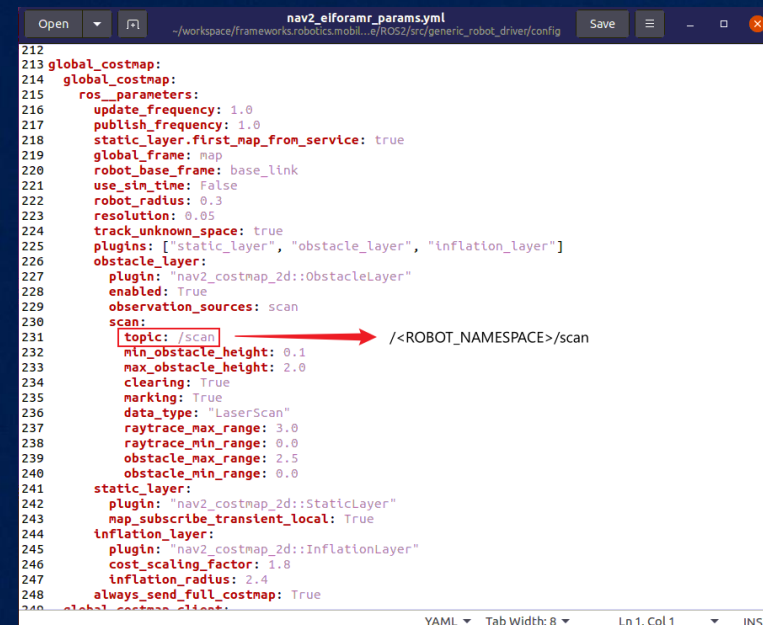
# Localization & Navigation

- Edit the Nav2 config file in `/home/<your_user_name>/workspace/robofun-1.0/platform/scuttle/ROS2/src/generic_robot_driver/config/nav2_eiforamr_params.yml`
- In the local map (line 195) and global map (line 231) section, change the scan observation source topic to `/<ROBOT_NAMESPACE>/scan`



```
165 use_sim_time: False
166
167 local_costmap:
168   local_costmap:
169     ros_parameters:
170       update_frequency: 5.0
171       publish_frequency: 2.0
172       global_frame: odom
173       robot_base_frame: base_link
174       use_sim_time: False
175       rolling_window: true
176       width: 8
177       height: 8
178       resolution: 0.05
179       robot_radius: 0.3
180     plugins: ["obstacle_layer", "inflation_layer"]
181     inflation_layer:
182       plugin: "nav2_costmap_2d::InflationLayer"
183       cost_scaling_factor: 1.8
184       inflation_radius: 2.4
185     obstacle_layer:
186       plugin: "nav2_costmap_2d::ObstacleLayer"
187       enabled: True
188       origin_z: 0.0
189       z_resolution: 0.05
190       z_voxels: 16
191       max_obstacle_height: 2.0
192       mark_threshold: 0
193       observation_sources: scan
194       scan:
195         topic: /scan
196         min_obstacle_height: 0.1
197         max_obstacle_height: 2.0
198         clearing: True
199         marking: True
200         data_type: "LaserScan"
201         raytrace_max_range: 3.0
202         raytrace_min_range: 0.0
```

Change local costmap scan observation source topic



```
212
213 global_costmap:
214   global_costmap:
215     ros_parameters:
216       update_frequency: 1.0
217       publish_frequency: 1.0
218       static_layer.first_map_from_service: true
219       global_frame: map
220       robot_base_frame: base_link
221       use_sim_time: False
222       robot_radius: 0.3
223       resolution: 0.05
224       track_unknown_space: true
225     plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
226     obstacle_layer:
227       plugin: "nav2_costmap_2d::ObstacleLayer"
228       enabled: True
229       observation_sources: scan
230       scan:
231         topic: /scan
232         min_obstacle_height: 0.1
233         max_obstacle_height: 2.0
234         clearing: True
235         marking: True
236         data_type: "LaserScan"
237         raytrace_max_range: 3.0
238         raytrace_min_range: 0.0
239         obstacle_max_range: 2.5
240         obstacle_min_range: 0.0
241     static_layer:
242       plugin: "nav2_costmap_2d::StaticLayer"
243       map_subscribe_transient_local: True
244     inflation_layer:
245       plugin: "nav2_costmap_2d::InflationLayer"
246       cost_scaling_factor: 1.8
247       inflation_radius: 2.4
248     always_send_full_costmap: True
249
```

Change global costmap scan observation source topic

# Localization & Navigation

- Edit the `robot_model_type` in `nav2_eiforamr_params.yml` (line 29)
- Change the `"differential"` to `"nav2_amcl::DifferentialMotionModel"`

```
22 min_particles: 2000
23 odom_frame_id: "odom"
24 pf_err: 0.05
25 pf_z: 0.99
26 recovery_alpha_fast: 0.0
27 recovery_alpha_slow: 0.0
28 resample_interval: 1
29 robot_model_type: "differential"
```

Old model type "differential"

```
22 min_particles: 2000
23 odom_frame_id: "odom"
24 pf_err: 0.05
25 pf_z: 0.99
26 recovery_alpha_fast: 0.0
27 recovery_alpha_slow: 0.0
28 resample_interval: 1
29 robot_model_type: "nav2_amcl::DifferentialMotionModel"
```

New model type "nav2\_amcl::DifferentialMotionModel"



# Localization & Navigation

- Replace the old bt\_navigator in nav2\_eiforamr\_params.yml (line58 –line92) to a new bt\_navigator.
- The new bt\_navigator can be found in day2.txt

```
bt_navigator:
  ros__parameters:
    use_sim_time: False
    global_frame: map
    robot_base_frame: base_link
    odom_topic: odom/amr
    bt_loop_duration: 10
    default_server_timeout: 20
    enable_groot_monitoring: True
    groot_zmq_publisher_port: 1666
    groot_zmq_server_port: 1667
    default_bt_xml_filename: "navigate_w_replanning_and_recovery.xml"
    plugin_lib_names:
      - nav2_compute_path_to_pose_action_bt_node
      - nav2_follow_path_action_bt_node
      - nav2_back_up_action_bt_node
      - nav2_spin_action_bt_node
      - nav2_wait_action_bt_node
      - nav2_clear_costmap_service_bt_node
      - nav2_is_stuck_condition_bt_node
      - nav2_goal_reached_condition_bt_node
      - nav2_goal_updated_condition_bt_node
      - nav2_initial_pose_received_condition_bt_node
      - nav2_reinitialize_global_localization_service_bt_node
      - nav2_rate_controller_bt_node
      - nav2_distance_controller_bt_node
      - nav2_speed_controller_bt_node
      - nav2_truncate_path_action_bt_node
      - nav2_goal_updater_node_bt_node
      - nav2_recovery_node_bt_node
      - nav2_pipeline_sequence_bt_node
      - nav2_round_robin_node_bt_node
      - nav2_transform_available_condition_bt_node
      - nav2_time_expired_condition_bt_node
      - nav2_distance_traveled_condition_bt_node
```

Old bt\_navigator

```
bt_navigator:
  ros__parameters:
    use_sim_time: False
    global_frame: map
    robot_base_frame: base_link
    odom_topic: odom/amr
    bt_loop_duration: 10
    default_server_timeout: 20
    # 'default_nav_through_poses_bt.xml' and 'default_nav_to_pose_bt.xml' are use defaults:
    # nav2_bt_navigator/navigate_to_pose_w_replanning_and_recovery.xml
    # nav2_bt_navigator/navigate_through_poses_w_replanning_and_recovery.xml
    # They can be set here or via a RewrittenYaml remap from a parent launch file to Nav2.
    plugin_lib_names:
      - nav2_compute_path_to_pose_action_bt_node
      - nav2_compute_path_through_poses_action_bt_node
      - nav2_follow_path_action_bt_node
      - nav2_back_up_action_bt_node
      - nav2_spin_action_bt_node
      - nav2_wait_action_bt_node
      - nav2_clear_costmap_service_bt_node
      - nav2_is_stuck_condition_bt_node
      - nav2_goal_reached_condition_bt_node
      - nav2_goal_updated_condition_bt_node
      - nav2_initial_pose_received_condition_bt_node
      - nav2_reinitialize_global_localization_service_bt_node
      - nav2_rate_controller_bt_node
      - nav2_distance_controller_bt_node
      - nav2_speed_controller_bt_node
      - nav2_truncate_path_action_bt_node
      - nav2_goal_updater_node_bt_node
      - nav2_recovery_node_bt_node
      - nav2_pipeline_sequence_bt_node
      - nav2_round_robin_node_bt_node
      - nav2_transform_available_condition_bt_node
      - nav2_time_expired_condition_bt_node
      - nav2_distance_traveled_condition_bt_node
      - nav2_single_trigger_bt_node
      - nav2_is_battery_low_condition_bt_node
      - nav2_navigate_through_poses_action_bt_node
      - nav2_navigate_to_pose_action_bt_node
      - nav2_remove_passed_goals_action_bt_node
      - nav2_planner_selector_bt_node
      - nav2_controller_selector_bt_node
      - nav2_goal_checker_selector_bt_node
```

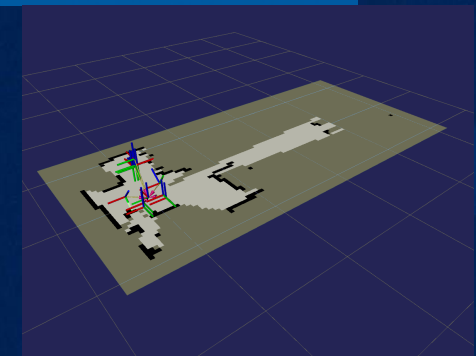
New bt\_navigator

# Localization & Navigation

- In the amr-nav2 containers terminal (2<sup>nd</sup> terminal), run the following command to start the amr-navigation node

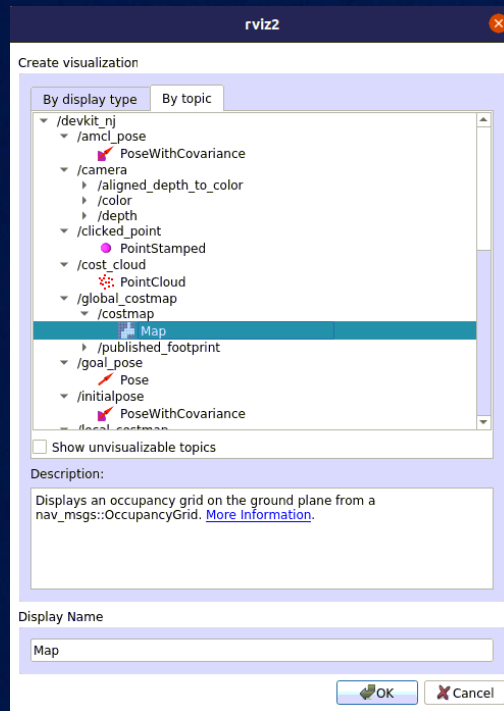
```
$ ros2 launch nav2_bringup bringup_launch.py  
params_file:=/home/$USER/workspace/robofun-  
1.0/platform/scuttle/ROS2/src/generic_robot_driver/config/n  
av2_eiforamr_params.yml map:=/home/$USER/workspace/robofun-  
1.0/my_map.yaml use_sim_time:=false namespace:=/robot_ns  
use_namespace:=True
```

- You should be able to see the saved map in the mapping session showed up in RVIZ2
- Note:
  - Ensure the path to `nav2_eiforamr_params.yml` and `my_map.yaml` is correct

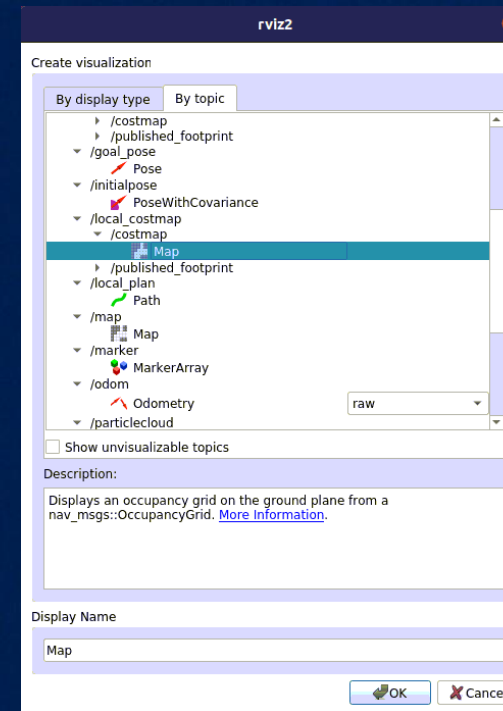


# Localization & Navigation

- To view the costmap generated, add the following topic in RVIZ2
  - `<ROBOT NAMESPACE>/local_costmap/costmap`
  - `<ROBOT NAMESPACE>/global_costmap/costmap`



Selecting global costmap

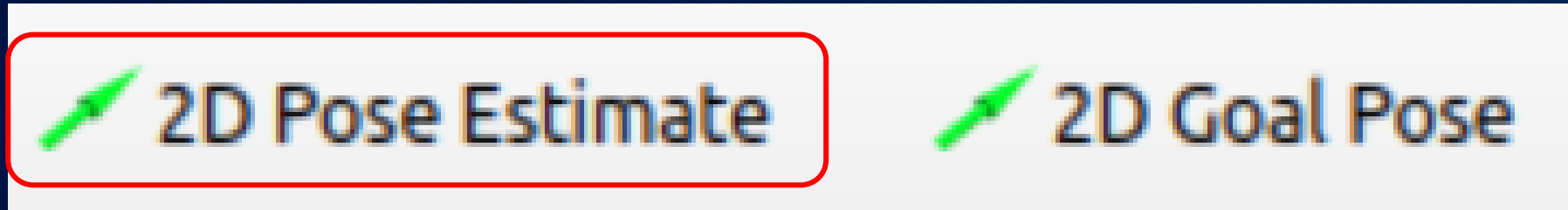


Selecting local costmap



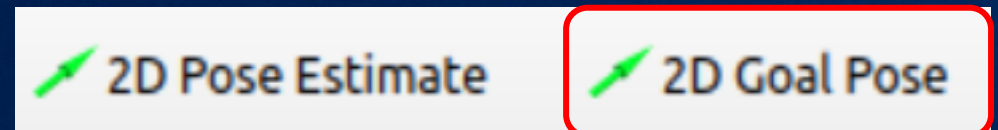
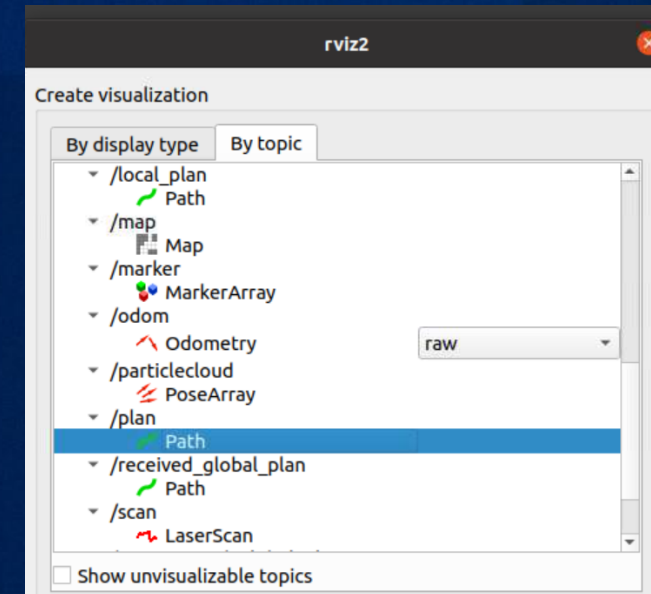
# Localization & Navigation

- Set initial pose estimate from rviz '2D Pose Estimate' on your initial robot location.



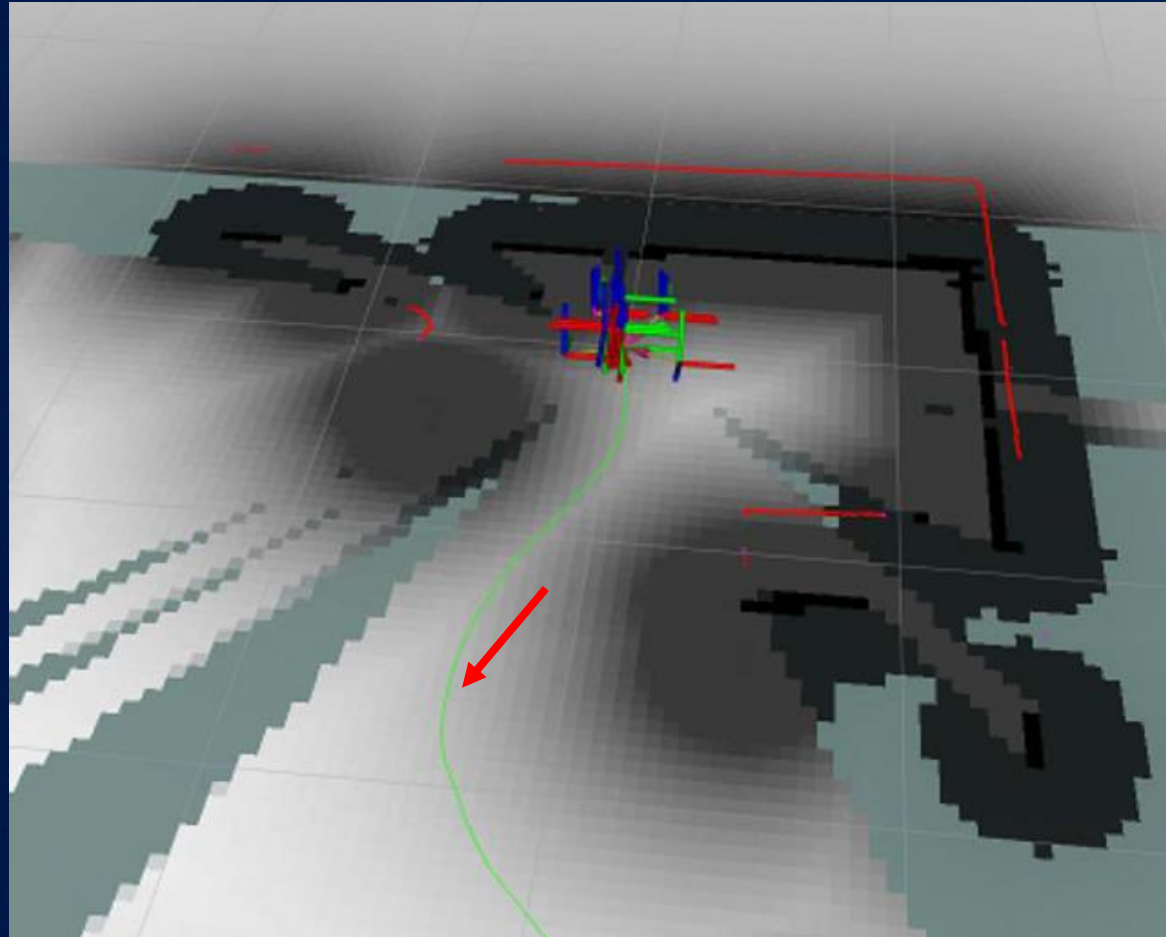
# Localization & Navigation

- To check whether the set pose is accurate or not, display `<ROBOT_NAMESPACE>/plan` topic messages and check if it aligns with the map.
- To navigate the robot, set your end location with '2D goal pose' in RVIZ2.



# Localization & Navigation

- Ensure the green line is available after selecting your goal pose. The robot will navigate itself to the targeted location.

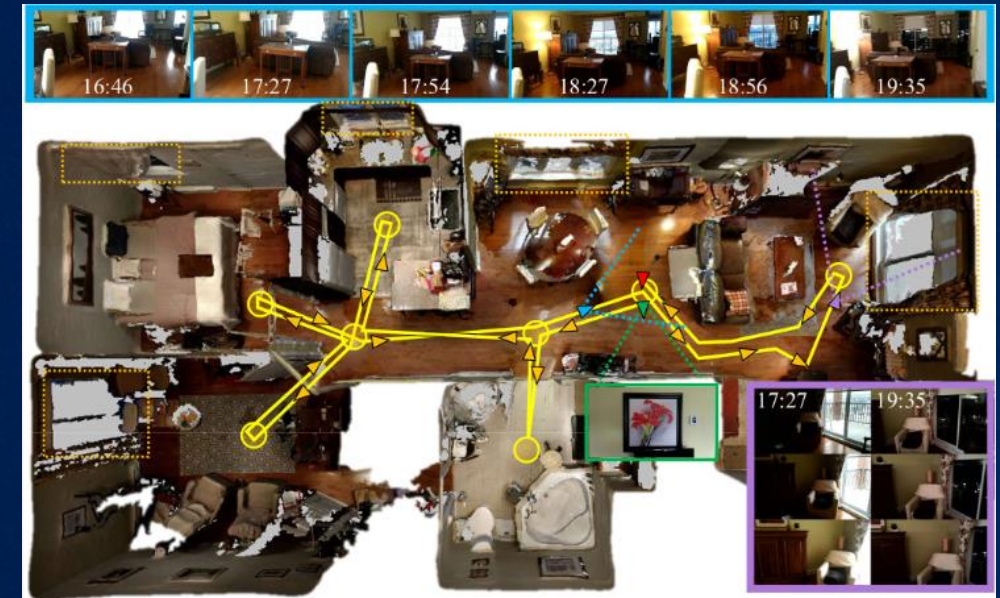




# 3D mapping – RTAB-Map

# What is RTAB-Map

- RTAB-Map (Real-Time Appearance-Based Mapping) is a popular open-source SLAM (Simultaneous Localization and Mapping) algorithm used in robotics for creating 3D maps of environments in real-time. It uses a combination of visual odometry, loop closure detection, and graph optimization to build highly accurate and detailed maps..
- Some key features of RTAB-Map:
  - Real-time mapping: RTAB-Map can generate 3D maps in real-time, making it suitable for real-world robotics applications.
  - Appearance-based mapping: RTAB-Map uses a unique approach to mapping based on visual features, which allows it to build highly accurate and detailed maps even in challenging environments.
  - Loop closure detection: RTAB-Map can detect when a robot revisits a previously explored location and use this information to improve map accuracy.
  - Graph optimization: RTAB-Map uses graph optimization techniques to refine the map and improve its accuracy over time.
  - Open-source: RTAB-Map is an open-source algorithm, meaning it is free to use and can be modified and improved by the robotics community.



3D map generated by RTAB-Map in an indoor environment



# 3D mapping – RTAB-Map

- Open a new terminal (1<sup>st</sup> terminal)
- Run command below (Replace the highlighted green text with your ROBOT\_NAMESPACE environment parameter):

```
$ ros2 run rtabmap_slam rtabmap --ros-args -r odom:=odom/amr  
-r rgb/image:=camera/color/image_raw -r  
depth/image:=camera/aligned_depth_to_color/image_raw -r  
rgb/camera_info:=camera/color/camera_info -r odom:=odom/amr -  
r /tf:=tf -r /tf_static:=tf_static -r __ns:=/robot_ns
```

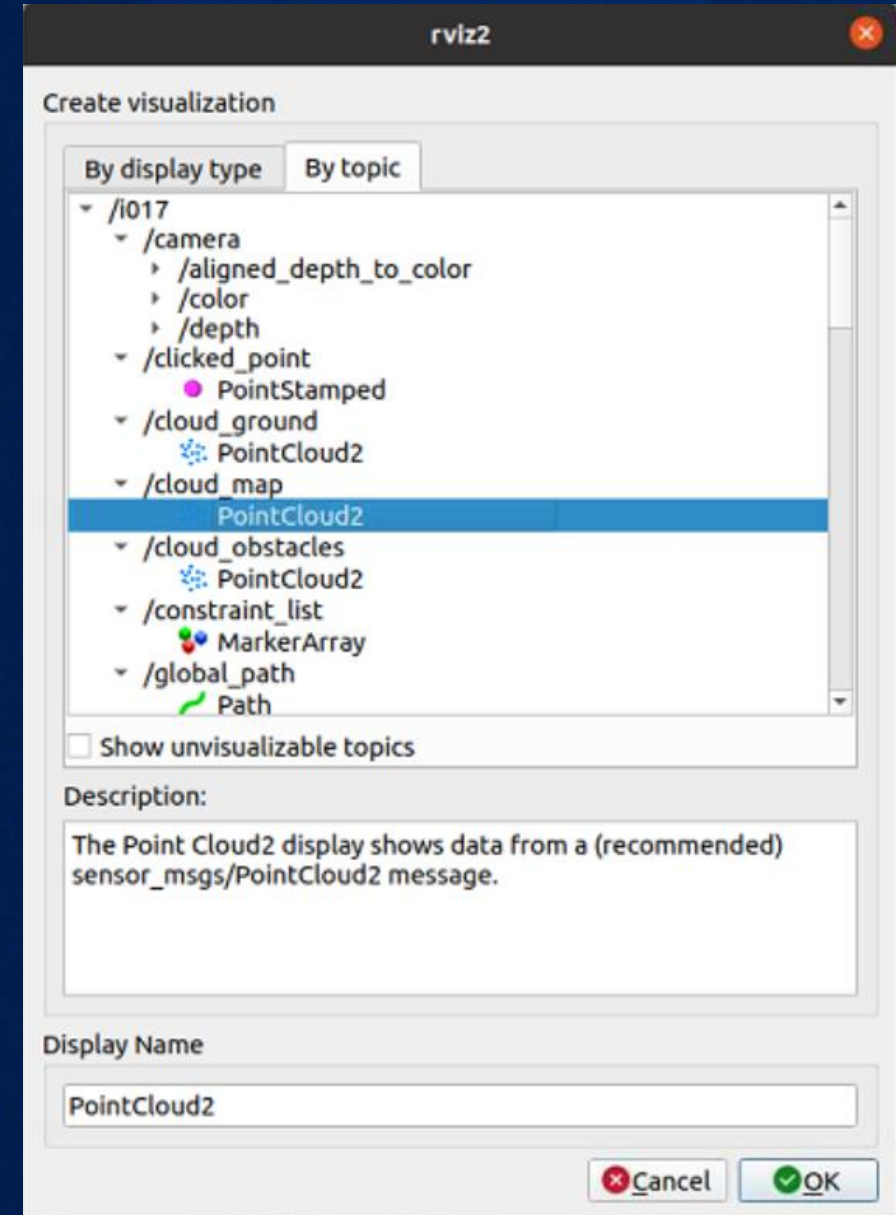
*Note: each \_\_ns:= must change to your own ROBOT\_NAMESPACE*



# 3D mapping – RTAB-Map

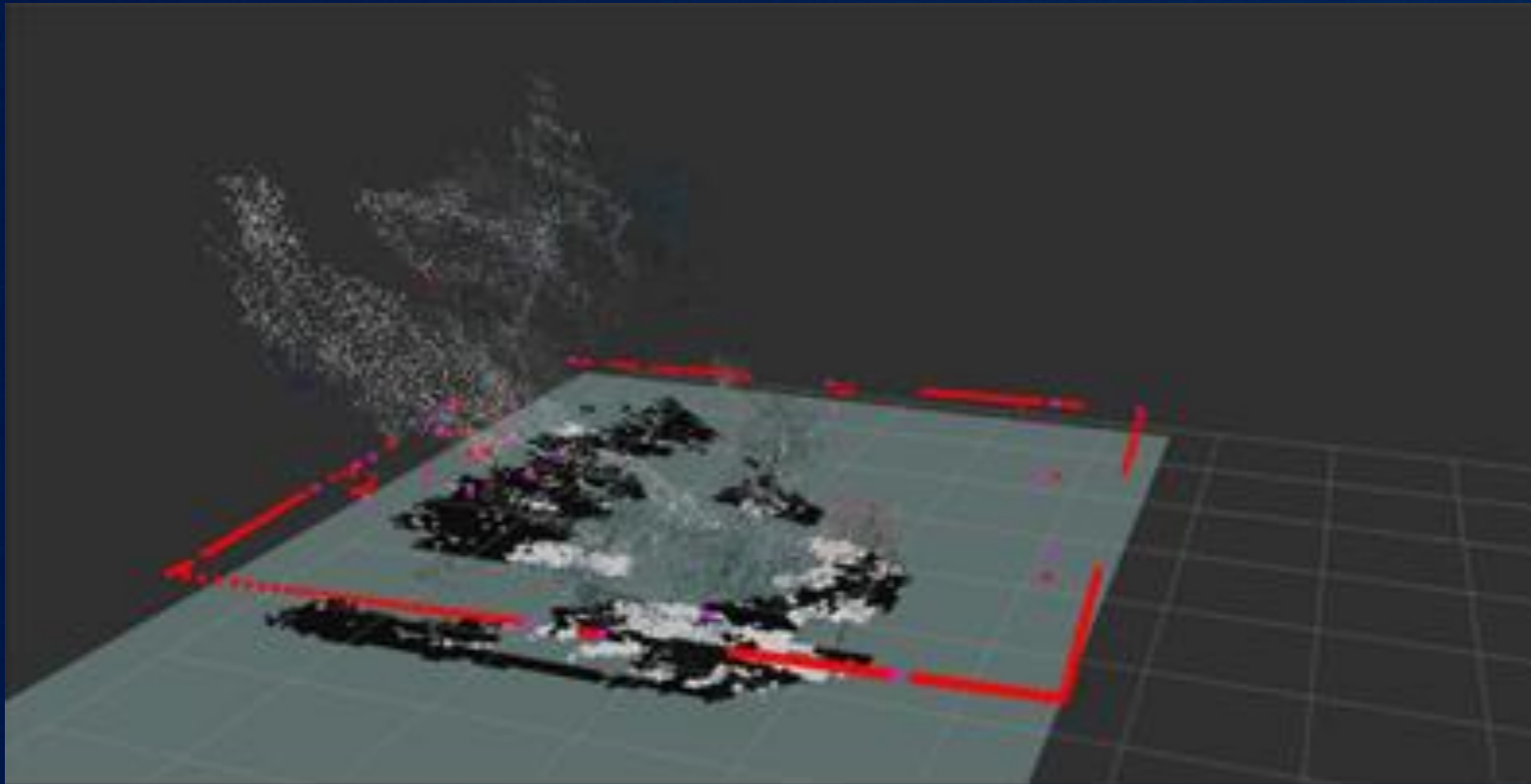
- View the generated 3D map in rviz with the topic `<ROBOT_NAMESPACE>/cloud_map`.

```
$ rviz2 --ros-args -r  
_ns:=/ <robot_ns> -r /tf:=tf -r  
/tf_static:=tf_static -r  
/clicked_point:=clicked_point -r  
/goal_pose:=goal_pose -r  
/initialpose:=initialpose
```



# 3D mapping – RTAB-Map

- The map will display as below:



# Link

## 1. RTAB-Map:

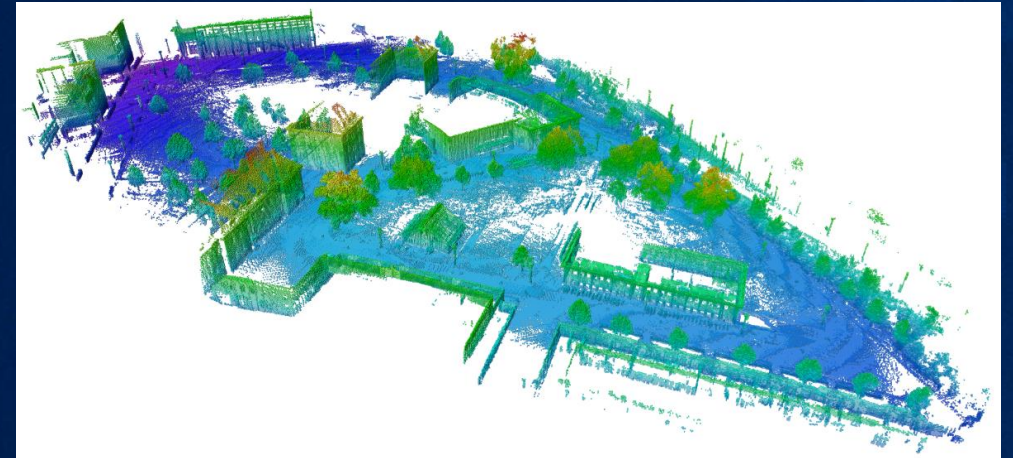
[http://introlab.github.io/rtabmap/#:~:text=RTAB%2DMap%20\(Real%2DTime,location%20or%20a%20new%20location.](http://introlab.github.io/rtabmap/#:~:text=RTAB%2DMap%20(Real%2DTime,location%20or%20a%20new%20location.)



# Octomap generation with FastMapping & RTAB-Map

# What is FastMapping

- Fast Mapping is a Robot Operating System (ROS) package for real-time scene modeling from sequential depth images.
- It can construct and maintain a volumetric map from a moving RGB-D camera, like octomap\_mapping but runs much faster with Intel® OneAPI Software Library Optimization.



Example of 3D Octomap from an AMR



# Octomap generation with FastMapping

- *Ensure that the RTAB-Map from the last session (1<sup>st</sup> terminal) is still running.*
- *Ensure that the fast\_mapping package has been installed properly.*



# Octomap generation with Fast Mapping

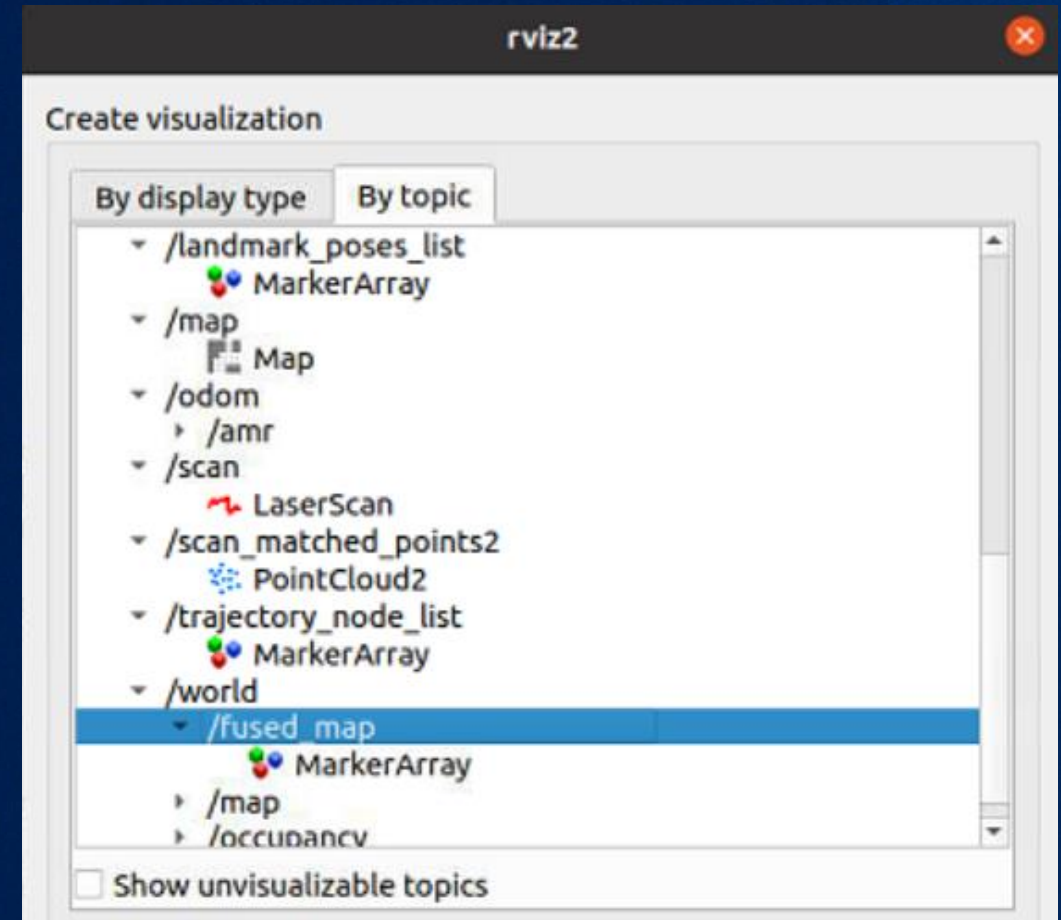
- Open a new terminal (2<sup>nd</sup> terminal) and run command below:

```
$ ros2 run fast_mapping fast_mapping_node --ros-args -r
/camera/aligned_depth_to_color/camera_info:=camera/aligned_de
pth_to_color/camera_info -r
/camera/aligned_depth_to_color/image_raw:=camera/aligned_dept
h_to_color/image_raw -r
/camera_left/aligned_depth_to_color/image_raw:=camera_left/al
igned_depth_to_color/image_raw -r
/camera_rear/aligned_depth_to_color/image_raw:=camera_rear/al
igned_depth_to_color/image_raw -r
/camera_right/aligned_depth_to_color/image_raw:=camera_right/
aligned_depth_to_color/image_raw -r /tf:=tf -r
/tf_static:=tf_static -r /world/fused_map:=world/fused_map -r
/world/map:=world/map -r /world/occupancy:=world/occupancy -r
__ns:=/<robot_ns>
```

Note: each `__ns:=` must change to your own `ROBOT_NAMESPACE`

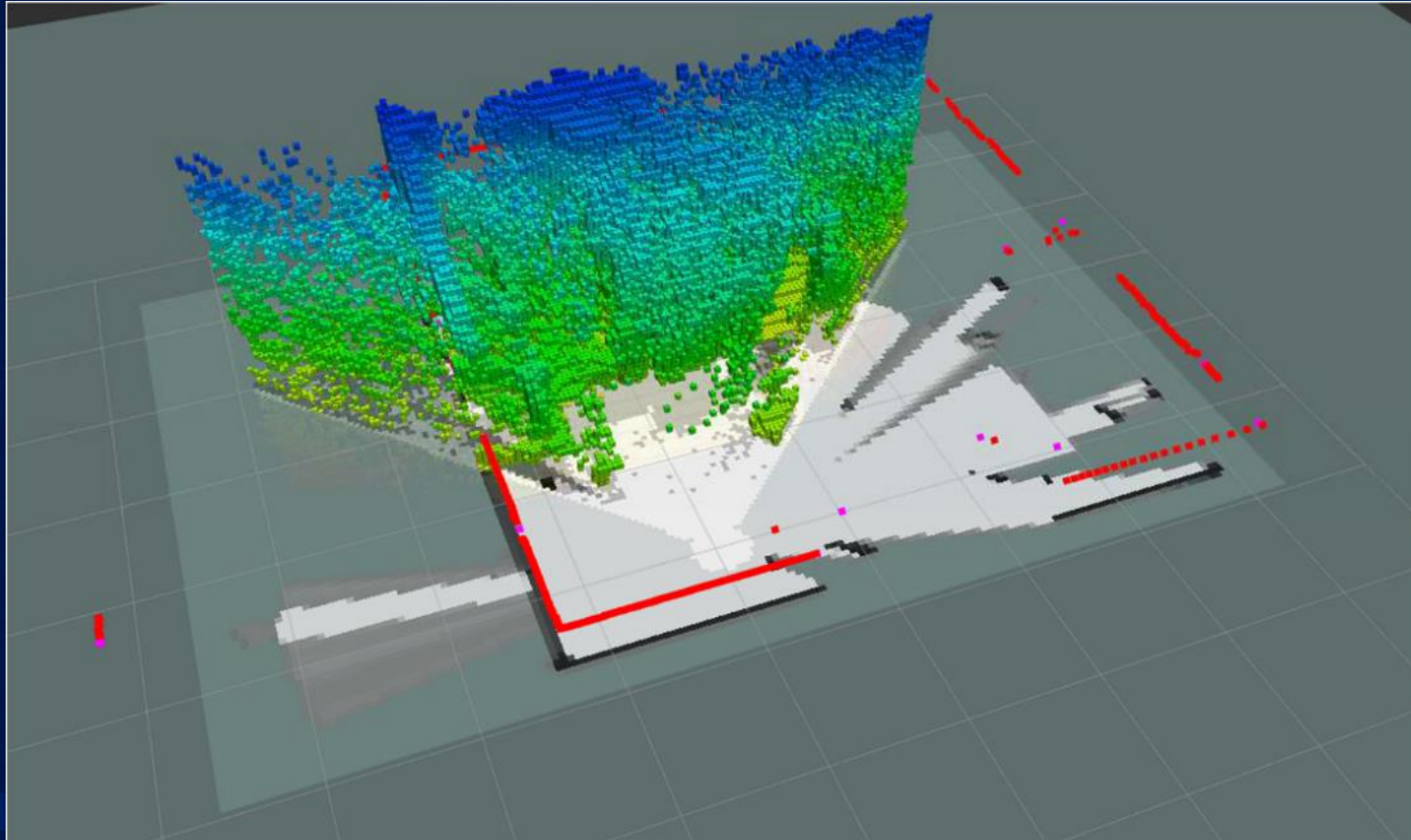
# Octomap generation with Fast Mapping

- In RVIZ2, select the topic of Octomap generated by Fast Mapping
- View the occupancy grid in rviz with the topic <ROBOT\_NAMESPAC>/world/fused\_map



# Octomap generation with Fast Mapping

- The map will display as below:





# Link

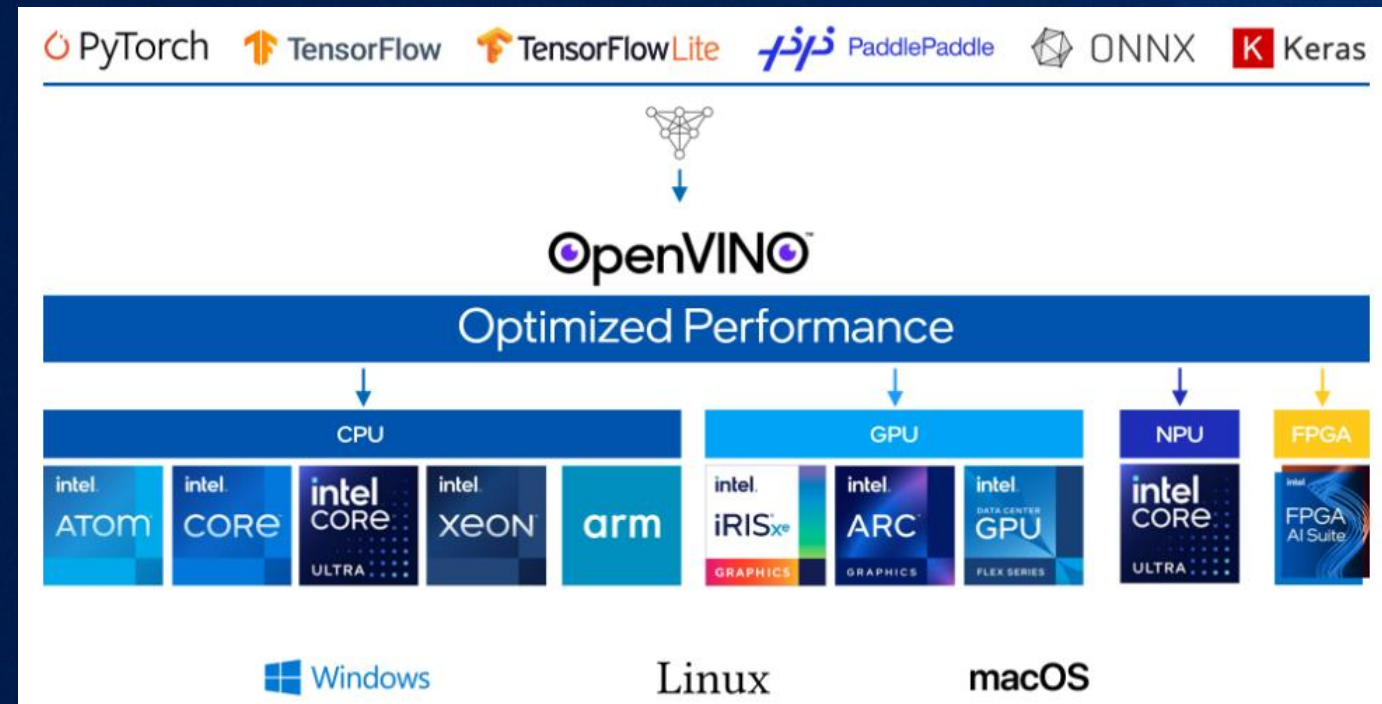
1. OctoMap: <https://octomap.github.io/>

# Training Object Detection

# What is Intel® OpenVino?



- OpenVINO stands for **Open Visual Inference and Neural network Optimization**, and it is a toolkit developed by Intel to accelerate deep learning inference on various hardware platforms.
  - Designed to optimize and deploy deep learning models quickly and efficiently
  - Supports a wide range of frameworks (TensorFlow, PyTorch, Caffe, etc.) and hardware (CPU, GPU, FPGA, VPU, etc.)
  - Provides pre-trained models and tools for model optimization, conversion, and deployment
- Wide range of pre-trained, deep learning models and demo applications that provide full application templates to facilitate the deployment process.





# Creating a YOLOv8 OpenVINO Object Detection Package

- Install the necessary python packages

- Python packages:

```
$ python3 -m pip install numpy pandas openvino-dev nncf onnx
$ python3 -m pip install ultralytics==8.0.43
$ python3 -m pip install openvino-dev==2023.2
```

- Creating a workspace directory for Object Detection Package.

```
$ mkdir -p ~/workspace/robofun-1.0/object-detection-ws/src
```

- Create a ROS2 package

```
$ source /opt/ros/humble/setup.bash
$ cd ~/workspace/robofun-1.0/object-detection-ws/src
$ ros2 pkg create --build-type ament_python object_detection
```

# Creating OpenVINO Object Detection ROS2 Node

- Create a file name: *object\_detection\_node.py* in *~/workspace/robofun-1.0/object-detection-ws/src/object\_detection/object\_detection*.

```
$ gedit ~/workspace/robofun-1.0/object-detection-  
ws/src/object_detection/object_detection/object_detection_node.py
```

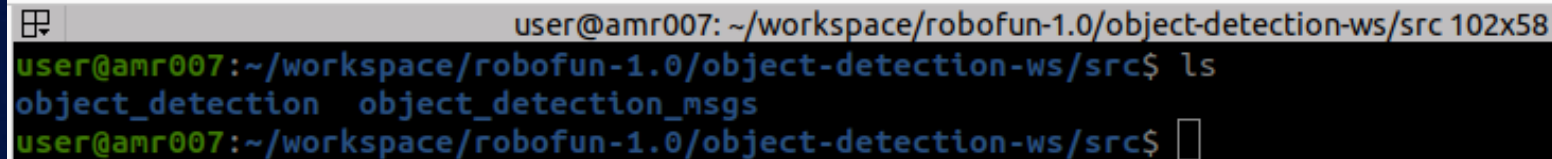
- Copy the code from *object\_detection\_node.txt* into the file.
- Modify the line 21 in *~/workspace/robofun-1.0/object-detection-ws/src/object\_detection/setup.py* to add the entry point for node.

```
entry_points={  
    'console_scripts': [  
        'object_detection_node =  
object_detection.object_detection_node:main'  
    ],  
},
```

# Create a custom msg for detected object

- Create a ROS2 package for custom message for detected object.

```
$ source /opt/ros/humble/setup.bash
$ cd ~/workspace/robofun-1.0/object-detection-ws/src
$ ros2 pkg create --build-type ament_cmake object_detection_msgs
```



A terminal window titled "user@amr007: ~/workspace/robofun-1.0/object-detection-ws/src 102x58". The prompt is "user@amr007:~/workspace/robofun-1.0/object-detection-ws/src\$". The user enters "ls", and the output is "object\_detection object\_detection\_msgs". The prompt then changes to "user@amr007:~/workspace/robofun-1.0/object-detection-ws/src\$" with a cursor.



# Create a custom msg for detected object

- Create a msg folder in the package

```
$ mkdir ~/workspace/robofun-1.0/object-detection-ws/src/object_detection_msgs/msg
```

- Create the Object.msg and Objects.msg in the

```
$ gedit ~/workspace/robofun-1.0/object-detection-ws/src/object_detection_msgs/msg/Object.msg
```

```
int16 class_id      # class id
int16 id            # track id
string label        # object class
float32 confidence  # confidence of result

float32 x           # normalized value of box top-left point coordinate x
float32 y           # normalized value of box top-left point coordinate y
float32 height      # normalized value of box height
float32 width       # normalized value of box width

float32 vel_x       # normalized velocity of box top-left point coordinate x
float32 vel_y       # normalized velocity of box top-left point coordinate y
```

# Create a custom msg for detected object

- Create the Object.msg and add Objects.msg in the msg directory:

```
$ gedit ~/workspace/robofun-1.0/object-detection-ws/src/object_detection_msgs/msg/Objects.msg
```

```
std_msgs/Header header  
Object[] objects
```

# Create a custom msg for detected object

- In *object\_detection\_msgs/CMakeLists.txt*, add the following lines:

```
$ gedit ~/workspace/robofun-1.0/object-detection-ws/src/object_detection_msgs/CMakeLists.txt
```

After line 9:

```
find_package(rosidl_default_generators REQUIRED)  
find_package(std_msgs REQUIRED)
```

After line 25:

```
rosidl_generate_interfaces(${PROJECT_NAME}  
  "msg/Object.msg"  
  "msg/Objects.msg"  
  DEPENDENCIES std_msgs  
)
```

After adding in all the lines, click save and exit the text editor.



# Create a custom msg for detected object

Open the package.xml file to add the dependencies for building the message package

```
$ gedit ~/workspace/robofun-1.0/object-detection-ws/src/object_detection_msgs/package.xml
```

Add the following lines after line 10 :

```
<depend>std_msgs</depend>  
<buildtool_depend>roscpp</buildtool_depend>  
<member_of_group>roscpp</member_of_group>
```

Click save and exit the text editor after inputting the changes.

# Compile and Run OpenVINO Object Detection App

## Compile the application

```
$ cd ~/workspace/robofun-1.0/object-detection-ws  
$ colcon build --symlink-install
```

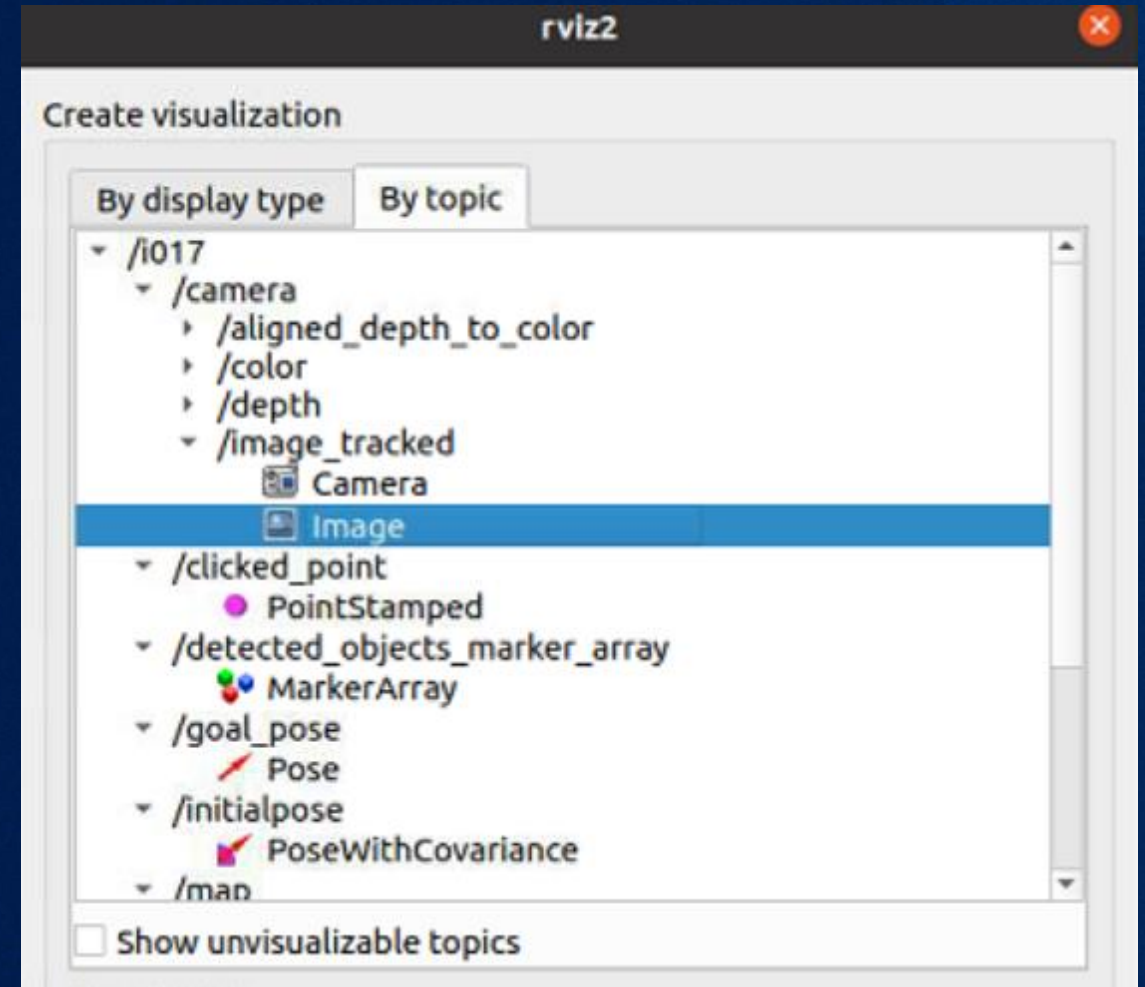
## Run the OpenVINO application after compiling

```
$ cd ~/workspace/robofun-1.0/object-detection-ws  
$ source install/setup.bash  
$ export ROS_DOMAIN_ID=<ROBOT_DOMAIN_ID>  
$ ros2 run object_detection object_detection_node --ros-args -r __ns:=/<ROBOT-NAMESPACE>
```

\*Note: Every time you enter a new terminal, remember to export ROS\_DOMAIN\_ID to the environment.

# Object Detection

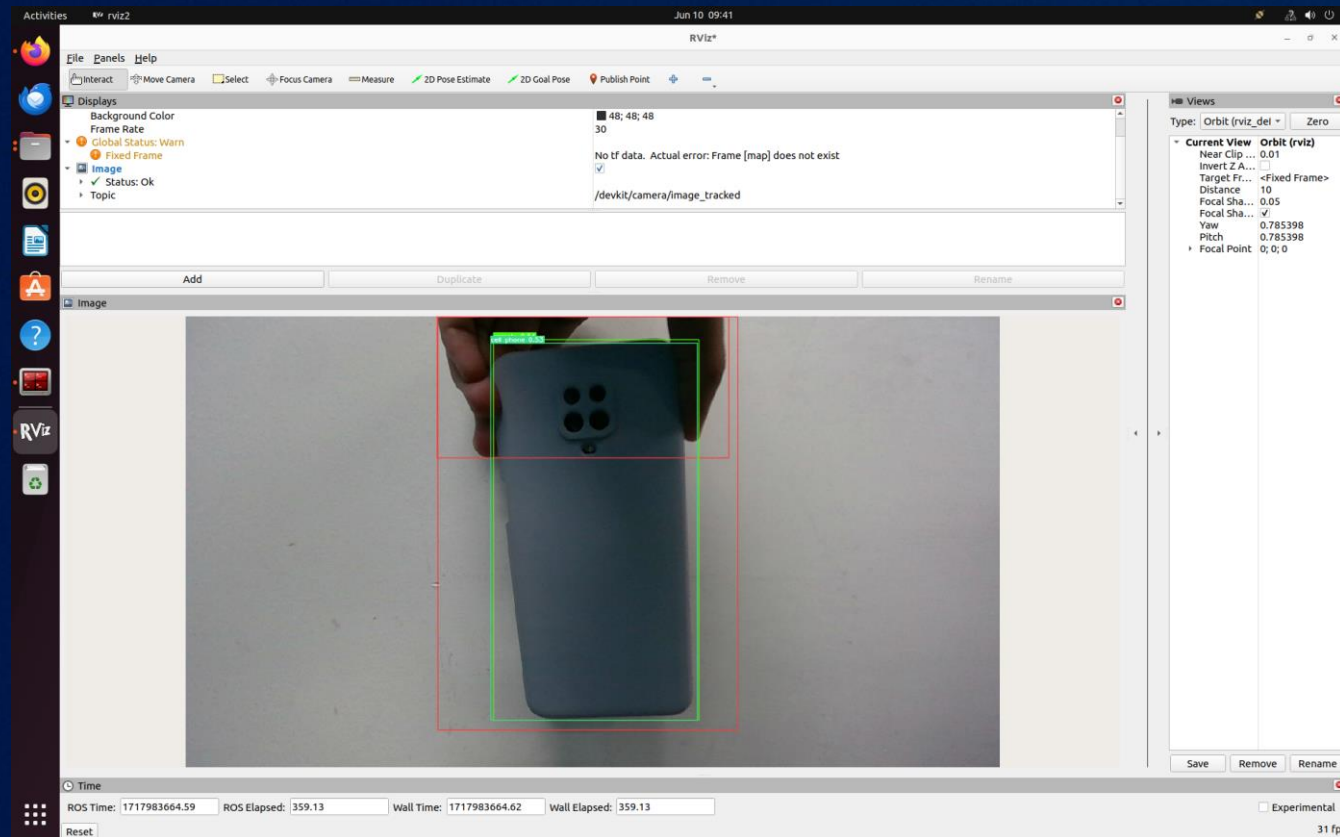
- Add the detected object frame in RVIZ with the topic `<ROBOT_NAMESPAC>/camera/image_tracked/Image`.





# Object Detection

- The image will display as below:



# Link

1. OpenVINO: [link](#)
2. YOLOv8: [link](#)

Training

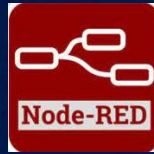
# Node-RED Visualization



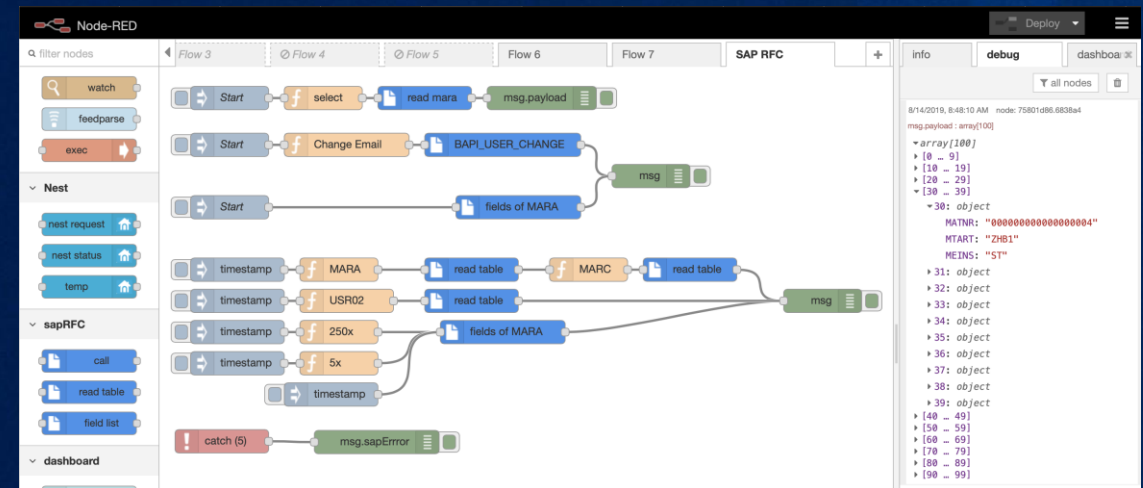
# Outcome of session

- Learn how to create a ROS2 package that publishes the base64 image data to Node-RED.
- Create a ROS2 message package and use the created ROS2 message to publish detected object data.
- Visualize the object detection stream data on the Node-RED dashboard.
- Save the detected object using the Node-RED dashboard.

# What is Node-red



- open-source visual programming tool used for connecting IoT (Internet of Things) devices, APIs, and online services.
- web-based flow editor that allows users to create, edit and manage "flows," which are sequences of events that define how data is collected, processed, and transmitted between different systems.
- has a built-in dashboard feature that allows users to create web-based dashboards to display data in real time.
- useful for monitoring sensors and other IoT devices, displaying data analytics, and creating user interfaces for control systems.



Example of node-red dashboard

# Node-RED Visualization

## Node-red and mosquitto installation

- Ensure the Scuttle driver software is running (Refer to Day1 materials to bringup scuttle platform container using docker compose)
- Ensure that Object Detection is running. (Refer to Day3 materials to run Object Detection)
- Update the apt repository.

```
$ sudo apt update
```

- Install nodejs 18.x, npm and mosquitto package.

```
$ cd ~/workspace  
$ curl -sL https://deb.nodesource.com/setup_18.x | sudo bash -  
$ sudo apt install -y nodejs mosquitto
```



# Node-RED Visualization

## Node-red and mosquitto installation

- The mosquitto package should now be load on your server. Confirm the status by using:

```
$ sudo systemctl status mosquitto
```

```
user@anrk1017:~$ sudo systemctl status mosquitto
[sudo] password for user:
● mosquitto.service - Mosquitto MQTT v3.1/v3.1.1 Broker
   Loaded: loaded (/lib/systemd/system/mosquitto.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2023-02-08 10:26:18 +08; 6h ago
     Docs: man:mosquitto.conf(5)
           man:mosquitto(8)
    Main PID: 1280 (mosquitto)
      Tasks: 3 (limit: 38145)
    Memory: 2.5M
    CGroup: /system.slice/mosquitto.service
            └─1280 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

Feb 08 10:26:18 anrk1017 systemd[1]: Starting Mosquitto MQTT v3.1/v3.1.1 Broker...
Feb 08 10:26:18 anrk1017 mosquitto[1280]: [ 11.406979]-DLT- 1280-INFO -FIFO /tmp/dlt cannot be opened. Retrying later...
Feb 08 10:26:18 anrk1017 systemd[1]: Started Mosquitto MQTT v3.1/v3.1.1 Broker.
user@anrk1017:~$
```

Check the package is loaded and active

# Node-RED Visualization

## Node-red and mosquitto installation

- The mosquitto package has three services (Information Only. Only need to run if has issues in launching or starting mosquitto service)

```
$ sudo systemctl stop mosquitto # stop mosquitto service  
$ sudo systemctl start mosquitto # start mosquitto service  
$ sudo systemctl restart mosquitto # restart mosquitto service
```

- Install the necessary Python packages.

```
$ pip3 install pybase64 paho-mqtt numpy
```

# Node-RED Visualization

Node-red and mosquitto installation

- Install node-red package using npm

```
$ sudo npm install -g --unsafe-perm node-red
```

```
user@anrk1t017:~/Desktop$ sudo npm install -g --unsafe-perm node-red
/usr/bin/node-red-pl -> /usr/lib/node_modules/node-red/bin/node-red-pl
/usr/bin/node-red -> /usr/lib/node_modules/node-red/red.js
+ node-red@3.0.2
added 1 package from 1 contributor and updated 6 packages in 32.158s
user@anrk1t017:~/Desktop$
```

- Start node-red server

```
$ node-red
```



# Node-RED Visualization

Node-red and mosquitto installation

- Click the link to enter Node-RED.

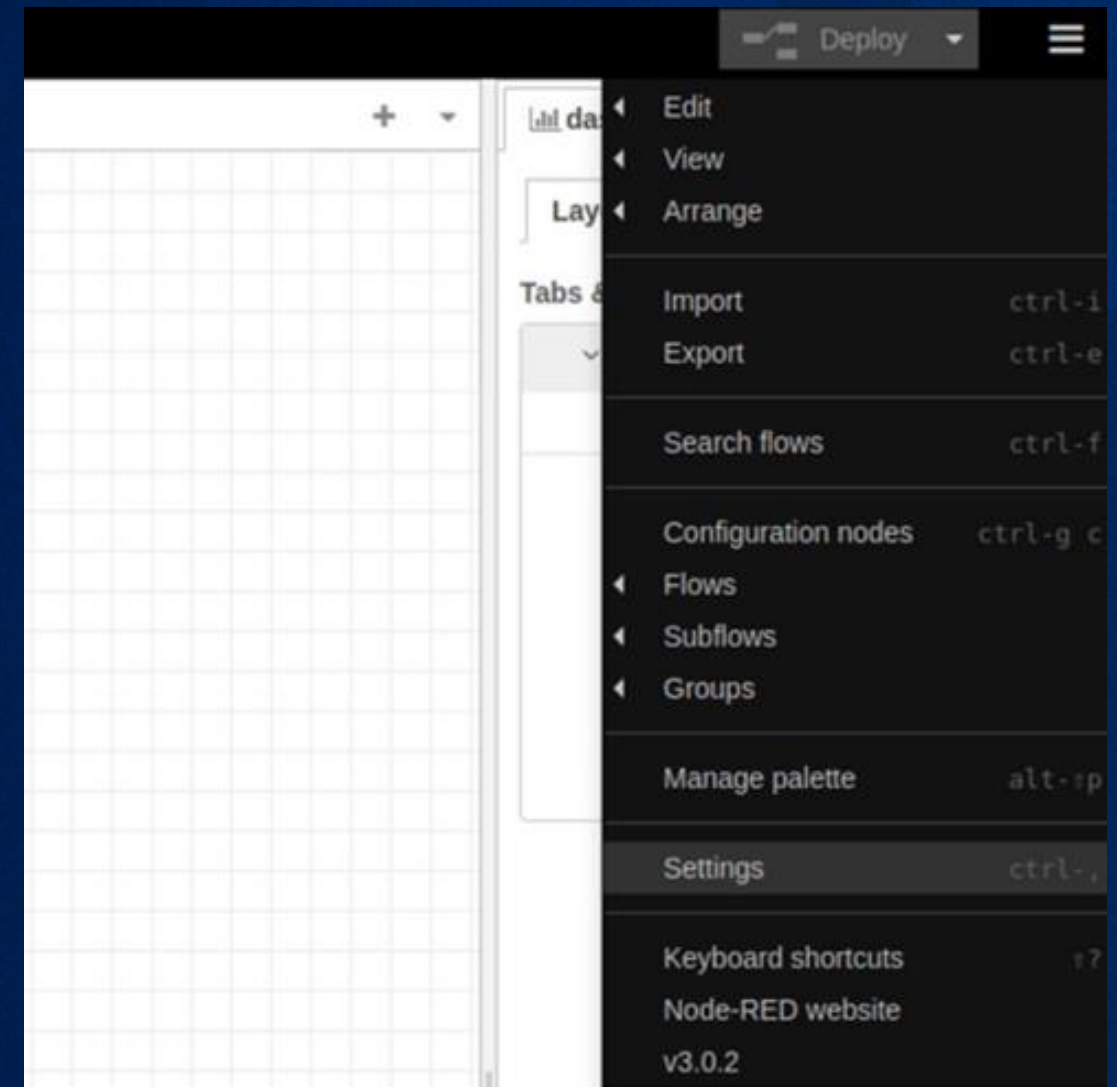
```
Welcome to Node-RED
*****

1 Feb 13:12:25 - [info] Node-RED version: v3.0.2
1 Feb 13:12:25 - [info] Node.js version: v14.21.2
1 Feb 13:12:25 - [info] Linux 5.15.0-58-generic x64 LE
1 Feb 13:12:25 - [info] Loading palette nodes
1 Feb 13:12:26 - [info] Dashboard version 3.3.1 started at /ui
1 Feb 13:12:26 - [info] Settings file : /home/user/.node-red/settings.js
1 Feb 13:12:26 - [info] Context store : 'default' [module-memory]
1 Feb 13:12:26 - [info] User directory : /home/user/.node-red
1 Feb 13:12:26 - [warn] Projects disabled : editorTheme.projects.enabled=false
1 Feb 13:12:26 - [info] Flows file : /home/user/.node-red/flows.json
1 Feb 13:12:26 - [info] Server now running at http://127.0.0.1:1880/
1 Feb 13:12:26 - [warn]
```

# Node-RED Visualization

Node-red and mosquitto installation

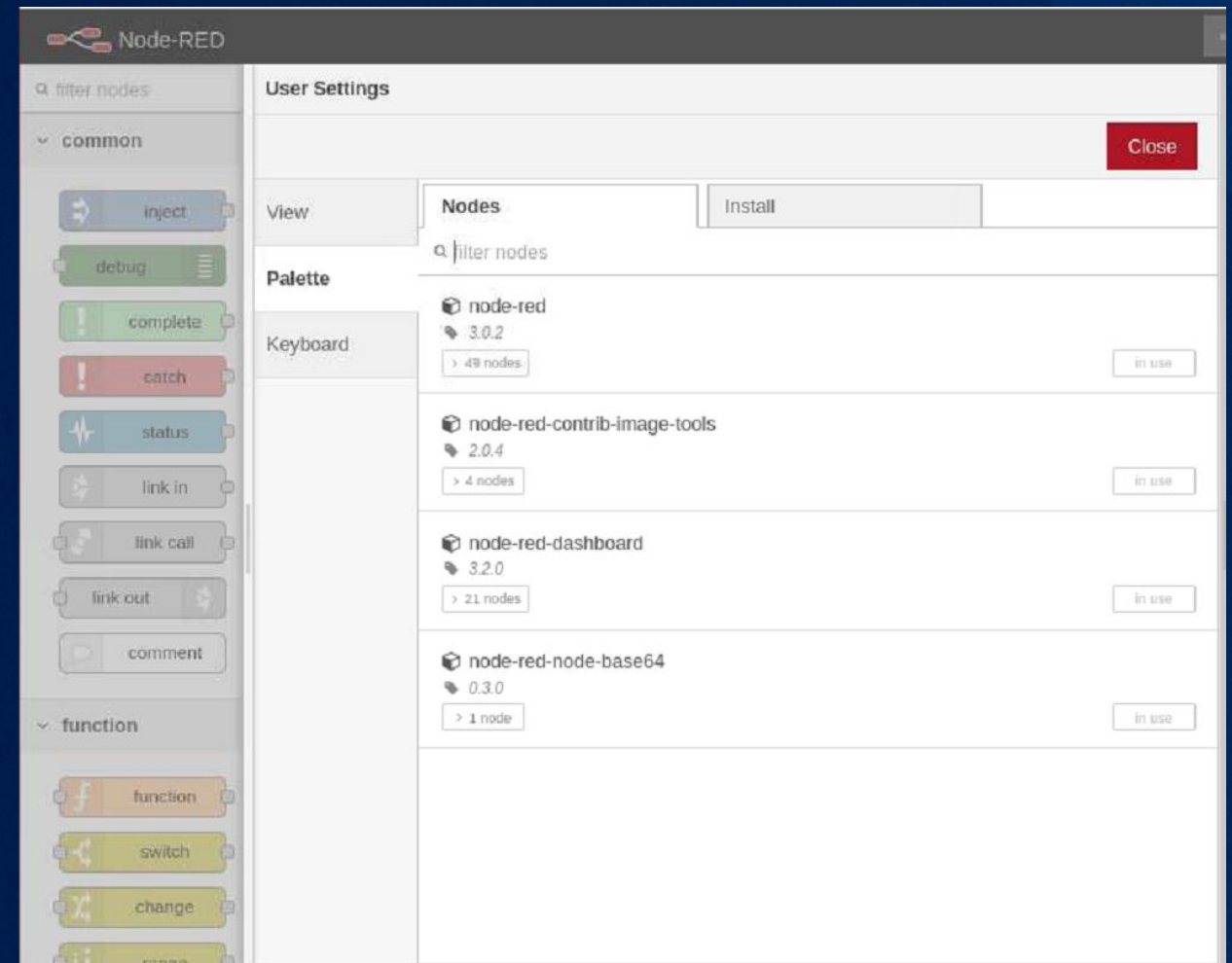
- Click 3 bar on the right side of your screen -> Manage palette -> Install



# Node-RED Visualization

Node-red and mosquitto installation

- Install the following nodes in the node-red palettes menu:
  - node-red-contrib-image-tools
  - node-red-node-base64
  - node-red-dashboard





# Node-RED Visualization

## Object Detection Custom Message Types

- In new terminal, enter src directory in workspace

```
$ source /opt/ros/humble/setup.bash  
$ cd ~/workspace/robofun-1.0/object-detection-ws/src
```

# Node-RED Visualization

## ROS2 MQTT package

To facilitate the communication between the node-red dashboard and ei-for-amr stack, a package is created to republish the required topics from ei-for-amr stack to node-red using mqtt

- Inside src directory, create a new ros2 package named 'scuttle\_mqtt'

```
$ cd ~/workspace/robofun-1.0/object-detection-ws/src
$ ros2 pkg create --build-type ament_python scuttle_mqtt
```

```
user@devkit:~$ cd workspace/robofun-1.0/object-detection-ws/src/
user@devkit:~/workspace/robofun-1.0/object-detection-ws/src$ ros2 pkg create --build-type ament_python scuttle_mqtt
going to create a new package
package name: scuttle_mqtt
destination directory: /home/user/workspace/robofun-1.0/object-detection-ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['user <user@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_python
dependencies: []
creating folder ./scuttle_mqtt
creating ./scuttle_mqtt/package.xml
creating source folder
creating folder ./scuttle_mqtt/scuttle_mqtt
creating ./scuttle_mqtt/setup.py
creating ./scuttle_mqtt/setup.cfg
creating folder ./scuttle_mqtt/resource
creating ./scuttle_mqtt/resource/scuttle_mqtt
creating ./scuttle_mqtt/scuttle_mqtt/__init__.py
creating folder ./scuttle_mqtt/test
creating ./scuttle_mqtt/test/test_copyright.py
creating ./scuttle_mqtt/test/test_flake8.py
creating ./scuttle_mqtt/test/test_pep257.py

[WARNING]: Unknown license 'TODO: License declaration'. This has been set in the package.xml, but no LICENSE file has been created.
It is recommended to use one of the ament license identifiers:
Apache-2.0
BSL-1.0
BSD-2.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
```

# Node-RED Visualization

- Copy the script from the *image\_publisher.txt* into `~workspace/node-red/scuttle_mqtt/scuttle_mqtt` directory as `image_publisher.py`:

```
$ cd ~/workspace/robofun-1.0/object-detection-  
ws/src/scuttle_mqtt/scuttle_mqtt  
$ gedit image_publisher.py
```

- Note: the `image_publisher.txt` is shared in google drive



# Node-RED Visualization

- Modify setup.py to include the entry point for the image\_publisher node.

```
$ cd ~/workspace/robofun-1.0/object-detection-ws/src/scuttle_mqtt/  
$ gedit setup.py
```

- Add the following line after line 22:

```
'mqtt_image_publisher = scuttle_mqtt.image_publisher:main'
```

```
20 tests_require=['pytest'],  
21 entry_points={  
22     'console_scripts': [  
23         'mqtt_image_publisher = scuttle_mqtt.image_publisher:main'  
24     ],  
25 },  
26 )
```

- Click save and exit the text editor after finish editing.

# Node-RED Visualization

- Build the package for scuttle\_mqtt and object\_detection directory.

```
$ cd ~/workspace/robofun-1.0/object-detection-ws/  
$ source /opt/ros/humble/setup.bash  
$ colcon build
```

```
user@devkit:~/workspace/robofun-1.0/object-detection-ws$ colcon build  
Starting >>> object_detection  
Starting >>> object_detection_msgs  
Starting >>> scuttle_mqtt  
Finished <<< object_detection_msgs [1.49s]  
Finished <<< scuttle_mqtt [1.85s]  
Finished <<< object_detection [1.99s]  
Summary: 3 packages finished [2.56s]
```

```
$ source install/setup.bash
```

# Node-RED Visualization

- Before running the mqtt middleware node, ensure that you have already run the object detection node. (Refer to slide 112 for reference)
- Run the mqtt middleware node using the following command:

```
$ source /opt/ros/humble/setup.bash
$ export ROS_DOMAIN_ID=<ROBOT_DOMAIN_ID>
$ export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
$ ros2 run scuttle_mqtt mqtt_image_publisher -r
__ns:=/<robot_ns>
```

Note: each `__ns:=` must change to your own `ROBOT_NAMESPACE`



1

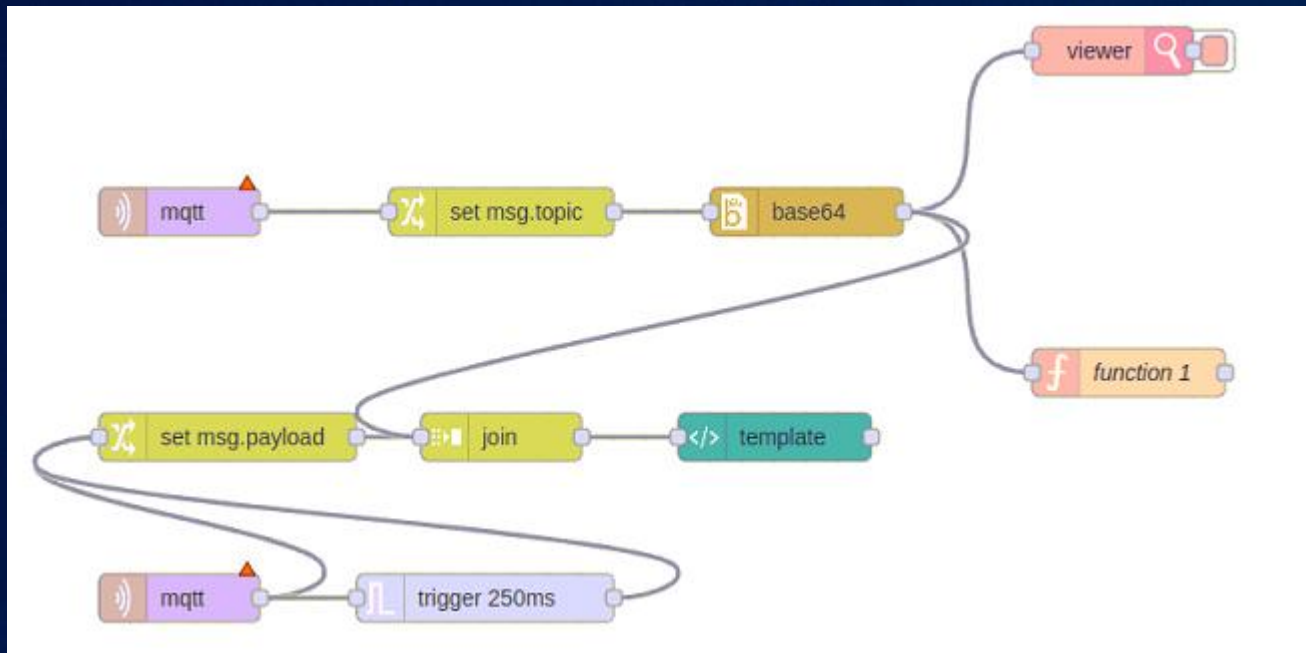


# Node-RED Visualization

Node-red flow 1

Image flow

- Create a flow that subscribes to the mqtt image topic and number of detected objects and visualize the data on a dashboard.
- Add the required node and connection the node based on the diagram below



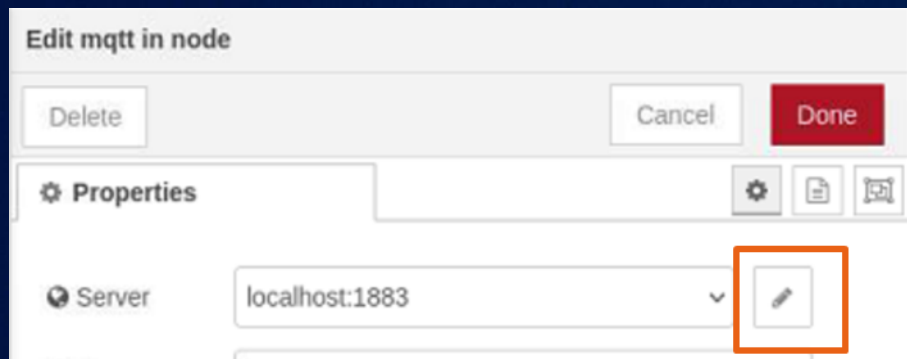
- Tips (Use filter function to search for the nodes effectively)
  - mqtt: *mqtt in*
  - set msg.payload: *change*
  - base64: *base64*
  - viewer: *viewer*
  - function1: *function*
  - join: *join*
  - template: *template*
  - trigger: *trigger*

# Node-RED Visualization

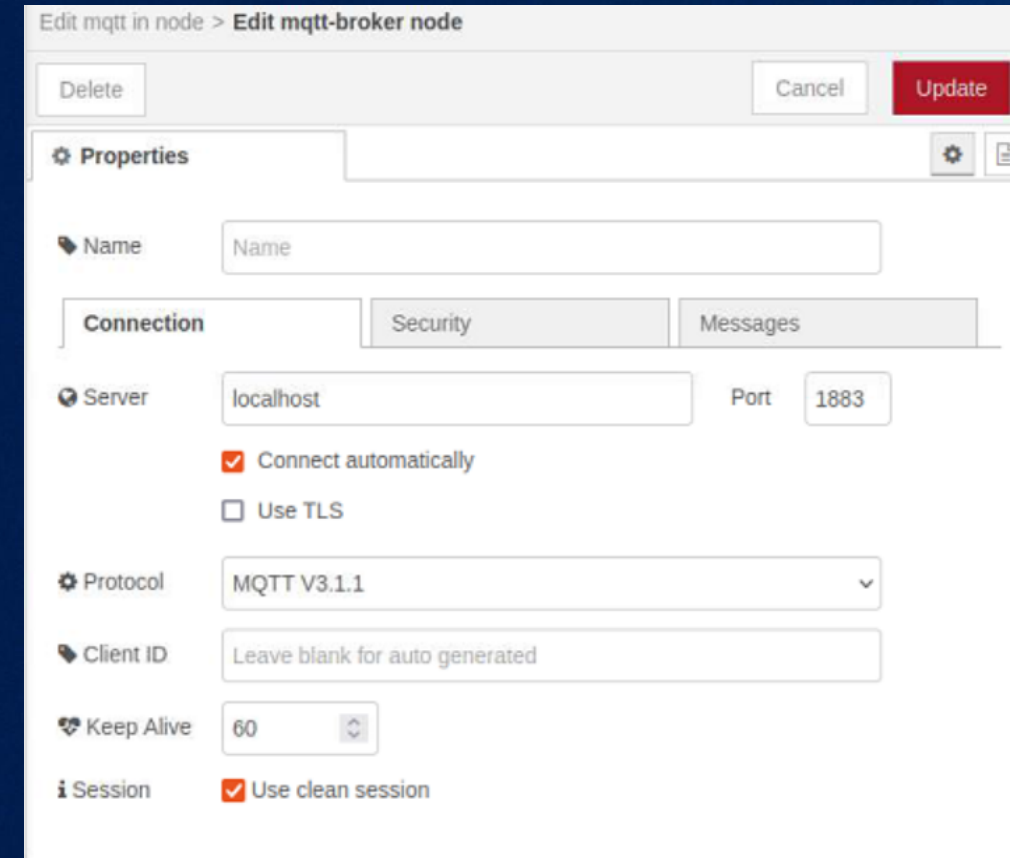
Node-red flow 1  
Image flow



- Click the mqtt node.
- Verify on the server settings as follow to the image given and set it if it is different.
- Click on the Update button if the settings are aligned.



Click on the edit button



Enter the mqtt-broker node



# Node-RED Visualization

Node-red flow 1  
Image flow



- Made the following changes:
  - Change topic section to `'/camera/image'`,
  - QoS set to `0`.
  - Change Output section to *a base64 encoded string*.
- Clicked on *Done* button after the settings are configured.

Edit mqtt in node

Delete Cancel Done

⚙ Properties

📶 Server localhost:1883

Action Subscribe to single topic

📄 Topic /camera/image

📶 QoS 0

➡ Output a Base64 encoded string

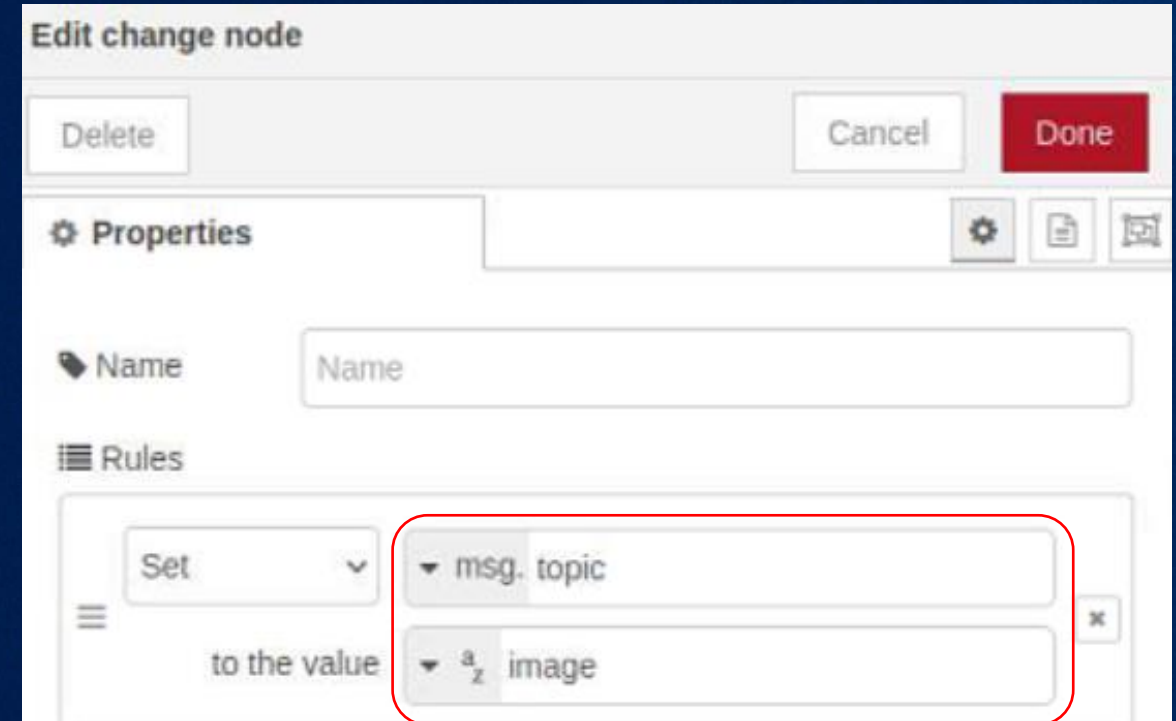
🏷 Name image

# Node-RED Visualization

Node-red flow 1  
Image flow



- On the change node, in the rules section, made the following changes:
  - Change *msg.payload* -> *msg.topic*
  - Set the value to '*image*'.
- This is to enable displaying of the image on the dashboard in the later steps.

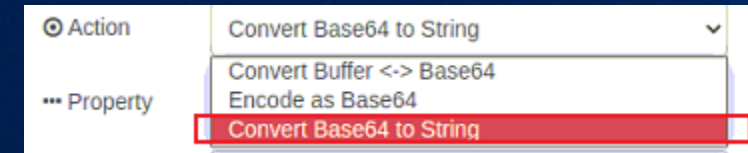


# Node-RED Visualization

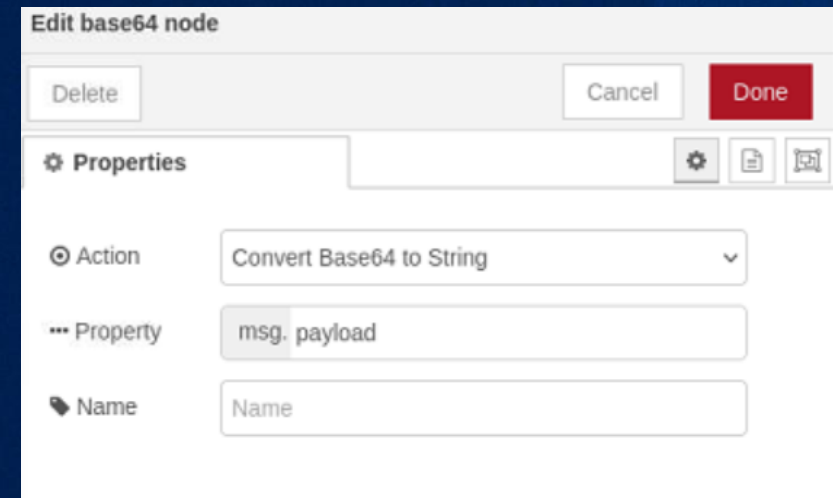
Node-red flow 1  
Image flow



- A base64 node is used to convert the base64 message to a string.
- Made the following changes to the base64 node:
  - In the Action section, change to Convert Base64 to String



Change to Convert Base64 to String



Final configurations

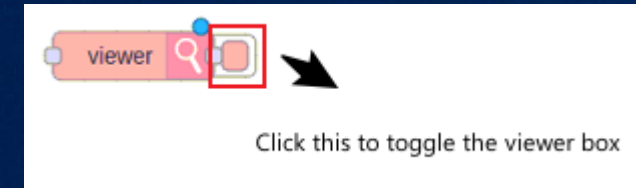


# Node-RED Visualization

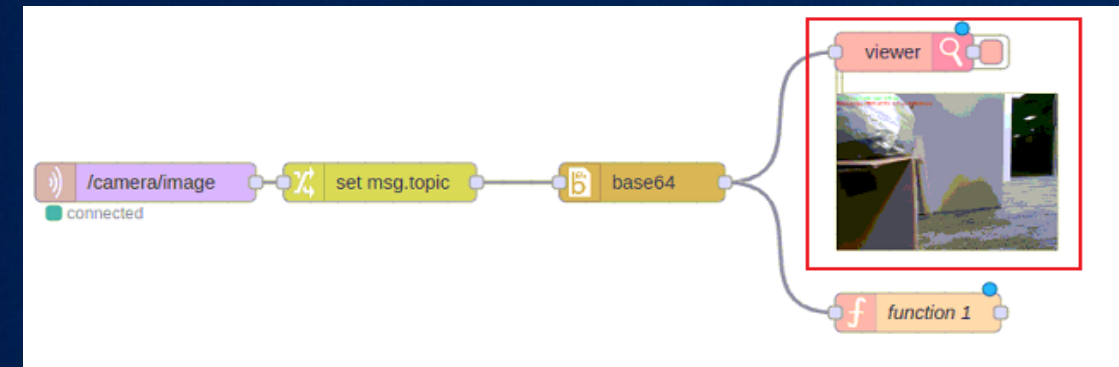
Node-red flow 1  
Image flow



- Verify if the object detection base64 image msg is received correctly using a viewer node.
- You can toggle the viewer screen using the button beside the viewer.
- You should be able to see the object detection stream data on the box opened by the viewer node.



Toggle viewer box



Object detection can be viewed from viewer

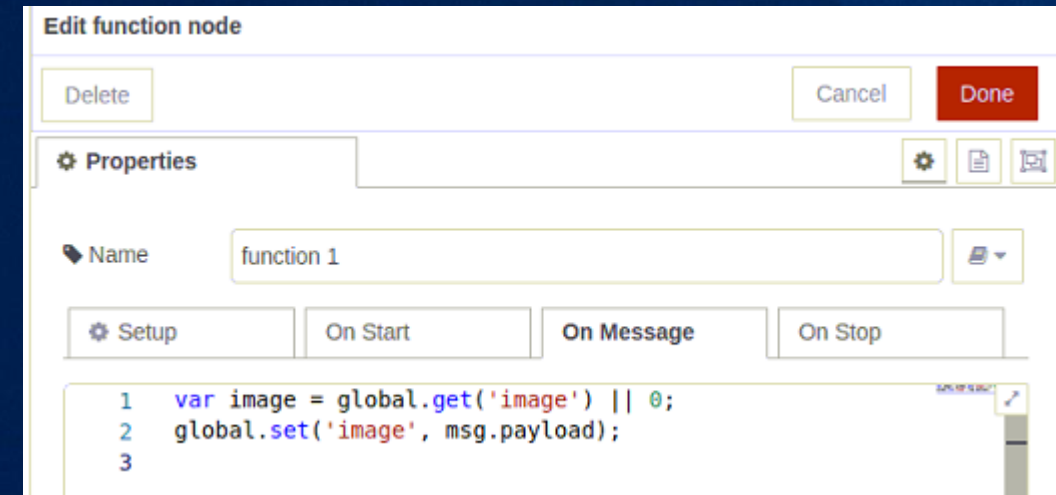
# Node-RED Visualization

Node-red flow 1  
Image flow



- A function node is used at the end of the flow to save the decoded image to a global variable "image". On message section, enter the script below:

```
var image = global.get('image') || 0;  
global.set('image', msg.payload);
```



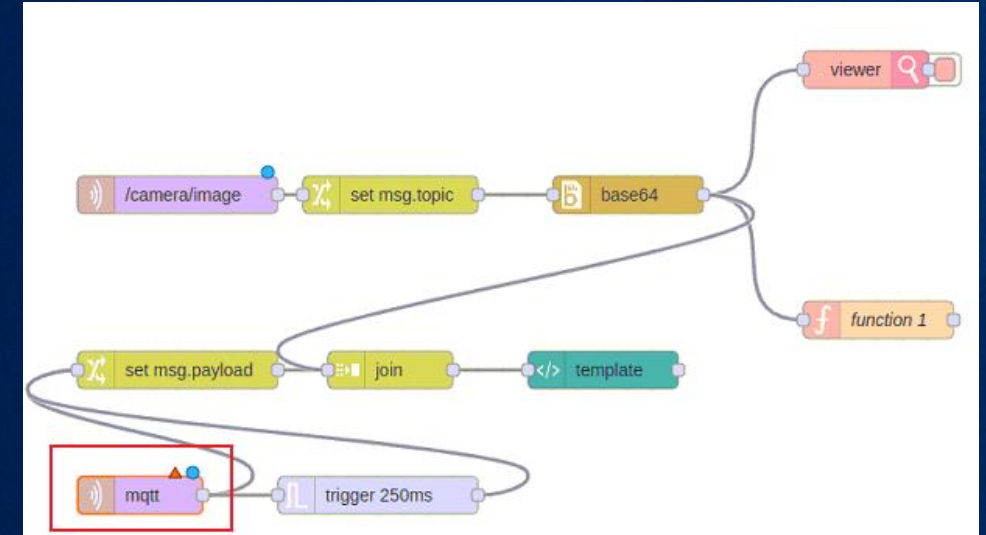
Edited function in On Message

# Node-RED Visualization

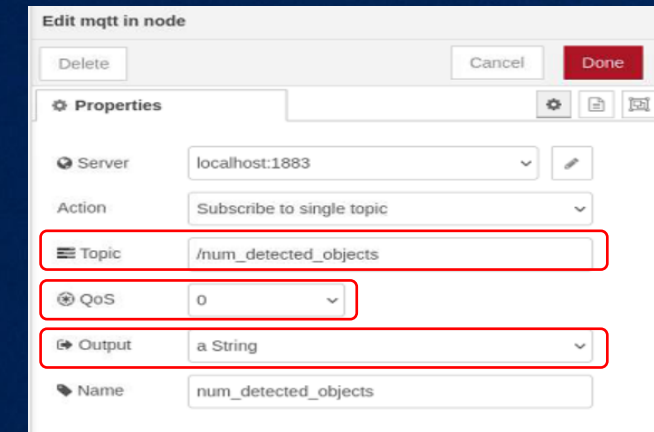
Node-red flow 1  
Image flow



- Made the following changes:
  - Change topic section to `'/num_detected_objects'`,
  - QoS set to `0`.
  - Change Output section to a *String*.
- Clicked on *Done* button after the settings are configured.



Edit on the 2<sup>nd</sup> mqtt node

A screenshot of the 'Edit mqtt in node' dialog box. The 'Properties' tab is selected. The 'Server' is set to 'localhost:1883'. The 'Action' is 'Subscribe to single topic'. The 'Topic' is '/num\_detected\_objects'. The 'QoS' is '0'. The 'Output' is 'a String'. The 'Name' is 'num\_detected\_objects'. Red boxes highlight the 'Topic', 'QoS', and 'Output' fields.

Configs for 2<sup>nd</sup> mqtt node



# Node-RED Visualization

Node-red flow 1  
Image flow



- A trigger node is added as no mqtt topic will be published when no images are detected in the current frame. Hence, the trigger node will reset the count to 0 whenever no messages are received on the topic.
- Edit the trigger node based on the settings in the red bounding boxes shown in the picture.
- Clicked on the Done button after the configs are modified.

Edit trigger node

Delete Cancel Done

⚙ Properties

Send ▼ nothing

then wait for ▼

1 Seconds ▼

☒ extend delay if new message arrives

☐ override delay with msg.delay

then send ▼ a<sub>z</sub> 0

☐ send second message to separate output

Reset the trigger if: 

- msg.reset is set
- msg.payload equals optional

Handling all messages ▼

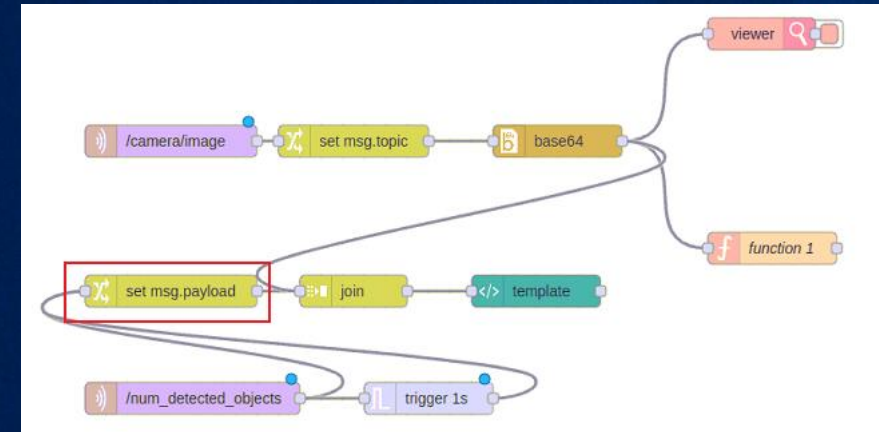
🔍 Name Name

# Node-RED Visualization

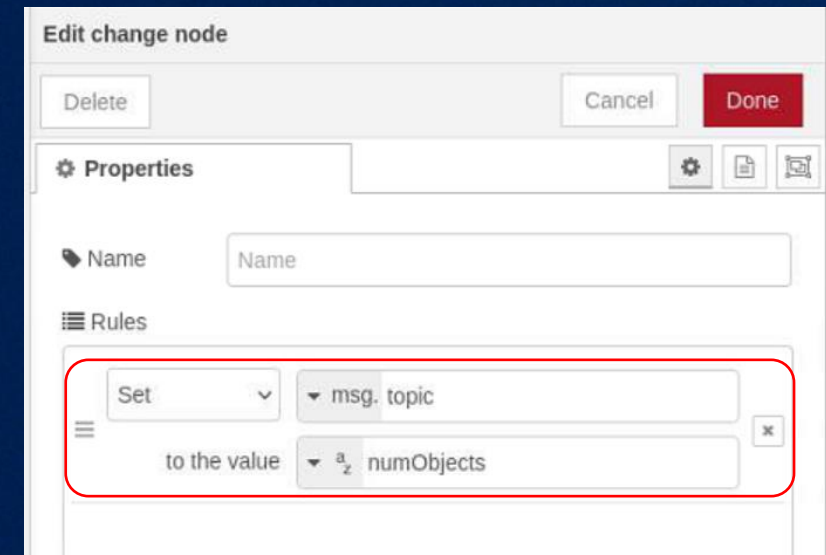
Node-red flow 1  
Image flow



- Made changes to the 2<sup>nd</sup> change node according to the red bounding box.
- Click on the Done button after modified the config



2<sup>nd</sup> change node



Configs for 2<sup>nd</sup> change node

# Node-RED Visualization

Node-red flow 1  
Image flow



- The join node will join the information of base64 string image data and the number of the object detected string data.
- On the join node, choose the mode to manual. Add number 1 to the section *After a number of message parts*.
- Click on the Done button after modifying the configs

Edit join node

Delete Cancel Done

⚙ Properties

Mode manual

Combine each msg. payload

to create a key/value Object

using the value of msg. topic as the key

Send the message:

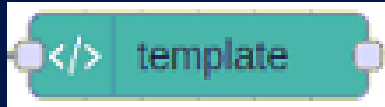
- After a number of message parts 1  
☒ and every subsequent message.
- After a timeout following the first message seconds
- After a message with the `msg.complete` property set

📌 Name Name

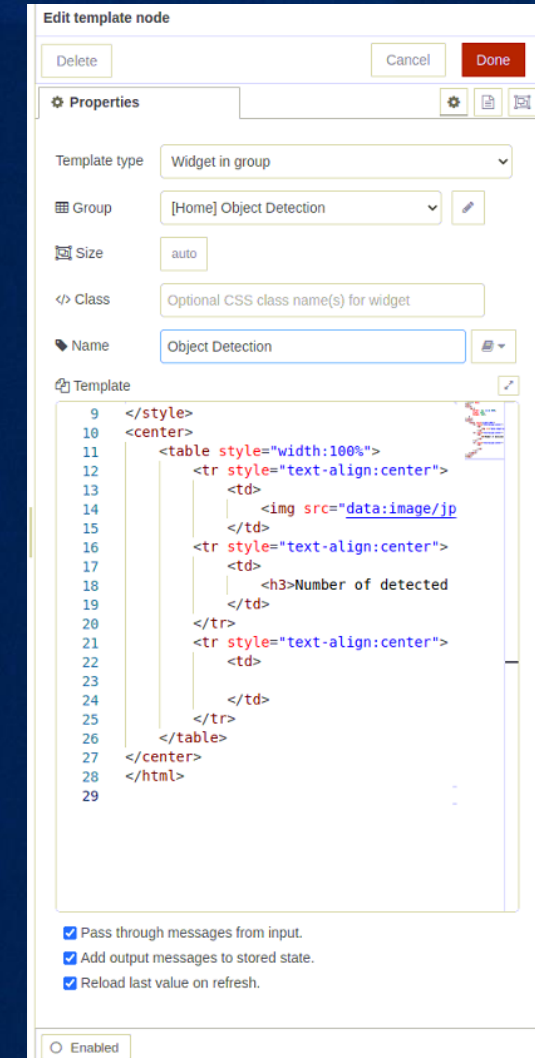


# Node-RED Visualization

Node-red flow 1  
Image flow

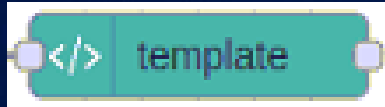


- The joined message is passed to a template node that will display the image frame and the number of detected objects on the dashboard.
- In the name section, change the name to *Object Detection*
- In the template section, input the HTML code that will generate the page for viewing the object detection stream, counter for the detected object, and save button. You can find the HTML code in the next slide.

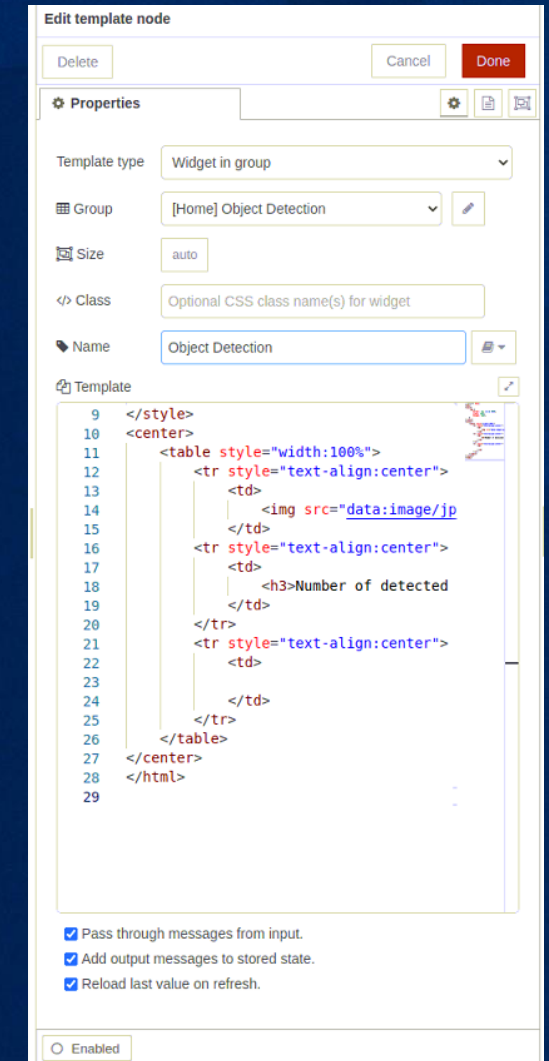


# Node-RED Visualization

Node-red flow 1  
Image flow



```
<!DOCTYPE html>
<html>
<style>
  .img {
    border: 8px solid #000;
    width: 100%;
    height: 100%;
  }
</style>
<center>
  <table style="width:100%">
    <tr style="text-align:center">
      <td>
        
      </td>
      <tr style="text-align:center">
        <td>
          <h3>Number of detected objects: {{msg.payload.numObjects}}
</h2>
        </td>
      </tr>
      <tr style="text-align:center">
        <td>
          </td>
        </tr>
      <tr style="text-align:center">
        <td>
          </td>
        </tr>
      </table>
</center>
</html>
```

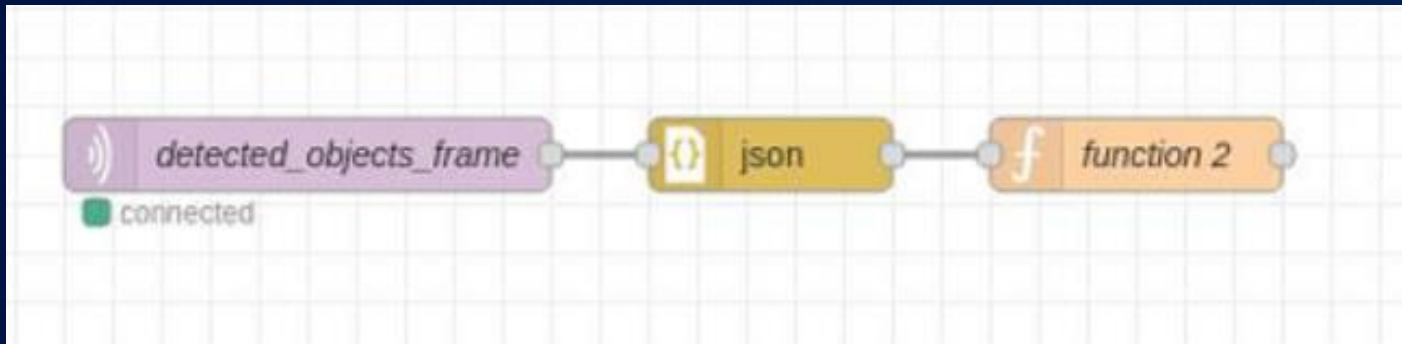


# Node-RED Visualization

Node-red flow 2

detected\_object\_frame flow

- Create a flow to save the cropped images of the detected objects to a global JSON object so that they can be saved by the button on the dashboard.

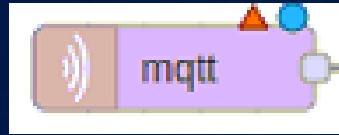


- Tips (Use the filter function to search for the nodes effectively)
  - detected\_object\_frame: *mqtt in*
  - json: *json*
  - function2: *function*



# Node-RED Visualization

Node-red flow 2  
detected\_object\_frame flow



- In this node, we will subscribe to the 'detected\_objects\_frame' mqtt topic and output the received data as a string.
- Made the following changes:
  - input topic to '/detected\_objects\_frame'
  - modify output to 'a String'

Edit mqtt in node

Delete Cancel Done

⚙ Properties

📶 Server localhost:1883

Action Subscribe to single topic

📄 Topic /detected\_objects\_frame

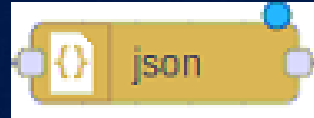
📶 QoS 2

➡ Output a String

🏷 Name detected\_objects\_frame

# Node-RED Visualization

Node-red flow 2  
detected\_object\_frame flow



- The function of the node is to convert the received msg.payload to a JSON object.
- No changes are required for this node

**Edit json node**

Delete Cancel Done

⚙ Properties

⦿ Action Convert between JSON String & Object ▼

⋮ Property msg. payload

👤 Name Name

---

Object to JSON options

☐ Format JSON string

# Node-RED Visualization

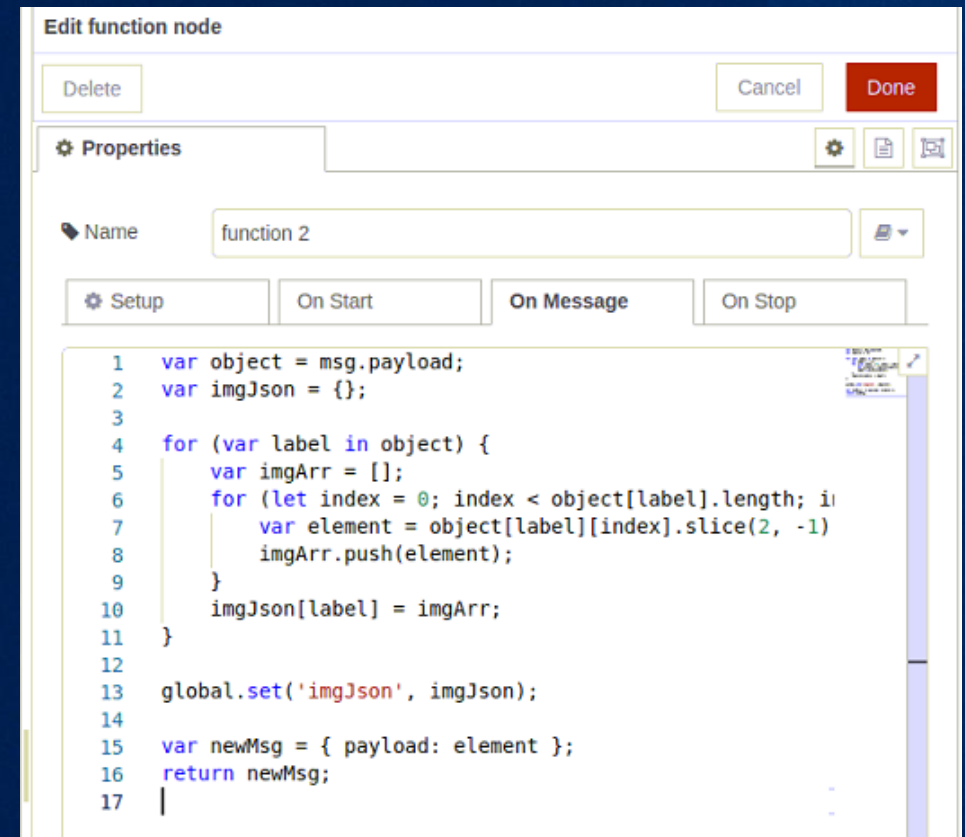
- A function node is used to iterate through the JSON object and save the images with the labels to a global JSON object. Add the following function code to the On Message function.

```
var object = msg.payload;
var imgJson = {};

for (var label in object){
    var imgArr = [];
    for (let index = 0; index < object[label].length;
index++) {
        var element = object[label][index].slice(2,-1);
        imgArr.push(element);
    }
    imgJson[label] = imgArr;
}

global.set('imgJson', imgJson);

var newMsg = {payload: element};
return newMsg;
```



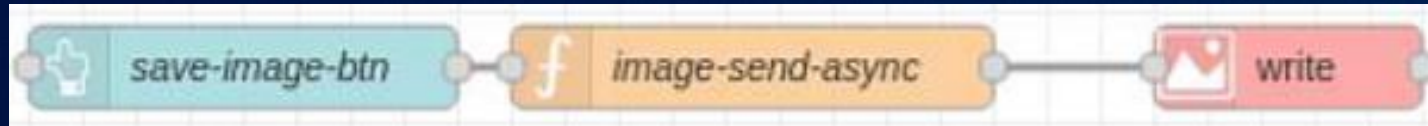


# Node-RED Visualization

Node-red flow 3  
save-image-btn flow



- Create a new flow that will save the cropped images of the detected objects which are retrieved from the global JSON object.



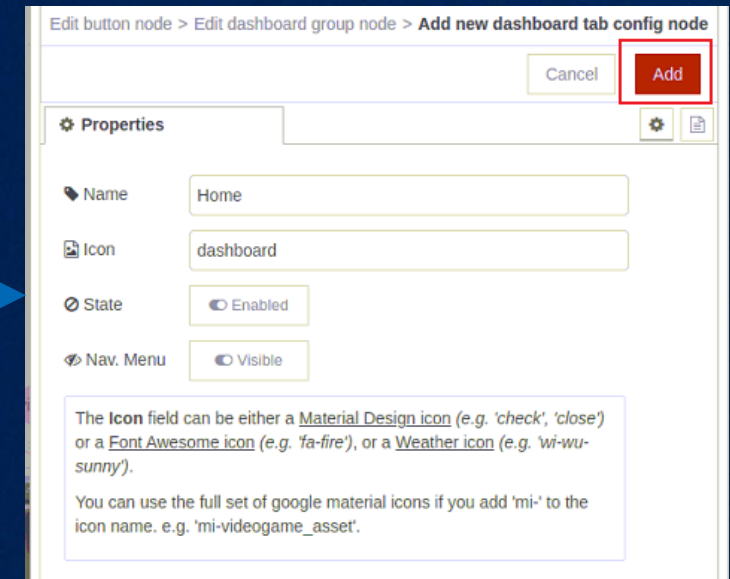
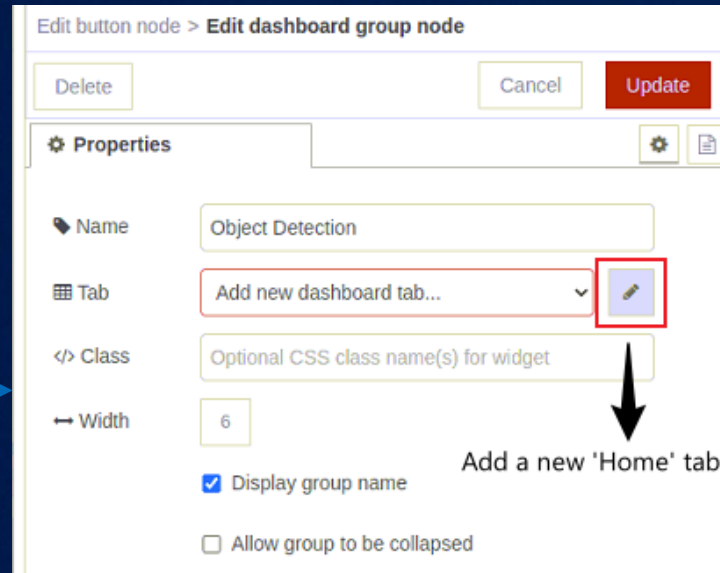
- Tips (Use the filter function to search for the nodes effectively)
  - save-image-bin: *button*
  - image-send-async: *function*
  - write: *image*

# Node-RED Visualization

Node-red flow 3  
save-image-btn flow



- Click on button node and enter the dashboard group node by clicking the edit button next to "Group".
- This group will be your dashboard Tabs. Enter 'Object Detection' as your Home tab.
- Update the node.



# Node-RED Visualization

Node-red flow 3  
save-image-btn flow



- After the modification is done, click update to update the button node.

Edit button node > Edit dashboard group node

Delete Cancel Update

⚙ Properties

📌 Name Object Detection

📄 Tab Home

</> Class Optional CSS class name(s) for widget

↔ Width 6

☒ Display group name

☐ Allow group to be collapsed

Settings for button node

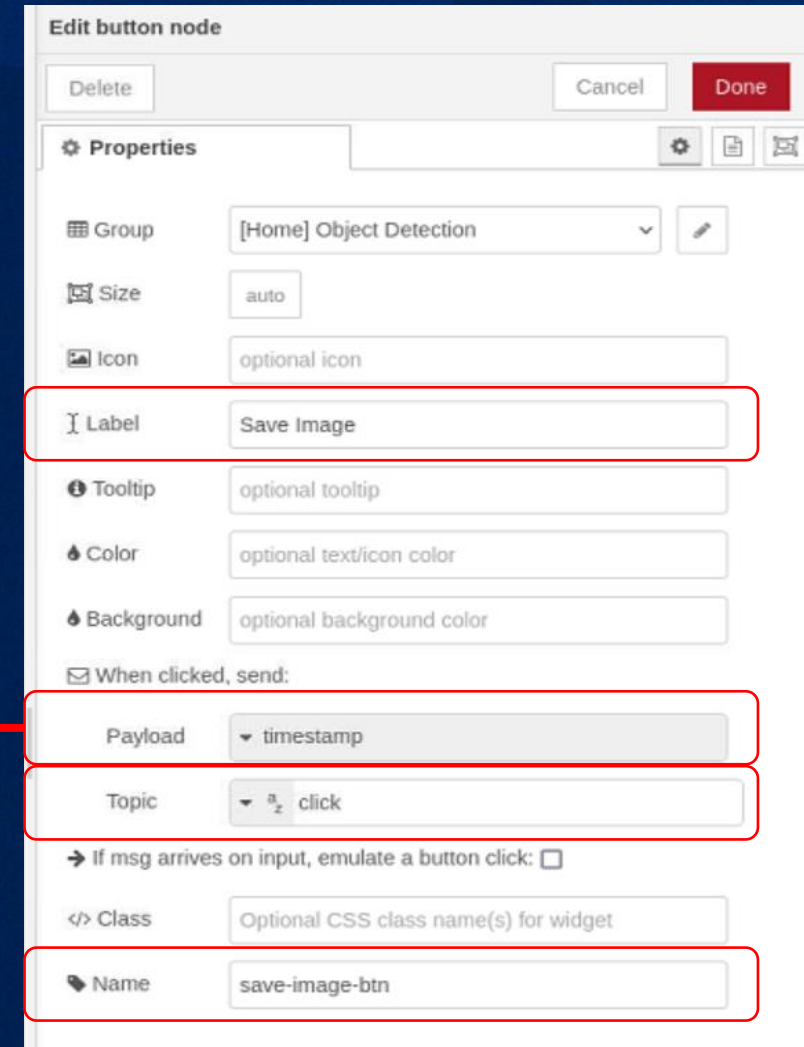
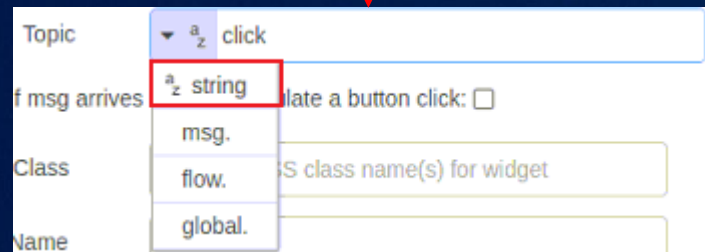


# Node-RED Visualization

Node-red flow 3  
save-image-btn flow

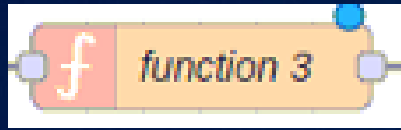


- Made the following changes
  - Label the button as "Save Image"
  - In the *when clicked send* section, change the payload to timestamp
  - In the *when clicked send* section, change the topic to string and put the value with *click*
  - In Name, set the value to *save-image-btn*
- Click on the Done button after all the changes are mode



# Node-RED Visualization

Node-red flow 3  
save-image-btn flow



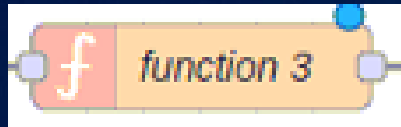
- Create a function that iterates through the global JSON object and each label images array contained in the object, and asynchronously sends the base64 string to a write image node. The filename of the image is also inserted as part of the output message and depends on the number of each detected objects in the current frame. The function is as below:

```
var imgJson = global.get('imgJson') || 0;

for (var label in imgJson){
  var imgArr = imgJson[label];
  for (let idx = 0; idx < imgArr.length; idx++) {
    var msg = { payload: imgArr[idx], filename: "./images/" + label + "_" +
idx.toString() + '.png'};
    node.send(msg);
  }
}
```

# Node-RED Visualization

Node-red flow 3  
save-image-btn flow



- Expected node function in On Message

Edit function node

Delete Cancel Done

Properties

Name function 3

Setup On Start On Message On Stop

```
1 var imgJson = global.get('imgJson') || 0;
2 node.warn("my warning");
3 node.warn("img: " + imgJson);
4
5 for (var label in imgJson){
6   var imgArr = imgJson[label];
7   for (let idx = 0; idx < imgArr.length; idx++) {
8     var msg = { payload: imgArr[idx], filename: "./images/" + label + "_" + idx.toString() + '.png'};
9     node.send(msg);
10  }
11 }
12
```



# Node-RED Visualization

Node-red flow 3  
save-image-btn flow



- Create an image node and configure it as shown in the picture.
- The red bounding boxes are the section that required changes.
- After changing these sections, click on the Done button.

Edit image node

Delete Cancel Done

⚙ Properties

Name write

image msg. payload  
A string containing a file path, URL or base64 image can be used as an image source. NOTE: Passing in an image object is faster as there is no conversion required before processing.

🔧 Function write  
Write to file. NOTE: You can specify an alternative file extension type to change the type. Currently support types are jpg, png, bmp.

➡ Output image  
Sending an image is much faster as there is no additional conversion required.

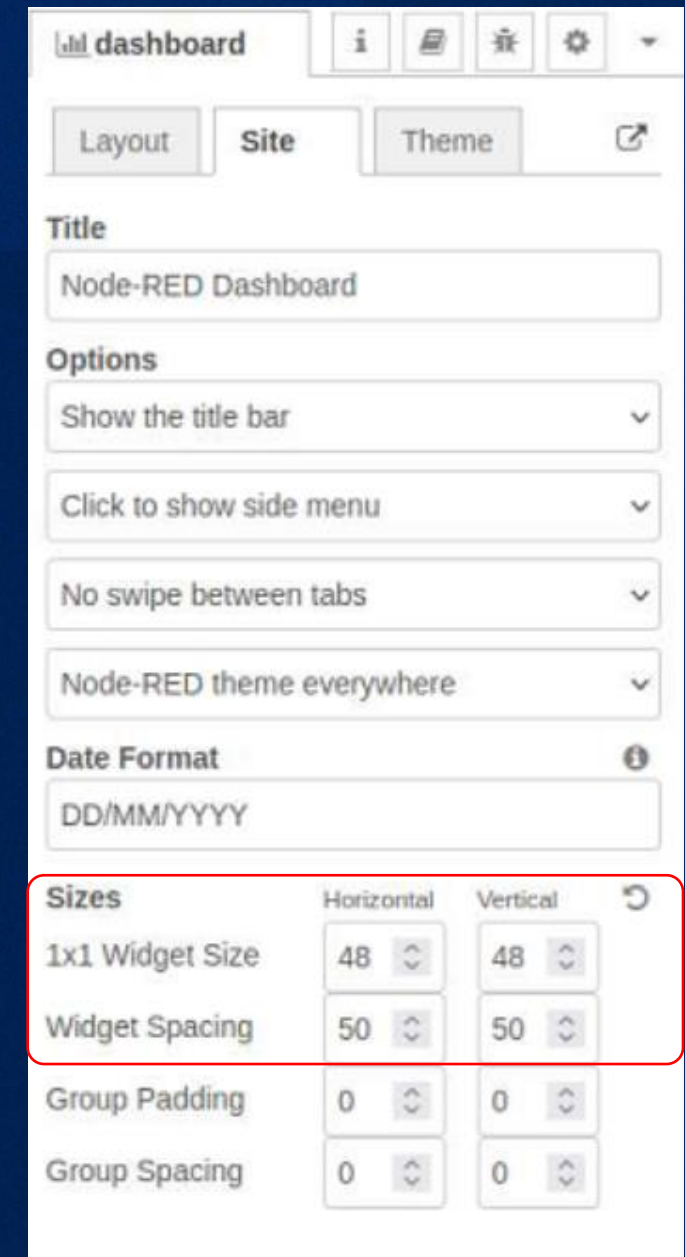
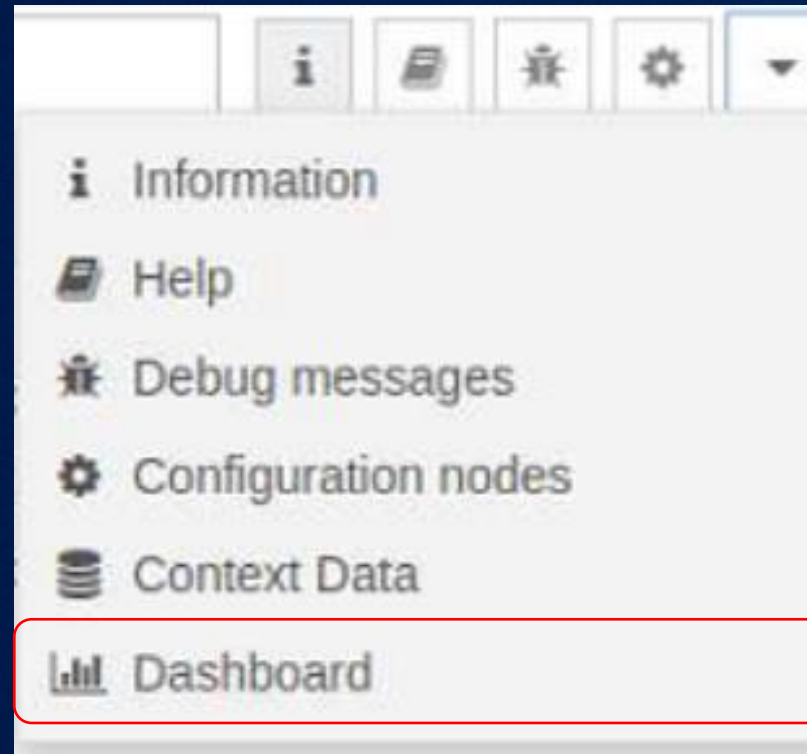
➡ Output Property msg. payload  
The msg property in which to send the resulting image.

</> filename msg. filename  
(Required) Name of the file

# Node-RED Visualization

## Running Dashboard

- You may change the size of your dashboard under the site section of the dashboard.



# Node-RED Visualization

## Running Dashboard

- The Tabs and Links group should be organised as shown below. You may reposition the group tab by dragging it. On the right side of the page, click deploy.

After successfully deploying, go to the dashboard on the right side of the screen and click the button.

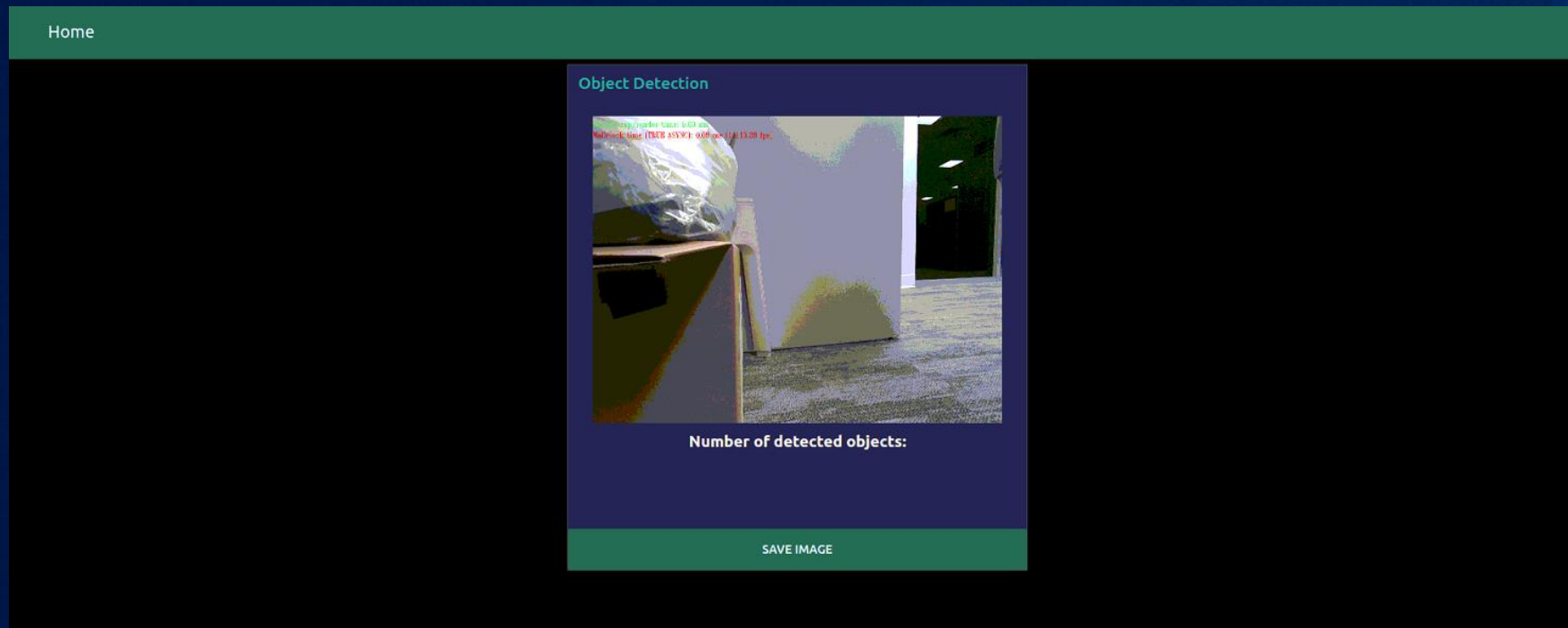




# Node-RED Visualization

## Running Dashboard

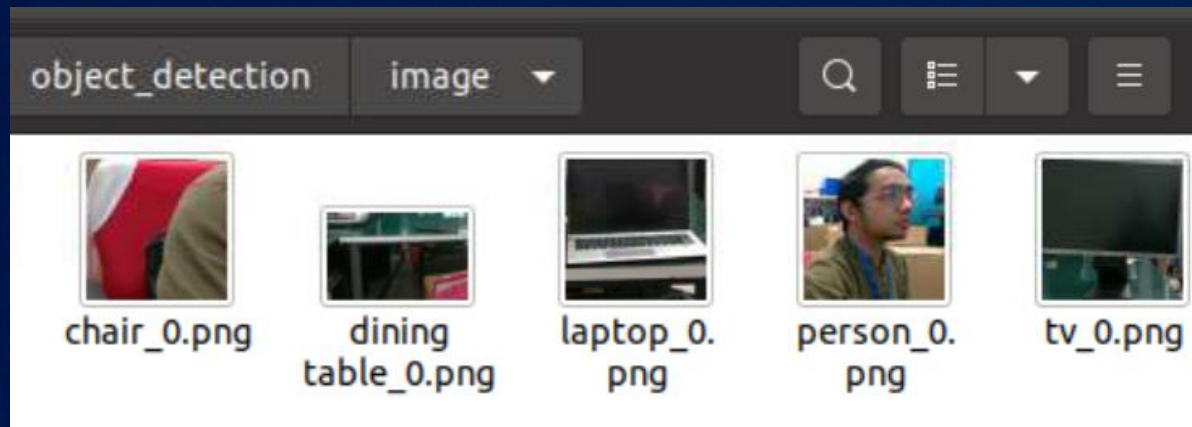
- The dashboard should look like the screenshot below if it is set up properly.



# Node-RED Visualization

## Running Dashboard

- Locate your saved picture in `~/workspace/object_detection/image/` after clicking the "Save Image" button.



# Link

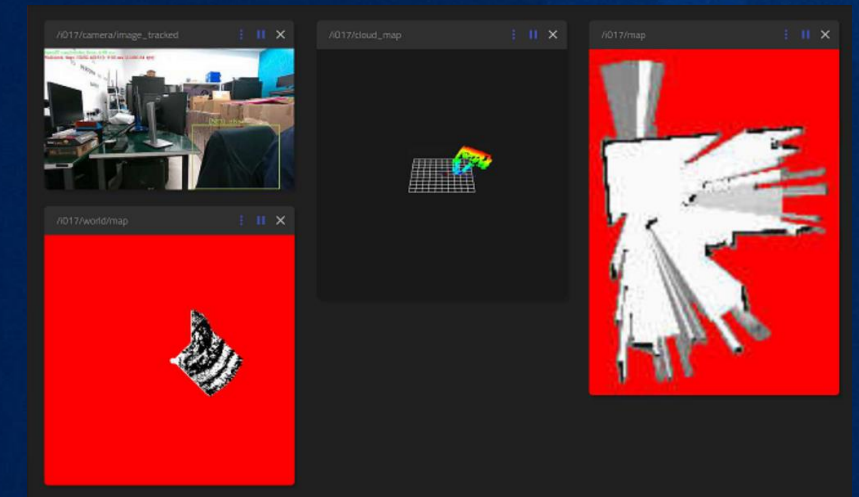
1. Node-RED: <https://nodered.org/docs/>



# ROSboard

# What is ROSBoard?

- A third-party open source ROS2 visualization program.
- It contains a server that will create a ROS2 subscriber to subscribe to the ROS2 topic publish by the AMR.
- User can access the UI to visualize the data captured on the AMR.
- Why visualization?
  - **Debugging and troubleshooting:**
    - Visualization is an essential tool in robotics for debugging and troubleshooting. Visualizing robot movements, sensor data, and environment can help identify issues and errors, allowing engineers to improve the robot's performance.
  - **Planning and simulation**
    - Visualization is crucial in planning and simulating robotic applications. Before building a physical robot, engineers can use visualization tools to test and optimize robot design, programming, and control
  - **Human-Robot Interaction**
    - Visualization is also necessary for human-robot interaction. In complex robotic applications, visualizing the robot's actions and intentions can help humans understand and interact with the robot more effectively.



Sensor data visualization in ROSBoard



# ROSboard

ROSboard is a ROS node that runs a web server on your robot and streams live visuals to <https://localhost:8888/>. It can display all the output from the topic that you run without the need for any additional software such as rviz.

- Ensure that you already exported the ROS\_DOMAIN\_ID to your environment.
- Install the necessary python packages.

```
$ sudo pip3 install tornado  
$ sudo pip3 install simplejpeg
```



# ROSboard

- Clone the ROSboard file inside your workspace.

```
$ cd ~/workspace  
$ git clone https://github.com/dheera/rosboard.git
```

- Source ROS folder.

```
$ source /opt/ros/humble/setup.bash
```

- Build the ROS package for ROSBoard.

```
$ cd ~/workspace/rosboard  
$ colcon build  
$ source install/setup.bash
```

# ROSboard

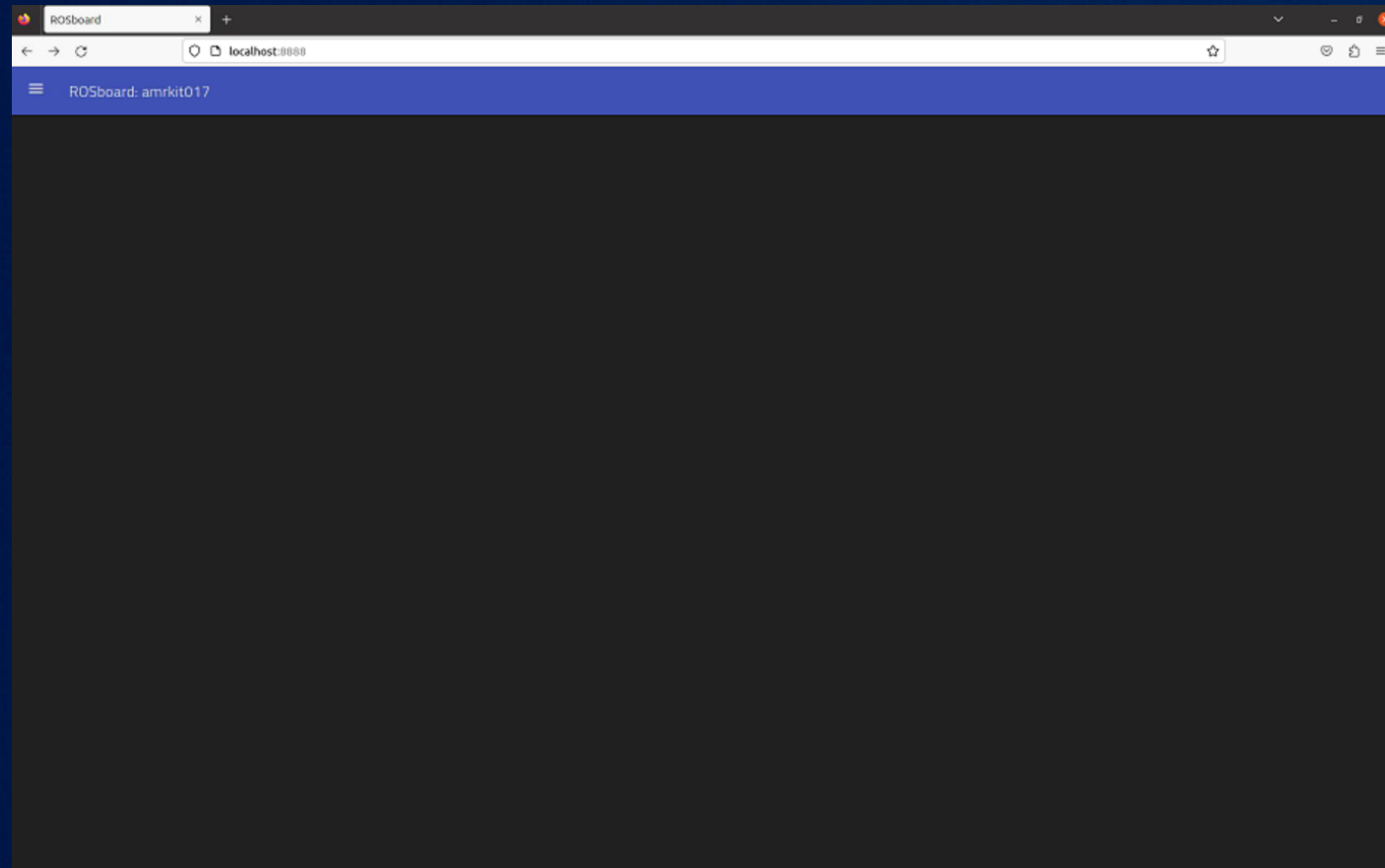
- Run ROSBoard

```
$ export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp  
$ export ROS_DOMAIN_ID=<ROBOT_DOMAIN_ID>  
$ ros2 run rosboard rosboard_node
```

- Ensure that all the previous topics were run. (Mapping, Object Detection, Rtabmap, Fast Mapping).

# ROSboard

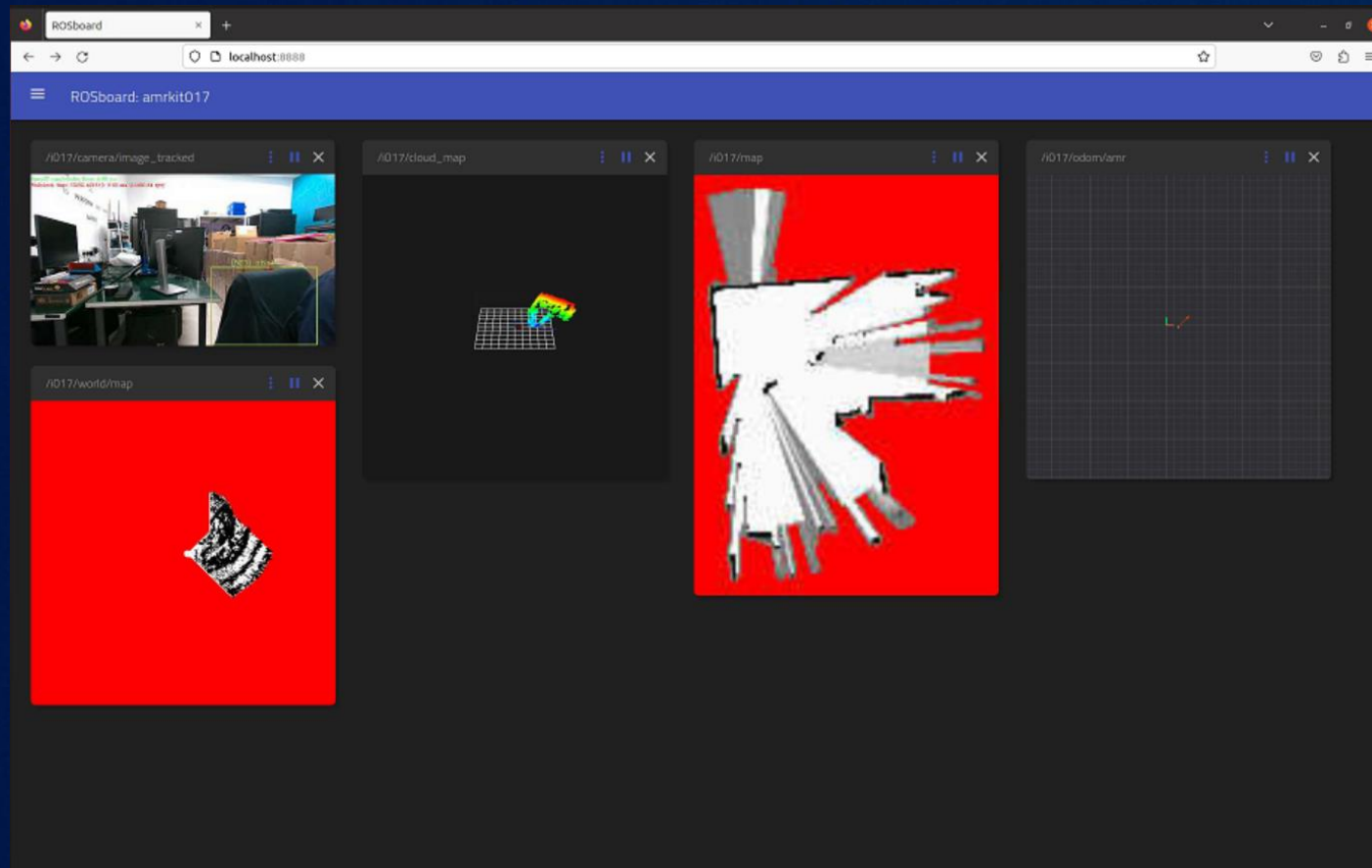
- Point your web browser at `http://localhost:8888` (replace localhost with your robot's IP)





# ROSboard

- By clicking the three bars on the left side, you may get visualizations of your topics. Select the topic you wish to display.



# Link

1. ROSboard: <https://github.com/dheera/rosboard>

# End